

SheDrive

High-Level Design Document

Prepared by

ARCorp

Prepared for

SheDrive Team

Version: **1.0. Draft**

Date: **April 2026**

Classification: **Confidential**

Table of Contents

- 1. Document Control 5
 - 1.1 Change Record 5
 - 1.2 Reviewers..... 7
 - 1.3 Document Conventions 7
- 2. Introduction 9
 - 2.1 Context..... 9
 - 2.2 Key Stakeholders 9
 - 2.2.1 Client-Side Stakeholders (SheDrive Team) 9
 - 2.2.2 Delivery-Side Stakeholders (ARCorp) 10
 - 2.3 Document Organisation 10
 - 2.3.1 Intended Audience 10
 - 2.3.2 Document Structure 11
- 3. Solution Needs..... 12
 - 3.1 Architectural Principles 12
 - 3.2 Architectural Techniques Leveraged..... 14
 - 3.3 Architectural Constraints..... 16
 - 3.4 Business View 18
 - 3.4.1 Personas and Main Scenarios 18
 - 3.4.2 Deliverables and Solution Elements..... 20
 - 3.4.3 Out of Scope 24
 - 3.5 Quality Attribute View 25
 - 3.5.1 User Experience and Visual Design 25
 - 3.5.2 Performance 26
 - 3.5.3 Scalability..... 26
 - 3.5.4 Availability and Reliability 26
 - 3.5.5 Security..... 27
 - 3.5.6 Maintainability and Evolvability..... 27
 - 3.5.7 Operability and Monitoring 27
 - 3.5.8 Auditability and Compliance 28
 - 3.5.9 Interoperability 28
 - 3.5.10 Testability..... 29
- 4. Solution Architecture 30
 - 4.1 System Context View 30
 - 4.2 Functional Blocks 33
 - 4.2.1 Rider App..... 35
 - 4.2.2 Driver App..... 37
 - 4.2.3 Admin & Operations Portal..... 39
 - 4.2.4 Trip & Dispatch Service..... 41

4.2.5 Identity & Verification Service.....	45
4.2.6 Payment Service.....	46
4.2.7 Safety & SOS Service	48
4.2.8 Pricing & Zones Service.....	50
4.2.9 User Management Service.....	51
4.2.10 Notification Service	52
4.2.11 Reporting & Analytics Service	52
4.2.12 Maps & Location Service.....	53
4.2.14 Platform / Support Capabilities	56
4.2.14.1 API Gateway & Communication Layer	57
4.2.14.2 Background Jobs & Scheduling (Hangfire)	58
4.2.14.3 Audit Trail.....	59
4.2.14.4 Logging & Tracing	62
4.2.14.5 Health Check & Service Status	64
4.2.14.6 Real-Time Hub (SignalR)	65
4.3 Build View.....	66
4.3.1 Implementation Patterns	66
4.3.2 Technology Stack	72
4.3.3 Team Responsibilities	75
4.4 Information View.....	77
4.4.1 Major Business Entities.....	77
4.4.2 Data Ownership Matrix.....	78
4.4.3 Data Classification	80
4.4.4 Data Lifecycle and Retention	82
4.4.5 Data Residency.....	83
4.5 Security View.....	84
4.5.1 Authentication and Authorisation Flow	84
4.5.2 Role-Based Access Control Matrix.....	85
4.5.3 Encryption, Network, and Data Security Controls.....	86
4.6 Integration View.....	87
4.6.1 Paymob. Payment Service Provider.....	88
4.6.2 Google Maps Platform. Maps & Location Service	89
4.6.3 Firebase Cloud Messaging (FCM) / Apple Push Notification Service (APNS).....	90
4.6.4 Local Egyptian SMS Gateway	90
5. Deployment View	91
5.1 Environments	91
5.2 Host Configuration.....	93
5.2.1 Hosting Configuration Standards.....	94
5.2.2 Availability and Resilience Configuration.....	95
5.3 Mobile Application Rollout Plan	96
6. Operation View	97

6.1 DevOps Enablement and Operational Tooling	97
6.1.1 CI/CD Pipeline Strategy	97
6.1.2 Source Control and Branching Strategy	99
6.1.3 Monitoring, Observability, and Alerting	99
6.1.4 Automated Quality Gates	101
6.2 Inner Development Loop and Engineering Practices	101
6.3 Feedback Loop Between Operations and Development	103
7. Hardware Sizing.....	104
7.1 Assumptions.....	104
7.2 Production Environment	105
7.2.1 Network and Communication Prerequisites.....	107
7.3 Pre-Production Environment (UAT / Pre-PROD)	108
7.4 System Integration Testing (SIT) Environment	108
7.5 Scaling Roadmap and Growth Triggers	108
8. Risk View	110
8.1 ARCorp Enterprise Risk Management Approach.....	110
8.2 Risk Management Framework.....	111
8.3 Risk Classification Matrix.....	111
8.4 Risk Register	112
9. Compliance View	112
10. Appendices	114
10.1 Glossary of Key Terms	114
10.2 Use of Open Source	118
10.3 Licenses and Subscriptions	120
10.4 Technologies and Techniques Used.....	122
10.5 Code Generator: Entity Descriptor Schema.....	130
10.6 Acronyms and Abbreviations	135

1. Document Control

1.1 Change Record

Version	Date	Author	Role	Description
1.0	April 2026	Mohamed Talaat	Engineering Manager	Initial HLD Draft
1.1	June 2026	Mohamed Talaat	Engineering Manager	Client review updates: manual human-led KYC (no eKYC, no external verification/background-check providers), first-trip rider verification, removal of external legal constraints, AWS managed-servers-only hosting (EC2 + S3) with self-hosted stack (Vault, Prometheus/Grafana), Hangfire background processing, restructured Risk View with external Risk Register, reformatted data lifecycle/residency tables, split licenses tables, and added acronyms appendix.
1.2	June 2026	Mohamed Talaat	Engineering Manager	Second client-review update: child passenger age < 15, concurrency model (7.1), multi-gateway route design (4.2.14.1), expanded Audit Trail / Logging / Health Check sections, implementation patterns split by tier (backend / mobile / admin) with Saga and Circuit Breaker rationale, Materialised View pattern, mobile rollout plan (5.3), business-oriented scaling roadmap (7.5), removed IaC section (5.2.3), simplified 7.2 specs table, 8.4 risk register reference only, SQL Server → Enterprise Edition, removed Azure DevOps and AWS rows from licences, all VMs Windows Server only, removed specific TLS provider names.
1.3	June 2026	Mohamed Talaat	Engineering Manager	Third update: split API Gateway into three independent Ocelot instances (Rider Gateway, Driver Gateway, Admin Gateway) co-hosted on the same

Version	Date	Author	Role	Description
				Presentation & API Gateway VMs, replacing the single-instance multi-route design; removed Ocelot WebSocket proxy and introduced a dedicated ASP.NET Core SignalR Real-Time Hub (new section 4.2.14.6) as the sole real-time channel for GPS streaming and SOS alert fan-out; adopted Jason Taylor Clean Architecture template as the per-service project standard and expanded Clean Architecture and Selective CQRS patterns with MediatR pipeline detail; added internal CQRS code generator (Scriban CLI) as a new development tooling subsection; added entity descriptor schema as a new appendix; updated Technology Stack, Open Source, Licences, Scaling Roadmap, Container View, and Ocelot technology description accordingly.
1.4	June 2026	Mohamed Talaat	Engineering Manager	Fourth update: added a Table of Contents page following the cover page; corrected a stray blank page preceding Section 3 (Solution Needs); added Rate Limiting as a new architectural technique (3.2), enforced per-client at the Ocelot gateway layer; renamed the former “Appendix A” to numbered Section 10.5, Code Generator: Entity Descriptor Schema, added a worked trip.yaml example, and renumbered the Acronyms and Abbreviations section from 10.5 to 10.6 accordingly.
1.5	June 2026	Mohamed Talaat	Engineering Manager	Fifth update: clarified data store responsibilities (MongoDB owns live driver location with TTL; SQL Server is the master for zone polygon configuration; Redis caches both at runtime); added Client Interaction Model to Maps & Location Service (4.2.12) distinguishing mobile SDK usage from back-office polygon editor and explaining that Google Maps Platform

Version	Date	Author	Role	Description
				supplies routing inputs only — fare calculation is performed internally by the Pricing & Zones Service; removed KYC Module as a standalone section (former 4.2.14) and merged its verification workflow, tooling responsibilities, and audit dependencies into Driver & Vehicle Registration Service (4.2.13), with first-trip rider verification and gender-mismatch handling retained under Identity & Verification Service (4.2.5); renumbered Platform / Support Capabilities from 4.2.15 to 4.2.14 and updated all sub-section references accordingly.

1.2 Reviewers

Name	Role	Organization	Review Scope
[TBD]	Business Sponsor	SheDrive Team	Full document
[TBD]	Product Owner	SheDrive Team	Business View & Scope
Mohamed Talaat	Head of Engineering	ARCorp	Architecture & Build View

1.3 Document Conventions

The following conventions are applied throughout this document:

- Shall / Must, denotes a mandatory architectural decision or design constraint.
- Should, denotes a recommended approach subject to review and confirmation.
- May / Could, denotes an optional capability or future-phase consideration.
- MVP, refers exclusively to the scope defined in the approved Business Requirements Document (BRD v1.1).
- Out-of-Scope, refers to items explicitly excluded from MVP delivery per BRD Section 5.2.
- All monetary values are expressed in Egyptian Pounds (EGP) unless stated otherwise.
- All times are in Egyptian Eastern Time (EET / EEST, UTC+2 / UTC+3).
- Actor names (Rider, Driver, Administrator, etc.) are capitalised when referring to defined system roles, consistent with the BRD.
- Section references use the format X.Y (e.g., 4.2).

2. Introduction

This High-Level Design (HLD) document presents the proposed technical architecture for the SheDrive platform, a women-only ride-hailing service purpose-built for the Egyptian market. It defines the architectural structure of the system, including its major components, integration boundaries, deployment model, and operational considerations.

The document translates the approved Business Requirements Document (BRD v1.1, authored by the ARCorp team, January–April 2026) into a coherent architectural view that supports scalability, security, auditability, and future extensibility. It serves as the primary technical reference for all stakeholders involved in design review, implementation, deployment, and operation of the SheDrive MVP.

2.1 Context

SheDrive is a women-only ride-hailing platform designed to address a well-defined gap in the Egyptian mobility market: the absence of a dedicated, safety-first, gender-verified transport service for female riders and female drivers in Greater Cairo (Cairo and Giza). The platform serves female riders, including those travelling with child passengers under 15, and female drivers operating their own vehicles, within a daytime-only operating window at MVP launch.

The business context, problem statement, and functional scope are fully documented in the BRD. This HLD does not restate those requirements but translates them into architectural building blocks, integration patterns, deployment topology, and operational decisions. Where architectural choices require clarification of a business rule, the relevant BRD section is referenced.

The MVP is structured around ten business modules. Rider App, Driver App, Admin & Operations Portal, Identity & Verification, Payments (Cash + Digital), Trip & Dispatch Engine, Pricing & Neighborhood Zones, Safety & SOS, Notifications, and Reporting & Analytics, delivered across three primary surfaces: a native cross-platform Rider mobile application, a native cross-platform Driver mobile application, and a web-based Administration & Operations portal.

Client: SheDrive Team

Implementing Organisation: ARCorp

Document Owner: ARCorp. Solution Architect

Reference BRD: SheDrive Business Requirements Document, Version 1.1, April 2026

2.2 Key Stakeholders

This document is produced as a key deliverable of the SheDrive MVP implementation project and is to be reviewed and approved by representatives of both the SheDrive Team and ARCorp. The Solution Needs section, particularly the Business View subsection, is the primary point of interest for business-side stakeholders, while the Solution Architecture and Deployment View sections are intended for the engineering and delivery community.

2.2.1 Client-Side Stakeholders (SheDrive Team)

Name / Role	Organization	Interest / Responsibility
Business Sponsor / Executive Leadership	SheDrive Team	Funding, strategic alignment, final approval

Name / Role	Organization	Interest / Responsibility
Product Owner	SheDrive Team	Scope, priorities, ongoing requirement decisions
Operations Management	SheDrive Team	Daily platform operations, incident handling, service continuity
Customer Support Team	SheDrive Team	User inquiries, complaints, incident follow-ups
Finance & Accounting	SheDrive Team	Payment reconciliation, cash settlement oversight, financial reporting
Compliance & Legal	SheDrive Team	Regulatory adherence, data protection, policy review
Marketing & Growth	SheDrive Team	User acquisition, brand positioning, launch communications

2.2.2 Delivery-Side Stakeholders (ARCorp)

Name / Role	Organization	Responsibility
Solution Architect (HLD Owner)	ARCorp	Architecture design, technical decision authority, HLD authorship
Technical Leads	ARCorp	Module-level design, implementation oversight, code quality
DevOps / Release Engineers	ARCorp	CI/CD pipeline, infrastructure-as-code, environment management
Quality Control & QA Team	ARCorp	Test planning, acceptance criteria validation, defect management
Operations & Support Team	ARCorp	Post-go-live monitoring, incident response, SLA compliance

These stakeholders collectively contribute to defining requirements, approving architectural decisions, implementing the solution, and operating the system in production.

2.3 Document Organisation

2.3.1 Intended Audience

This document is structured to serve a mixed audience of business decision-makers and technical practitioners. The table below identifies the primary audience categories and their relevance to the content of this document.

Audience	Category	Relevance to this Document
Business Sponsor & Executive Leadership	Business	Strategic alignment, investment decisions, scope approval
Product Owner	Business	Validation of business-to-architecture traceability
Operations & Compliance Teams	Operational	Confirmation that operational and regulatory expectations are addressed
Project Management	Delivery	Planning, scheduling, stakeholder coordination
Engineering & Solution Architects	Technical	Primary consumers, design, build, integration decisions
Quality Assurance & Acceptance Teams	Technical	Basis for architecture-level acceptance criteria
External Reviewers	Advisory	Legal, financial, or regulatory review where applicable

2.3.2 Document Structure

This document is structured to present the architecture of SheDrive in a logical and progressive manner, from business context through to deployment and operational considerations. Each section builds on the preceding one to maintain full traceability from business need to architectural decision.

Section	Content Summary
1. Document Control	Versioning, authorship, reviewers, and document conventions.
2. Introduction	Business context, key stakeholders, document audience, and document structure.
3. Solution Needs	Architectural principles, techniques, constraints, business view (personas, deliverables, out-of-scope), and quality attribute targets.
4. Solution Architecture	System context view, functional block decomposition (all ten modules), build view (implementation pattern, technology stack, team responsibilities).
5. Deployment View	Target environments, host configuration, and environment lifecycle.
6. Operation View	DevOps enablement, CI/CD strategy, monitoring, logging, and the feedback loop between operations and development.
7. Hardware Sizing	Sizing assumptions, production environment specifications, and pre-production environment guidance.
8. Risk View	Risk management approach, framework, classification matrix, and the risk register reference.
9. Compliance View	Traceability from platform policies and adopted industry standards to the architectural components that enforce them.
10. Appendices	Glossary of key terms, open-source usage, licenses and subscriptions, technology descriptions, and acronyms.

3. Solution Needs

This section defines the Solution Needs of the SheDrive platform and considers architectural principles, architectural techniques, architectural constraints, a business view of the solution, and the quality attribute targets that the architecture must satisfy.

3.1 Architectural Principles

Architectural Principles are long-lasting, rarely changing statements and guidelines that govern all design, implementation, and operational decisions for the SheDrive platform. They are informed by the nature of the business domain, the BRD objectives, the mobile-first character of the product, and the safety-critical positioning of the service. The table below details each principle together with its rationale and the concrete implications it places on the architecture.

Principle	Rationale / Source	Implications
Microservices Architecture	The BRD defines ten distinct business modules with clearly separated data ownership, lifecycles, and team responsibilities. A microservices decomposition is the natural architectural expression of this domain structure, directly supporting independent deployability, fault isolation, and targeted scaling.	Each of the ten business modules is realised as an independently deployable service. Services own their data and expose versioned API contracts. A change to the Payment Service does not require redeployment of the Trip & Dispatch Service. Individual services can be scaled without touching others.
Module-Aligned Service Boundaries	SheDrive manages multiple distinct business areas including identity, rides, payments, safety, and pricing, each with its own rules and invariants. Aligning every service boundary to exactly one business module keeps business rules close to the data they govern without imposing the full ceremony of formal Domain-Driven Design, which is intentionally not adopted for Release 1.	Each microservice owns one business module and its data. Cross-module interactions are explicit and mediated through published API contracts or integration events; never through direct data coupling. Core entities (Trip, Payment, Driver, Rider) encapsulate their own rules. Formal DDD practices (aggregates, ubiquitous-language workshops, event storming) may be introduced selectively in later phases if domain complexity grows.
Clean Architecture	SheDrive is a safety-critical and long-lived platform. Clean Architecture ensures that core business logic remains stable and testable regardless of changes in external frameworks, databases, or third-party integrations.	Each service is structured into Domain, Application, and Infrastructure layers with strict inward dependency direction. The Domain layer has no dependency on Infrastructure. Business rules are unit-testable without databases or external services. Infrastructure adapters, for Paymob, Google Maps, and the SMS gateway, are replaceable without touching business logic.

Principle	Rationale / Source	Implications
Event-Driven Architecture	Ride matching, payment events, SOS alerts, notification dispatch, and driver onboarding outcomes are naturally asynchronous and span multiple services. Event-driven communication decouples services, improves resilience, and allows independent evolution.	Domain events (RideRequested, TripCompleted, SOSTriggered, PaymentCaptured) are published to the RabbitMQ message broker and consumed by downstream services asynchronously. Services do not call each other directly for non-transactional operations. The Outbox Pattern guarantees at-least-once event delivery without distributed transaction coupling.
API-First Design	The platform serves three distinct client surfaces. Rider App, Driver App, and Admin Portal, all consuming backend services through APIs. A consistent, versioned, and contract-driven API layer is required to support parallel development, client independence, and future extensibility.	All backend capabilities are exposed exclusively through versioned, documented API contracts. API contracts are defined before implementation begins. Mobile apps and the Admin Portal are decoupled from service internals. Breaking changes require a new API version; clients on older versions continue to function until explicitly migrated.
Security by Design	SheDrive handles sensitive personal identity data, financial transactions, and safety-critical SOS workflows. Security must be embedded in every architectural decision from registration through trip completion, it cannot be a deployment-time concern.	Authentication is enforced at the API Gateway before any request reaches a microservice. Authorisation is re-enforced at the service layer. Card data is never stored on platform infrastructure; PSP tokenisation is used exclusively. All personally identifiable data is encrypted at rest and in transit. Audit records are immutable and always produced for sensitive operations.
Configuration-Driven Behaviour	Business rules on the SheDrive platform including pricing, cancellation fees, commission rates, zone boundaries, operating hours, and notification templates change frequently and independently of platform code. Embedding them in code would require deployments for routine business decisions.	All configurable business rules are managed through the Admin Portal and take effect immediately without a code deployment. Pricing rule changes are versioned and effective-dated. Zone polygons, cancellation windows, and commission rates are runtime configuration, not code constants.
Mobile-First, Offline-Tolerant Design	The primary users, female Riders and female Drivers, interact exclusively through Flutter mobile applications, on consumer devices, over variable Egyptian mobile networks. All user-facing architectural decisions must prioritise the mobile context: intermittent connectivity, battery and background-execution constraints, GPS accuracy variance, device	All API contracts are designed for mobile consumption: minimal payloads, pagination, and idempotent retry-safe operations. The trip-state API is the authoritative source of truth that apps re-synchronise against on every foreground, reconnect, or restart; no critical flow depends solely on a pushed message arriving. Real-time location uses persistent WebSocket connections rather than polling; timers and counters are derived from server timestamps so they survive app restarts. Critical safety

Principle	Rationale / Source	Implications
	fragmentation, and best-effort push delivery.	communication always has an SMS fallback.
Fail Safely and Transparently	A safety-first platform cannot afford silent failures. Payment failures, SOS delivery failures, matching timeouts, and integration outages must degrade gracefully, preserving data integrity and providing actionable feedback; rather than cascading into wider system instability.	All async operations use dead-letter queues for unprocessable messages. Circuit breakers protect against cascading integration failures. Payment failures produce consistent terminal states before any notification is sent. SOS delivery failures escalate to operations. No failure path results in an inconsistent or unknown data state.
Auditability as a First-Class Concern	SheDrive operates under platform policies, women-only service, child-accompaniment declaration, financial traceability, and safety accountability, whose enforcement must be demonstrable to stakeholders at any time. Trust in the platform depends on the ability to evidence every sensitive decision.	Every financial transaction, identity verification decision, gender-mismatch report, SOS event, administrative configuration change, and driver approval produces an immutable audit record linked to a shared correlation identifier. The audit store is append-only and access-controlled. Audit data is retained per the platform data-governance policy (180-day minimum baseline).

3.2 Architectural Techniques Leveraged

The following architectural techniques are selectively applied across the SheDrive platform to support performance, resilience, real-time communication, mobile robustness, and maintainability. Each technique is applied where it provides a measurable benefit relative to the complexity it introduces.

Cache-Aside Pattern

Data that is frequently read and rarely changed, zone polygon configurations, pricing rules, commission rates, operating hours, notification templates, and active driver location indexes, is loaded on demand into an in-memory cache. The service checks the cache before querying the primary data store. On a cache miss, data is retrieved from the database and stored in cache for subsequent requests. This reduces database load, improves response times for high-volume read operations, and supports peak booking periods without scaling the primary data store.

WebSocket-Based Real-Time Streaming

WebSocket connections are maintained for active ride sessions, streaming the Driver's GPS coordinates to the Rider App and the Admin Operations dashboard at a maximum update interval of three seconds. The same channel delivers SOS alert notifications to the Operations queue in real time. WebSocket connections are established at driver assignment and terminated cleanly on ride completion, cancellation, or SOS resolution. REST remains the default for all transactional and administrative interactions.

Asynchronous Processing via Message Queues

Long-running, resource-intensive, or non-blocking operations, driver matching, notification dispatch, payment capture, and report generation are processed asynchronously via RabbitMQ message queues. Producers publish messages without waiting for consumer completion. This eliminates blocking of synchronous request threads, absorbs demand spikes gracefully, and isolates failures to individual consumers without affecting the rest of the platform.

Scheduled Background Processing (Hangfire)

Recurring and deferred jobs, cash settlement reminders, driver document and licence expiry sweeps, payment retry sweeps, report pre-aggregation, and stale-session cleanup are executed by Hangfire, the .NET background job framework, backed by SQL Server job storage. Hangfire provides scheduled (cron-style), delayed, and fire-and-forget job execution with automatic retries, a built-in monitoring dashboard, and persistence across service restarts. This separates time-driven platform behaviour from request-driven behaviour and gives operations full visibility of background work. Transactional Outbox draining and RabbitMQ relay is handled natively by the MassTransit built-in delivery service — not by Hangfire — as described in section 4.3.2.

Offline-Tolerant Mobile Interaction

The Rider and Driver apps are designed for intermittent connectivity: reference data (zones, saved places, language packs) is cached locally; state-modifying requests carry idempotency keys and are retried with exponential backoff; timers and counters (driver waiting time, OTP cooldowns) are recomputed from server timestamps rather than local clocks, so they survive app restarts and network drops; and the apps surface clear connectivity banners instead of failing silently. On every app foreground or reconnect, the app re-synchronises against the authoritative trip-state API.

Push + State-Reconciliation Hybrid

Push notifications (FCM/APNS) are treated as best-effort hints, never as the system of record. Every push-driven flow, driver approval, trip assignment, and payment outcome, has a pull-based reconciliation path: the app fetches authoritative state on open, resume, or reconnect. This guarantees correct behaviour on devices with disabled notifications, aggressive battery optimisation, or missed deliveries, which are common in the target device population.

Selective CQRS (Command Query Responsibility Segregation)

CQRS is applied selectively where a clear distinction exists between transactional operations and read-intensive operations. Write operations, ride requests, payment captures, SOS triggers, driver status updates, and admin configuration changes, use transactional command paths with full consistency guarantees. Dashboards, reports, and trip history consume read-optimised query models backed by pre-aggregated views. Full CQRS with event sourcing is not enforced globally to avoid unnecessary complexity at MVP scale.

Outbox Pattern

Domain services write domain events to a transactional outbox table within the same database transaction as the business operation. MassTransit's built-in delivery service (an IHostedService running inside each microservice host) polls the outbox on a configurable interval and publishes events to RabbitMQ. This guarantees at-least-once event delivery without introducing distributed transaction coupling between the primary data store and the message broker. The delivery service handles retry with configurable backoff natively — no separate Hangfire job is required or used for this purpose.

Geospatial Indexing

Active driver locations are stored with 2dsphere geospatial indexes, enabling sub-millisecond nearest-neighbour queries during driver matching. Zone polygon boundaries are persisted as GeoJSON objects, enabling efficient point-in-polygon checks for zone eligibility filtering and pickup zone detection. This replaces application-level proximity calculations with database-native spatial operations, reducing matching latency significantly.

Circuit Breaker

All external integration calls, to the PSP, mapping provider, SMS gateway, and push notification services are wrapped in circuit breakers. When an external service exceeds defined failure thresholds, the circuit opens and requests are failed fast, preventing blocked threads and cascading failures. The circuit transitions to a half-open state after a recovery window, allowing probe requests to test recovery before fully restoring traffic.

Rate Limiting

All inbound traffic to the Rider, Driver, and Admin Ocelot gateways is governed by rate limiting rules enforced at the gateway layer, scoped per client (authenticated user or API key, falling back to source IP for unauthenticated endpoints). Sensitive and abuse-prone endpoints, OTP requests,

login attempts, SOS triggers, and ride-matching requests, carry tighter, purpose-specific thresholds than general read traffic. Requests exceeding the configured threshold receive an HTTP 429 response with a Retry-After header rather than being queued or silently dropped. This protects backend services and downstream third-party integrations (Paymob, Google Maps, the SMS gateway) from traffic spikes, abusive clients, and unintentional retry storms, and preserves capacity for legitimate users during peak booking periods.

Idempotent Operations

Critical operations, payment capture, cash payment confirmation, SOS trigger, driver acceptance locking, and notification dispatch are designed to be idempotent. Retry attempts due to network failures, queue redelivery, or client reconnects produce the same deterministic outcome without duplicate side effects. Idempotency keys are propagated through the API Gateway and enforced at the service layer for all state-modifying operations.

Integration Adapters and Anti-Corruption Layer

Each external service is accessed through a dedicated integration adapter that encapsulates protocol details, authentication, payload mapping, and error normalisation. Core domain logic never depends on the external service's data model or contract. This isolates the domain from external changes and simplifies the onboarding of alternative providers in future phases without modifying business logic.

Versioned API Contracts

All service APIs exposed through the API Gateway are versioned. Version changes are backward-compatible by default; breaking changes require a new version path. This enables mobile clients and the Admin Portal to evolve at different release cadences from backend services. Mobile apps on older versions remain functional until explicitly migrated. API versioning is enforced at the gateway layer and re-asserted at each service.

Masked Number Proxy

Rider-to-Driver and Driver-to-Rider voice calls are routed through a masked-number proxy service. Neither party's personal mobile phone number is exposed to the other at any point. Proxy sessions are scoped to the active ride duration and are automatically terminated on ride completion or cancellation. This protects user privacy and supports the platform's women-only safety commitment.

3.3 Architectural Constraints

Architectural Constraints are derived from stakeholder directives, platform policy decisions, technology and hosting commitments, and the realities of the mobile delivery context. They define non-negotiable boundaries within which all design decisions must operate. The client has confirmed that no external legal or regulatory restrictions apply to the platform at this stage; the constraints below are therefore technical and platform-policy constraints only.

Constraint	Description and Architectural Impact
Women-Only Eligibility. Non-Waivable Platform Policy	All Riders and Drivers must be female. There is no minimum age restriction for female users. Enforcement in Release 1 is human-led: an Administrator verifies each Driver from her submitted profile photo and identity documents before approval (with an explicit "I confirm this applicant is female" confirmation), and the assigned Driver visually verifies each Rider on her first trip, with the ability to cancel and report a gender mismatch, which suspends the Rider account for review. The single exception is a declared child passenger (under 15), flagged by the Rider at booking and surfaced to the Driver. This policy cannot be waived by any administrative role or configuration setting.

Constraint	Description and Architectural Impact
Manual, Human-Led Identity Verification (Release 1)	No automated eKYC capability. OCR, liveness detection, or face matching, is in scope for Release 1, and no external identity verification provider is integrated. The architecture must therefore not depend on automated verification outcomes anywhere: driver approval is a human decision in the Admin Portal based on submitted documents and the profile photo, and rider verification is the driver visual check on the first trip. Document capture, storage, review tooling, and decision audit must be first-class; automated verification is a defined future enhancement that must be attachable without restructuring onboarding flows.
Daytime-Only Operating Window	The platform does not accept new ride requests outside the configured daytime operating window. This constraint is enforced at both the API Gateway and service layers. No booking path, including retry queues and matching re-broadcasts, may bypass the operating window enforcement. Administrators configure the window through the Admin Portal; changes take effect without a code deployment.
Payment Card Data Security (PCI Scope Reduction)	Card payment data must comply with PCI DSS requirements. The platform achieves PCI scope reduction by delegating all card data capture and tokenisation exclusively to the PSP's hosted payment fields rendered in the Rider App. No internal service receives, processes, or stores raw card numbers. The platform stores only PSP-issued tokens. This constraint governs the integration design of the Payment Service and the Rider App payment flow.
Personal Data Protection and Privacy (Platform Policy)	National ID images, profile photos, and payment card tokens are sensitive personal data. As a matter of platform policy and good practice, all such data must be encrypted at rest and in transit, access to personal data is restricted to authorised roles, and every access to sensitive data is recorded in the audit trail. This constraint affects data model design, storage configuration, access control policies, and retention schedules.
Data Retention Baseline. 180 Days Minimum	All trip, payment, and operational event data is retained for a minimum of 180 days as a platform data-governance baseline, ensuring that disputes, safety incidents, and financial reconciliation can always be investigated against complete records. Storage and archival strategy must guarantee full retrievability within this window. The baseline is a configurable policy, not a hard-coded value, so it can be extended if future obligations arise.
Hosting Constraint. AWS Managed Servers + S3 Only	The platform is hosted on AWS EC2 virtual machines used purely as managed servers, plus AWS S3 for object storage. No dependence on higher-level AWS managed application services (e.g., ECS, RDS, ElastiCache, CloudWatch, Secrets Manager) is permitted. All platform software, databases, message broker, cache, observability stack, and secrets management, is self-hosted on the VMs. This preserves full portability of the platform to any VM-based hosting provider.
Mobile Platform Constraints	The Rider and Driver apps must operate within the constraints of the iOS and Android ecosystems: App Store and Google Play review policies (including their rules on location, photo, and payment data handling), background-execution and battery-optimisation limits that affect GPS streaming and timers, best-effort push notification delivery, device and OS-version fragmentation, and variable mobile network quality. Architectural consequences: SMS fallback for safety-critical messages, server-timestamp-derived timers, idempotent retry-safe APIs, and pull-based state reconciliation on app resume.

Constraint	Description and Architectural Impact
Bilingual Operation. Arabic (RTL) and English	All user-facing surfaces. Rider App, Driver App, and Admin Portal, must support Arabic with full right-to-left layout rendering and English from the first release. All notification channels (SMS, push, in-app) must support bilingual templates with variable substitution. Language selection must persist across sessions. This constraint affects UI component design, notification template management, data models for locale-aware content, and the content management capability of the Admin Portal.
Audit Immutability and Access Control	All financial transactions, identity verification decisions, gender-mismatch reports, SOS events, and administrative configuration changes must produce immutable audit records. Audit records must not be deletable or modifiable by any user role, including Administrators. Access to audit data is restricted to authorised roles. All audit data access events are themselves logged. This constraint applies to the design of the audit store, the access control model, and the log retention policy.
Single-Provider Integrations at MVP	The MVP integrates exactly one PSP (Paymob), one mapping provider (Google Maps Platform), and one local SMS gateway. No automated failover to alternative providers exists in Release 1. The integration adapter and anti-corruption layer design must keep each provider replaceable, and degradation behaviour (cash fallback, cached estimates, retry queues) must be explicit for each provider outage scenario.

3.4 Business View

This section summarises the business-facing view of the SheDrive architecture, the personas the system serves, the main scenarios it supports, the solution deliverables it produces, and the boundaries of the MVP scope. Detailed business rules, functional requirements, and process flows are documented in the BRD (v1.1) and are referenced here for architectural context without duplication.

3.4.1 Personas and Main Scenarios

The following table defines the primary personas that interact with the SheDrive platform and their core system interactions.

Persona	Description	Primary Interactions with the Platform
Female Rider	Female user of any age, registered via mobile OTP and visually verified by the assigned driver on her first trip. May travel alone or declare a male child passenger under 15 (the only exception to the women-only rule).	Registration with mobile OTP, profile setup, ride booking, fare estimate, child passenger declaration, real-time driver tracking, in-app communication, SOS, payment (cash or digital), rating and feedback, trip history, live trip sharing with trusted contacts.
Female Driver	Licensed female user of any age, operating her own eligible vehicle, approved by an Administrator after human review of her documents and profile photo.	Onboarding and document submission, online/offline toggle, working zone selection, incoming ride handling, first-trip rider verification at pickup, navigation, cash confirmation, earnings and cash balance view, SOS, rider rating.

Persona	Description	Primary Interactions with the Platform
Administrator	Authorised back-office staff with full platform management permissions.	Driver onboarding review and approval, account management, zone and pricing configuration, commission setup, cancellation rules, audit log review, notification and content management.
Operations Supervisor	Senior operational role with incident and escalation authority.	SOS queue management, escalation workflow execution, live ride monitoring, incident resolution and closure.
Customer Support Agent	Back-office user handling user-facing inquiries and disputes.	Account lookup, trip history review, dispute logging and tracking, cancellation review, follow-up communications.
Finance User	Responsible for payment reconciliation and cash settlement oversight.	Cash reconciliation dashboard, driver cash balance and settlement tracking, manual refund processing, payment failure review, financial report export.
Compliance User	Responsible for regulatory adherence and verification auditing.	Identity document audit, verification decision review, gender-mismatch report review, audit trail oversight, structured data export for stakeholders.
Management User	Read-only stakeholder consuming operational and strategic dashboards.	KPI dashboard, daily and weekly ride summaries, zone performance, safety incident analytics, cancellation trends, payment reconciliation summary.

The following table defines the ten main business scenarios that drive the architectural design of the platform.

#	Scenario	High-Level Flow
S-01	Rider Registration & First-Trip Verification	Rider downloads app → registers with mobile OTP → completes profile setup → booking is unlocked immediately → on her first trip the assigned driver visually verifies at pickup that the rider is female → on confirmation the rider is recorded as verified; a gender mismatch cancels the trip and suspends the rider account for review. Declared child passengers (under 15) are the only exception to the women-only rule.
S-02	Driver Onboarding & Admin Approval	Driver registers → submits personal details (including 14-digit national ID number and background-check consent) + vehicle details + vehicle photo + driving licence and vehicle registration documents (front and back) + profile photo → admin reviews the onboarding queue, inspects documents inline, and explicitly confirms the applicant is female from the profile photo and ID documents → approves or rejects with mandatory reason → driver notified by push, with status visible on next app open.
S-03	Ride Booking & Driver Matching	Verified rider selects pickup and destination → fare estimate displayed → child passenger (under 15) declared if applicable → payment method selected → ride submitted → dispatch engine filters eligible drivers by zone and proximity → request broadcast → driver accepts → rider notified with driver card.
S-04	Trip Lifecycle Execution	Driver navigates to pickup → confirms arrival → wait window begins → driver starts trip → real-time location streamed to rider and operations → driver completes trip at destination → lifecycle transitions to TripCompleted.

#	Scenario	High-Level Flow
S-05	Digital Payment Processing	TripCompleted triggers fare calculation → pre-authorised amount captured via PSP → success: trip marked paid, receipt generated, rating flow unlocked → failure: rider notified, retry offered → persistent failure: future bookings blocked until resolved.
S-06	Cash Payment Processing	Rider pays driver cash → driver confirms receipt → rider confirms → dual confirmation closes trip → cash recorded in driver ledger → commission noted as owed → settlement reminder triggered when balance exceeds threshold.
S-07	SOS Trigger & Operations Response	Rider or driver taps SOS → system captures location, trip ID, and user details → real-time alert to Operations SOS queue → operator reviews and contacts user → if unresolved within SLA, escalates to supervisor → supervisor may contact external authorities → incident closed with resolution notes.
S-08	Ride Cancellation with Fee	Rider or driver initiates cancellation → selects reason → system evaluates ride status against zone cancellation rules → if within free window: cancels without charge → if outside: fee displayed, confirmed, and charged → both parties notified → audit record created.
S-09	Zone & Pricing Configuration	Admin draws zone polygon → sets base fare, per-km rate, per-minute rate, minimum fare, cancellation rules, and optional peak-hour multiplier → saves with effective date → pricing version recorded → cache invalidated → new rates applied to subsequent fare estimates.
S-10	Operational Reporting & KPI Dashboard	Operations or management user selects date range and zone filters → system queries read-optimised reporting models → GMV, active users, completion rate, cancellation patterns, SOS metrics, and zone performance rendered → exported to CSV or Excel as required.

3.4.2 Deliverables and Solution Elements

The SheDrive MVP is delivered as a set of ten business capability areas, each corresponding to one of the module boundaries defined in the BRD. The following inventory describes each deliverable from a business perspective, the capabilities it provides to users and operators.

Rider Application

- Account registration and mobile number verification.
- Profile setup and management with language preference.
- Map-based ride booking with pickup and destination selection.
- Fare estimate display before request submission.
- Child accompaniment declaration at booking.
- Payment method selection (cash or digital) per ride.
- Real-time driver tracking with continuously updated estimated arrival.
- Driver information card on assignment (photo, vehicle details, rating).
- In-trip text communication and masked-number voice call to assigned driver.
- SOS button accessible throughout the active ride.
- Live trip sharing link for trusted contacts.
- Trusted contact management (up to three contacts).
- Ride cancellation with reason selection and fee disclosure.

- Fare breakdown and receipt on trip completion.
- Cash payment dual-confirmation flow.
- Post-trip driver rating and feedback.
- Trip history and receipt access.
- Saved places management.
- Language switching between Arabic (RTL) and English.
- In-app help and customer support contact.

Driver Application

- Account registration and mobile number verification.
- Personal details submission, including 14-digit national ID number and background-check consent.
- Driving licence upload, front and back, with format and size validation.
- Vehicle document submission, vehicle registration, front and back.
- Vehicle photo upload for rider identification.
- Profile photo capture, shown to riders for identification and used by admins to confirm the applicant is female.
- First-trip rider verification at pickup, confirm the rider is female and board, or cancel with a gender-mismatch report (declared child passengers under 15 are exempt).
- Onboarding status tracking across all review stages.
- Online/offline availability toggle.
- Working zone selection.
- Incoming ride request display with accept or decline.
- Navigation to pickup location.
- Arrived at pickup confirmation.
- Trip start and real-time location sharing.
- Navigation to destination.
- Trip completion and fare display.
- Cash collection confirmation matched with rider confirmation.
- Digital payment outcome display.
- Ride cancellation with reason selection.
- Earnings dashboard (daily, weekly, per trip) with commission breakdown.
- Cash owed to platform balance and settlement reminders.
- Post-trip rider rating.
- In-trip text communication and masked-number voice call to rider.
- SOS button accessible throughout the active trip.
- Language switching between Arabic and English.
- In-app help and support contact.

Administration & Operations Portal

- Two-factor authenticated login with enforced session timeout.
- Role-based access control across all portal capabilities.
- Live operational dashboard, active rides, online drivers, open SOS events, payment failure rate.
- Driver onboarding queue with document review, approval, and rejection workflow.
- Driver account management, activation, suspension, and deactivation.
- Rider account management, search, view, suspension, and block.
- Live ride monitoring map and tabular active ride list.
- Complete ride detail view, event timeline, location trail, payment, and communication log.
- Cancellation review queue with abnormal pattern flagging.
- Incident management workflow with owner assignment, status tracking, and resolution.

- SOS alert queue with one-click response actions and escalation workflow.
- Neighbourhood and zone management, polygon drawing, activation, and boundary editing.
- Pricing configuration per zone, base fare, per-km rate, per-minute rate, minimum fare, zone-pair rules.
- Cancellation fee rule configuration per zone.
- Commission rate configuration with effective-date scheduling.
- Cash reconciliation dashboard with overdue settlement identification.
- Per-driver cash balance and settlement history.
- Manual refund processing with reason capture.
- Dispute management workflow.
- Audit log viewer, searchable and filterable.
- Notification template management per channel and language.
- Static content management. Terms and Conditions, privacy policy, FAQs.
- Driver document expiry indicators (driving licence, vehicle registration) surfaced for operational follow-up.
- Structured trip data export (CSV / Excel) for operational and stakeholder reporting.

Identity & Verification Module

- Driver onboarding application intake, personal details with 14-digit national ID number format validation and duplicate registration detection.
- Driver document collection, driving licence (front and back) and vehicle registration (front and back) with file type and size validation.
- Driver profile photo capture, used for admin gender confirmation and rider-facing driver identification.
- Background-check consent capture at application time.
- Admin application review workspace, full application detail with inline document, vehicle photo, and profile photo viewing.
- Mandatory human gender confirmation. Explicit admin confirmation that the applicant is female before approval. No minimum age requirement applies to female applicants.
- Approve / reject decision workflow with mandatory rejection reason and full decision audit.
- First-trip rider verification, `is_first_trip` flag computed at trip creation and exposed to the Driver App; the driver confirms the rider is female or cancels with a gender-mismatch report.
- Rider account suspension for review upon a gender-mismatch report.
- Declared child passenger exception, per-trip child flag captured at booking and surfaced to the driver.
- Verification status lifecycle and notifications, pending, approved, and rejected (with resubmission guidance).

Trip & Dispatch Module

- Ride request intake and operating-hours enforcement.
- Zone-aware driver eligibility filtering by working zone and pickup location.
- Driver proximity and rating-based matching.
- Ride request broadcasting with configurable timeout.
- Atomic driver acceptance locking, prevents double assignment.
- Decline and timeout reassignment to the next eligible driver.
- Estimated time of arrival calculation with continuous updates.
- Trip lifecycle state management. Requested, Searching, Assigned, EnRoute, Arrived, Started, Completed, Cancelled.
- Real-time driver location streaming to Rider App and Operations portal.
- Pickup and drop-off geofence detection.
- No-driver-available retry queue with rider notification.

Pricing & Neighbourhood Zones Module

- Map-based neighbourhood polygon definition and management.
- Per-zone base fare, per-kilometre rate, and per-minute rate.
- Per-zone minimum fare configuration.
- Zone-to-zone specific pricing rules.
- Per-zone cancellation fee rules and timing windows.
- Optional peak-hour pricing multiplier per zone.
- Pricing rule versioning with effective dates for audit and rollback.

Payment Module

- Integration with the configured Payment Service Provider for digital card transactions.
- Card tokenisation and saved card management.
- Payment pre-authorisation on trip start.
- Payment capture on trip completion.
- Cash payment recording via dual rider and driver confirmation.
- Commission calculation and application per completed trip.
- Driver cash balance ledger with running owed balance.
- Cash settlement workflow with confirmation recording.
- Payment failure detection, rider notification, and retry management.
- Unresolved payment failure ride blocking.
- Cancellation fee calculation and collection.
- Manual refund capability for digital trips.
- Trip receipt generation for cash and digital trips.

Safety & SOS Module

- Single-tap SOS button accessible to both Rider and Driver during active rides.
- Automated capture of location, trip identifier, and party details on SOS activation.
- Real-time SOS alert delivery to the Operations queue with audible and visual notification.
- Response SLA tracking from SOS trigger to first operator action.
- Defined escalation workflow, initial responder, supervisor, and external authority, with time-based triggers.
- Live trip sharing link generation and automatic delivery to Rider trusted contacts.
- Trusted contact management, up to three contacts per Rider.
- SOS incident record creation and status tracking through to resolution.
- Post-incident follow-up workflow with outcome and review notes capture.
- Women-only and age policy enforcement across all platform interactions.

Notification Module

- SMS delivery for one-time passwords and critical alerts via configured SMS gateway.
- Push notification delivery via platform notification services for iOS and Android.
- In-app notification inbox for Rider, Driver, and Administrator applications.
- Email notifications for administrative and finance events.
- Bilingual notification templates (Arabic and English) per channel with variable substitution.
- Delivery status tracking and retry with exponential backoff.
- User-level preferences for non-critical notification channels.
- Critical alert priority mode that bypasses device quiet hours for SOS and safety events.

Reporting & Analytics Module

- Live operational dashboard, active rides, online drivers, payment success rate, open SOS events, cancellation rate.

- Daily and weekly ride summaries, trip count, gross merchandise value, completion rate, and average fare, by zone.
- Driver performance reports, trips completed, earnings, cancellation rate, rating, hours online.
- Cancellation analytics, reasons, timing patterns, zone and driver correlation.
- Payment reconciliation report, cash and digital split, success and failure rates, outstanding cash balances.
- Safety incident reports. SOS counts, response times, escalation outcomes, repeat-incident accounts.
- Zone performance reports, trips, GMV, supply versus demand balance, cancellation rate.
- Management KPI dashboard. GMV, active users, retention, safety indicators, financial health.
- CSV and Microsoft Excel export for all reports.
- Structured trip data export (trip lengths, timing, geographic distribution) for stakeholder reporting.

3.4.3 Out of Scope

Any capability not listed under Deliverables and Solution Elements above is considered out of scope for the MVP. The following table makes explicit the areas most likely to be queried, together with the rationale for their exclusion.

Area	Comment
Ride Pooling and Shared Rides	Multi-passenger shared ride models are excluded. The platform supports one rider per trip (plus declared child passengers under 15) only. Deferred to a subsequent phase subject to operational maturity review.
Scheduled and Recurring Rides	Riders cannot pre-book rides for a future time or configure recurring journeys. All requests are immediate. Deferred to a subsequent phase.
Intercity, Hourly, and Rental Ride Models	The platform operates within Greater Cairo only, for point-to-point single trips. Intercity transfers, hourly charter, and vehicle rental models are not supported in the MVP.
Multiple Service Tiers	A single service tier applies to all rides. Economy, premium, and family-plus tiers are not modelled in the MVP fare or matching logic.
24/7 Operating Hours	The platform operates within a defined daytime-only window at MVP launch. Around-the-clock operation is planned for a subsequent phase, contingent on safety performance and operational readiness.
In-App Rider and Driver Wallets	Neither Riders nor Drivers hold a balance within the platform. Payment is per ride (digital card or cash). Wallet functionality is deferred to a post-MVP phase.
Automated Driver Payouts and Settlement Automation	Driver cash settlement is an operationally supported manual process in the MVP. End-to-end automated payout is out of scope.
Promotional Codes, Discounts, and Loyalty Programmes	No promotional pricing, referral incentives, or loyalty reward mechanisms are included. Deferred to a subsequent phase.
Corporate, Enterprise, and Family Accounts	The platform serves individual Riders and Drivers only. Corporate accounts, family plans, multi-user accounts, and guardian dashboards are not in scope.

Area	Comment
Subscription and Membership Pricing	All rides are charged at the standard per-trip rate. Subscription-based pricing models are out of scope.
AI-Driven Safety Scoring and Behavioural Analytics	Automated risk scoring, behavioural profiling, and predictive safety analytics are not included. Safety enforcement in the MVP is policy-based and human-reviewed.
In-Trip Audio and Video Recording	The platform does not record, store, or process any audio or video content from trips.
Full Business Intelligence (BI) Platform	Advanced analytical dashboards, predictive demand forecasting, cross-platform data warehousing, and ad-hoc BI queries are excluded. Foundational operational reporting is in scope; full BI capability is deferred.
Dispatcher and Call-Centre Booking	Ride booking is self-service through the Rider App only. No call-centre or manual dispatch booking capability is included.
Marketing Campaign Management	Campaign creation, audience segmentation, A/B testing, and promotional communications management are not in scope.
Platform-Supplied or Leased Vehicles	The platform does not own, supply, lease, or maintain vehicles. Drivers operate their own vehicles subject to platform eligibility criteria.
Multiple PSPs and Advanced Payment Routing	A single Payment Service Provider is integrated in the MVP. Multi-PSP routing, fallback PSP switching, and advanced payment orchestration are deferred.
Multi-Country or Multi-Currency Operation	The MVP is a single-country (Egypt), single-currency (EGP) deployment. International expansion and currency management are out of scope.

3.5 Quality Attribute View

This section defines the key quality attributes that drive the architectural design of SheDrive. Each attribute is expressed in objective terms and is directly linked to architectural responsibilities and design decisions. Quality attributes are not aspirational; they are constraints that the architecture must satisfy.

3.5.1 User Experience and Visual Design

The system must support multiple user categories. Rider, Driver, Administrator, Operations Supervisor, Customer Support, Finance, Compliance, and Management, each interacting through a distinct application surface with a role-appropriate feature set. The architecture must enforce strict role-based separation of capabilities and data at both the API and service layers.

When users interact with any application surface, the interface must expose only the functions and data relevant to their assigned role, preventing unauthorised access paths at the UI level while enforcing the same boundaries server-side.

When a Rider submits a booking or a Driver accepts a ride, the system must provide immediate visual confirmation of the state transition. Asynchronous processing must not block user-visible responses.

All user-facing surfaces must support bilingual operation. Arabic with full RTL layout and English from the first release. Language selection must be persistent across sessions and must not affect business logic, data storage, or reporting.

When ride lifecycle events occur, both Rider and Driver must receive consistent, timely, channel-appropriate notifications without requiring manual refresh.

3.5.2 Performance

The system must sustain responsive user interactions and time-sensitive real-time flows under the MVP load profile of fewer than 5,000 registered users and a lean concurrent session volume.

When a Rider submits a ride request, the matching algorithm must return a result within a target of two seconds under normal operating conditions.

When a trip is in progress, driver GPS coordinates must be streamed to the Rider App and the Admin Operations dashboard at a maximum update interval of three seconds.

When a Rider requests a fare estimate before booking, the response must be returned within one second under normal conditions.

When an SOS alert is triggered, it must appear in the Operations SOS queue within five seconds of the trigger event.

When dashboards and reports are accessed during peak periods, read operations must execute against optimised query models and must not degrade transactional processing on the primary data store.

3.5.3 Scalability

The MVP architecture is sized for a lean initial user base of fewer than 5,000 registered users within Greater Cairo. The architecture is designed to scale to significantly higher volumes in subsequent phases without requiring core redesign.

When ride request volumes increase due to demand growth or geographic expansion, the Trip & Dispatch and Notification services must scale horizontally without requiring changes to service logic or data schema.

When additional neighbourhoods are onboarded, the Pricing & Zone service must accommodate new configurations without service downtime or data migrations.

When reporting usage grows, read workloads must scale independently from transactional processing through CQRS-light separation and read-optimised query models.

Scalability is achieved through stateless microservice design, horizontal container scaling, message queue-based workload buffering, caching for read-heavy operations, and geospatial indexing for proximity queries.

3.5.4 Availability and Reliability

The platform must provide consistent and predictable service availability during the configured daytime operating window. The target production availability is 99.5% of scheduled operating hours.

When an individual microservice fails or restarts, the platform must continue to function for unaffected capabilities. Stateless service design and container orchestration enable fast restart and recovery without manual intervention.

When a payment capture fails after trip completion, the system must maintain a consistent trip state, notify the Rider of the failure, and allow retry without duplicating the trip record or the fare.

When an external integration becomes unavailable, the circuit breaker pattern prevents repeated failed calls, and the affected flow degrades gracefully with a clear user-facing message.

When asynchronous operations fail, dead-letter queues capture failed messages for operational review without data loss.

When database connectivity is temporarily disrupted, the Outbox Pattern ensures no domain events are lost, events remain in the transactional outbox and are re-published once connectivity is restored.

3.5.5 Security

Security is a foundational quality attribute for SheDrive, given the sensitivity of personal identity data, financial transaction records, and the safety-critical nature of the SOS workflow.

When users or services access any API endpoint, all requests must carry a valid, verified authentication token. Unauthenticated requests are rejected at the entry point before reaching any service.

When back-office users access the Admin Portal, authentication must require a second factor in addition to credentials.

When Riders enter card payment details, raw card numbers must never be transmitted to or stored on SheDrive platform infrastructure. All card data is tokenised by the PSP's hosted payment fields exclusively.

When user data is stored, all personally identifiable information and financial records must be encrypted at rest. All data in transit must use TLS 1.2 or higher.

When Rider and Driver communicate via voice call during an active ride, both parties' personal phone numbers must be protected by the masked-number proxy.

When security-sensitive events occur, identity verification decisions, payment captures, SOS triggers, admin configuration changes, and driver approvals, they must generate immutable audit records linked to the actor, timestamp, and execution context.

Role-based access control is enforced at the entry point and re-enforced at the service layer, aligned to the seven back-office roles defined in the BRD plus Rider and Driver roles.

3.5.6 Maintainability and Evolvability

The platform must support ongoing change driven by business growth, regulatory updates, new integrations, and future phases; without requiring core architectural redesign.

When pricing rules, cancellation fees, commission rates, zone boundaries, or operating hours change, administrators must be able to apply those changes through the Admin Portal without a code deployment.

When a new service capability is introduced or an existing one is modified, the change must be localisable to the relevant service and its data store. Cross-service cascading changes are a design anti-pattern to be avoided through clean API contract boundaries.

When APIs evolve, versioning ensures that mobile clients on older versions continue to function until explicitly migrated.

Clean architecture is applied within each backend service: the domain layer has no dependency on infrastructure concerns; application services orchestrate domain logic; infrastructure adapters implement external integrations. This layering makes individual services testable, replaceable, and independently evolvable.

3.5.7 Operability and Monitoring

The platform must be fully observable and operable in the production environment. Operational teams must be able to detect, diagnose, and respond to issues without requiring system downtime or access to source code.

When abnormal behaviour occurs in any service, elevated error rates, latency spikes, queue depth growth, and circuit breaker trips, monitoring alarms must surface the condition to the operations team within a defined detection window.

When distributed request flows span multiple services, distributed tracing must provide end-to-end visibility with a single correlation ID.

When the SOS response SLA threshold is breached, the system must raise a priority alert in the Operations portal and via the configured alerting channel.

Every service exposes standard liveness and readiness health endpoints. A centralised health aggregation view surfaces the collective health state of the platform to the operations team.

Structured logs from all services are emitted to centralised log storage with consistent fields including service name, correlation ID, and ride ID where applicable. Log retention is aligned to the 180-day regulatory minimum.

3.5.8 Auditability and Compliance

The platform must support internal governance, stakeholder assurance, and any future external oversight requirements throughout its operation.

When financial transactions are executed, each event must be recorded with actor, amount, timestamp, ride reference, and payment method in the immutable audit trail.

When identity verification decisions are made, the decision, confidence scores, actor, and timestamp must be persisted.

When administrative configuration changes are applied, the previous value, new value, actor, and effective timestamp must be recorded.

When SOS events are triggered and handled, the full event chain, trigger time, response time, actions, escalation events, and resolution must be preserved as a linked incident record.

When required by regulatory authorities, the system must export trip data records in a structured format covering the 180-day minimum retention window.

Access to audit log data is restricted to authorised roles. All audit log access events are themselves logged.

3.5.9 Interoperability

The SheDrive platform integrates with a defined set of external services: the configured Payment Service Provider, Google Maps Platform, push notification services (FCM/APNS), and an SMS gateway. Each integration must be designed to be replaceable without modifying core platform business logic.

When an external provider is replaced, for example, substituting the PSP or migrating to a different SMS gateway, the impact must be confined to the relevant integration adapter. Domain services must remain unchanged.

When external APIs evolve or deprecate capabilities, the anti-corruption layer and adapter design must absorb those changes before they reach the platform domain.

When new external integrations are required in future phases, additional PSPs, mapping providers, and communication channels, the adapter pattern must allow onboarding without rearchitecting existing services.

All external integration points are documented with explicit request and response contracts, authentication mechanisms, timeout policies, and circuit breaker thresholds. No integration is allowed to block the core ride lifecycle beyond its defined timeout.

3.5.10 Testability

The platform must be structured to support automated testing at multiple levels, unit, integration, and end-to-end, as a prerequisite for the CI/CD pipeline. Untestable code is treated as an architectural defect.

When domain logic is implemented, it must be unit-testable without spinning up databases, message queues, or external service dependencies. Clean Architecture layering and dependency inversion are the primary mechanism for achieving this.

When service-level integration behaviour is validated, integration tests must be runnable against local or containerised equivalents of dependencies without requiring access to production or third-party systems.

When external integrations are tested, they must be testable against stable mocks or sandboxes. No integration test must depend on a live third-party production endpoint.

When the CI/CD pipeline runs, all tests must execute automatically as a quality gate before any build is promoted. A build that fails tests must not be deployable.

Critical platform flows, ride matching, SOS trigger and alert delivery, payment capture, and dual cash confirmation must have explicit automated test coverage and must not be left to manual testing alone.

4. Solution Architecture

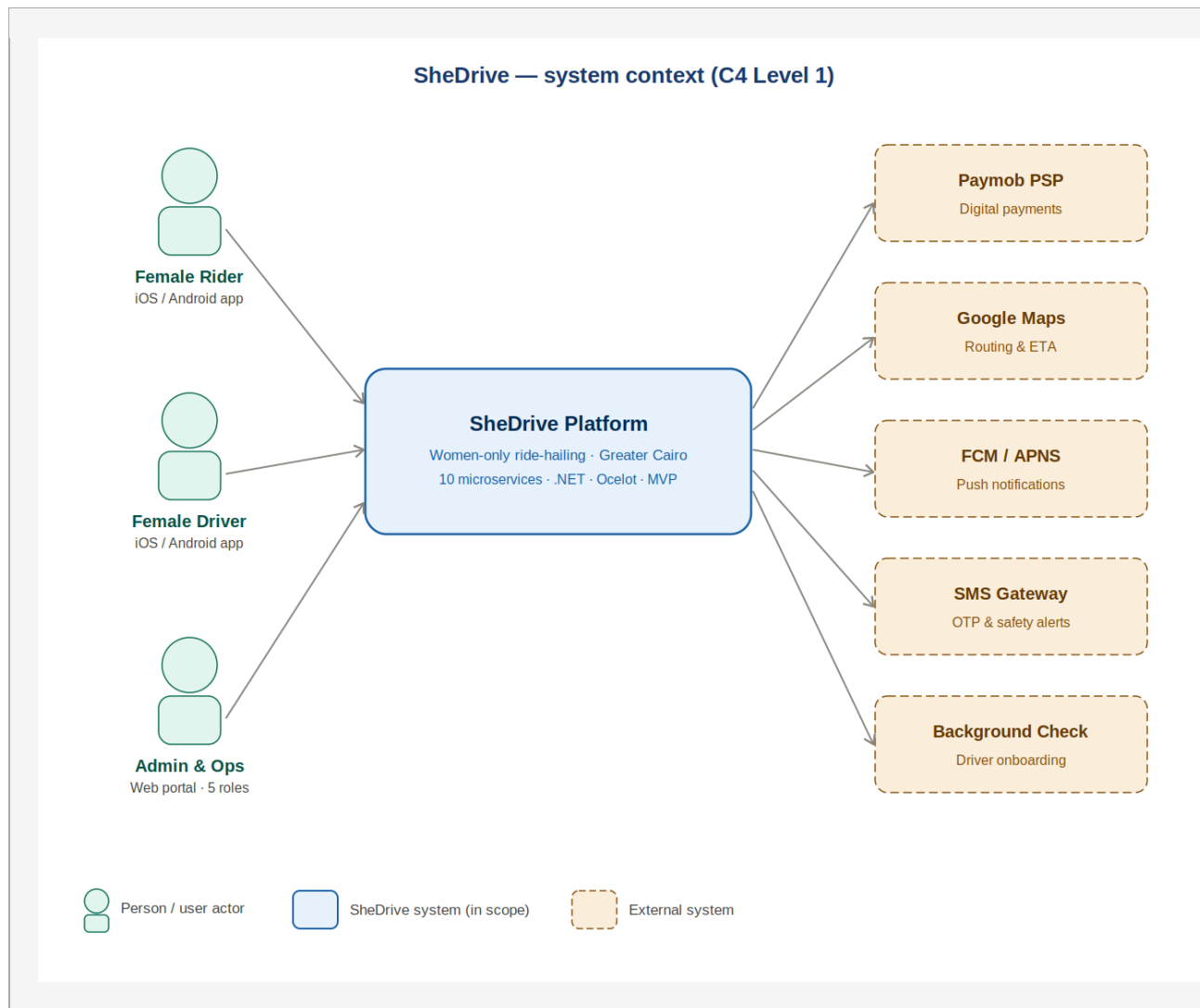
This section describes the SheDrive solution architecture and explains how its structural decisions realise the business objectives, quality attributes, and constraints defined in Sections 2 and 3. The architecture is presented at three levels of detail: the system context view (external boundaries and integrations), the functional blocks view (internal module decomposition and responsibilities), and the build view (implementation pattern, technology stack, and team accountability).

4.1 System Context View

The System Context View defines the SheDrive platform as a single logical system and describes its interactions with external actors and external service providers. It establishes the solution boundary, identifies trust boundaries, and clarifies high-level responsibilities without exposing internal structure.

Figure 1 below presents the Level 0 Context Diagram. The SheDrive platform is shown as a single bounded system. External actors, Female Rider and Female Driver, interact with the platform through their respective mobile applications. Back-office Users (seven roles) access the platform through the Administration and Operations Portal. Four external service providers are called by the platform: Paymob (digital payments), Google Maps Platform (mapping, routing, and ETA), Firebase FCM / APNS (push notifications), and a local SMS Gateway (OTP and critical alerts). Identity verification is manual and human-led in Release 1; no external verification or background-check provider is integrated. All communication between the platform and external services crosses a clearly defined boundary over HTTPS. The platform is responsible for the correctness and integrity of all operations within its boundary; external providers are responsible for the availability and reliability of their own interfaces.

[Figure 1. System Context Diagram Level 0]



Release 1 Concurrency Model

With 5,000 registered users at Release 1 (approximately 70% Riders / 30% Drivers), the following concurrency estimates form the design ceiling for VM sizing and load-balancing configuration. Values are conservative for a single-city, daytime-window platform:

Parameter	Estimate	Basis / Notes
Registered users (Release 1)	5,000	Release 1 target (BRD v1.1), approx. 3,500 Riders and 1,500 Drivers.
Peak concurrent Rider sessions	≤ 350	10% of 3,500 registered Riders active simultaneously at peak booking window.
Peak concurrent Driver sessions	≤ 150	10% of 1,500 registered Drivers online simultaneously.
Total peak concurrent sessions	≤ 500	Design ceiling for gateway and application-layer sizing.
Peak active trips (WebSocket)	≤ 200	Approx. 40% of concurrent rider sessions in an active trip, each holding one WebSocket streaming connection.
Peak API request rate (gateway)	≤ 80 req/sec	Booking burst + polling for non-streaming clients. Validated against Section 7.2 gateway VM specs.

Parameter	Estimate	Basis / Notes
Concurrency scaling trigger	≥ 400 concurrent sessions sustained	Breach triggers the infrastructure review defined in Section 7.5.

Figure 1. SheDrive System Context View (Level 0)

Figure 2 below presents the Level 1 Container Diagram, which opens the platform boundary and exposes the primary containers, the deployable units, that together form the SheDrive system. At this level the platform is decomposed into three client-facing applications (Rider App, Driver App, and Admin Portal), three independent API Gateway instances (Rider Gateway, Driver Gateway, and Admin Gateway) each serving a distinct client surface for REST traffic only, a dedicated SignalR Real-Time Hub handling all WebSocket streaming for GPS location fan-out and SOS alert delivery, eight backend .NET microservices organised into domain and cross-cutting groups, an asynchronous message broker backbone (RabbitMQ), and three data stores (SQL Server for transactional data, MongoDB for events and audit records, and Redis Cache for in-memory configuration and session data). Each container is independently deployable and communicates with others through defined interfaces: REST APIs via the respective gateway instance, persistent WebSocket connections via the Real-Time Hub, or RabbitMQ messages for asynchronous operations.

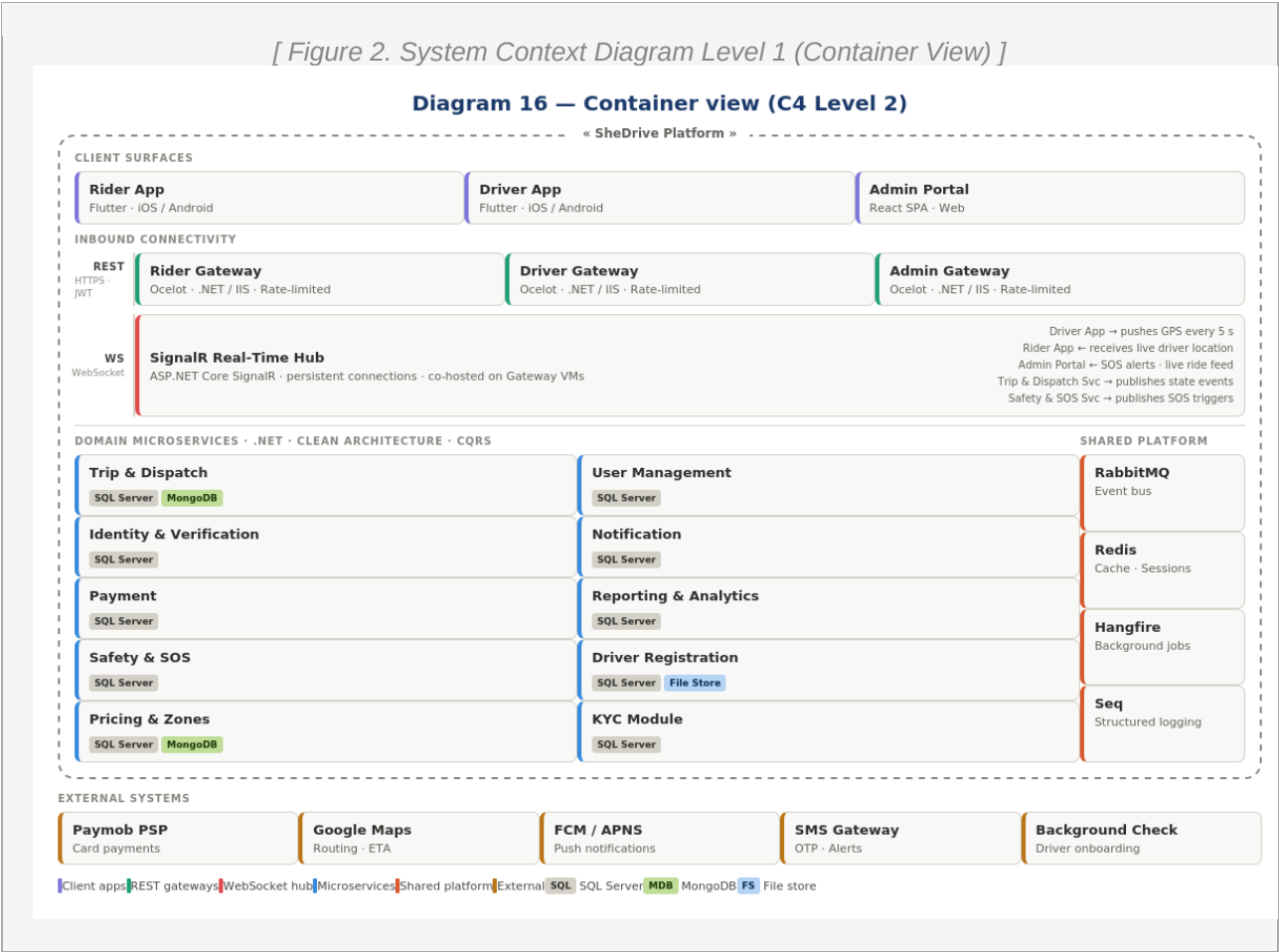


Figure 2. SheDrive Container View (Level 1)

4.2 Functional Blocks

This section decomposes the SheDrive solution into its major functional blocks and describes the responsibilities of each block. The purpose is to present a clear functional breakdown that supports the build view and deployment view, while remaining independent of low-level implementation details.

The solution is realised as a set of independently deployable microservices within a single platform boundary. These services are organised into four logical layers, as summarised in the table below and illustrated in Figure 3.

Layer	Modules
User-Facing Surfaces	Rider App · Driver App · Admin & Operations Portal
Domain Capabilities	Trip & Dispatch · Identity & Verification · Payment · Safety & SOS · Pricing & Zones · User Management · Maps & Location · Driver & Vehicle Registration
Cross-Cutting Capabilities	Notification · Reporting & Analytics
Platform / Support	API Gateway & Communication · Audit Trail · Logging & Tracing · Health Check

[Figure 3. Business Module View (10 modules across 3 business layers)]

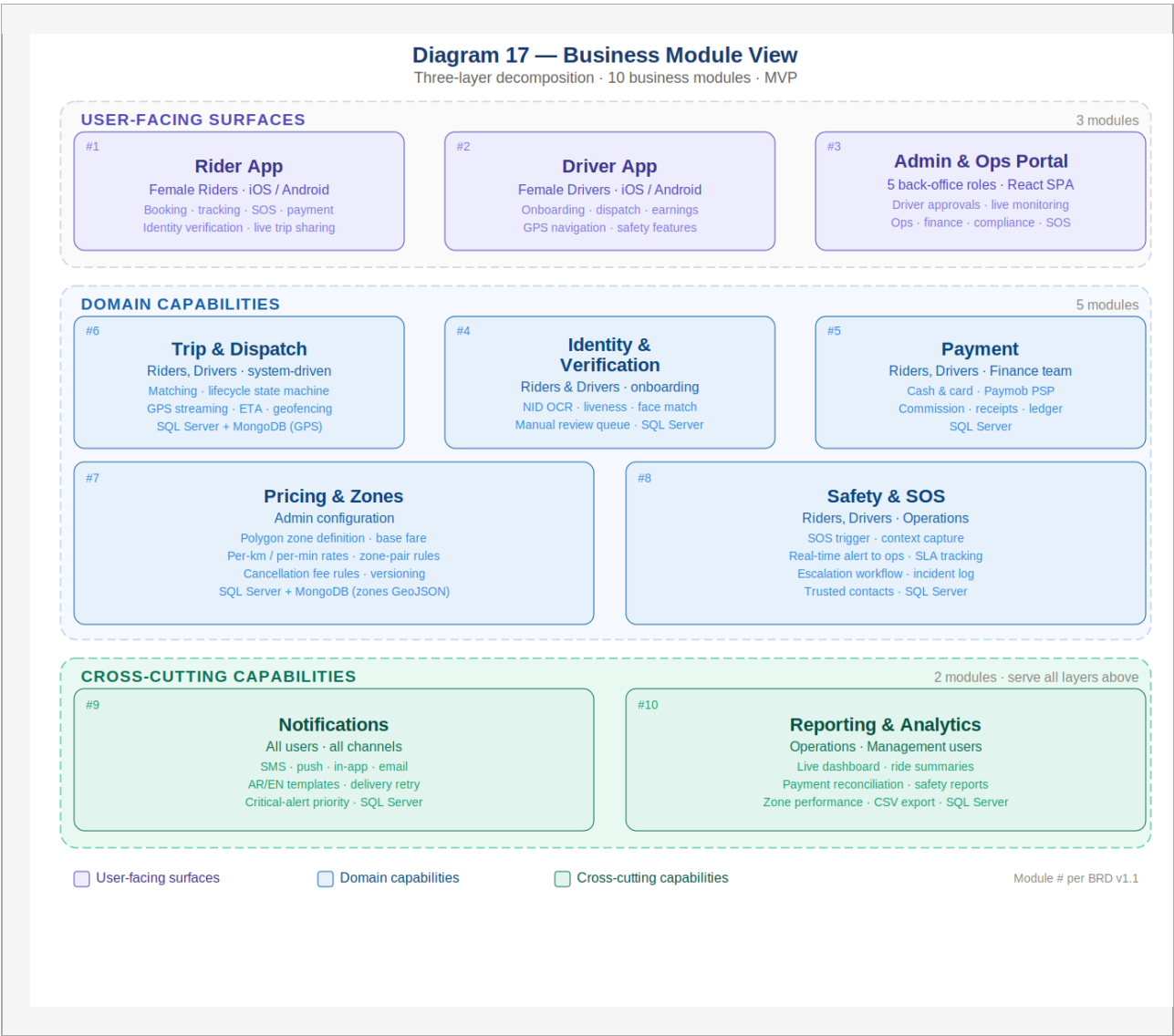


Figure 3. SheDrive Business Module View

Figure 4 below presents the Microservice Decomposition service map, showing the three client surfaces, three Ocelot API gateways, the SignalR Real-Time Hub, and all ten domain microservices grouped by capability area. Each service owns its database exclusively; cross-service communication occurs only through REST APIs (via the respective Ocelot gateway) or asynchronous domain events (via RabbitMQ). The far-right column represents the shared RabbitMQ event bus to which all services publish and subscribe.

[Figure 4. Microservice decomposition — API gateways — event bus]

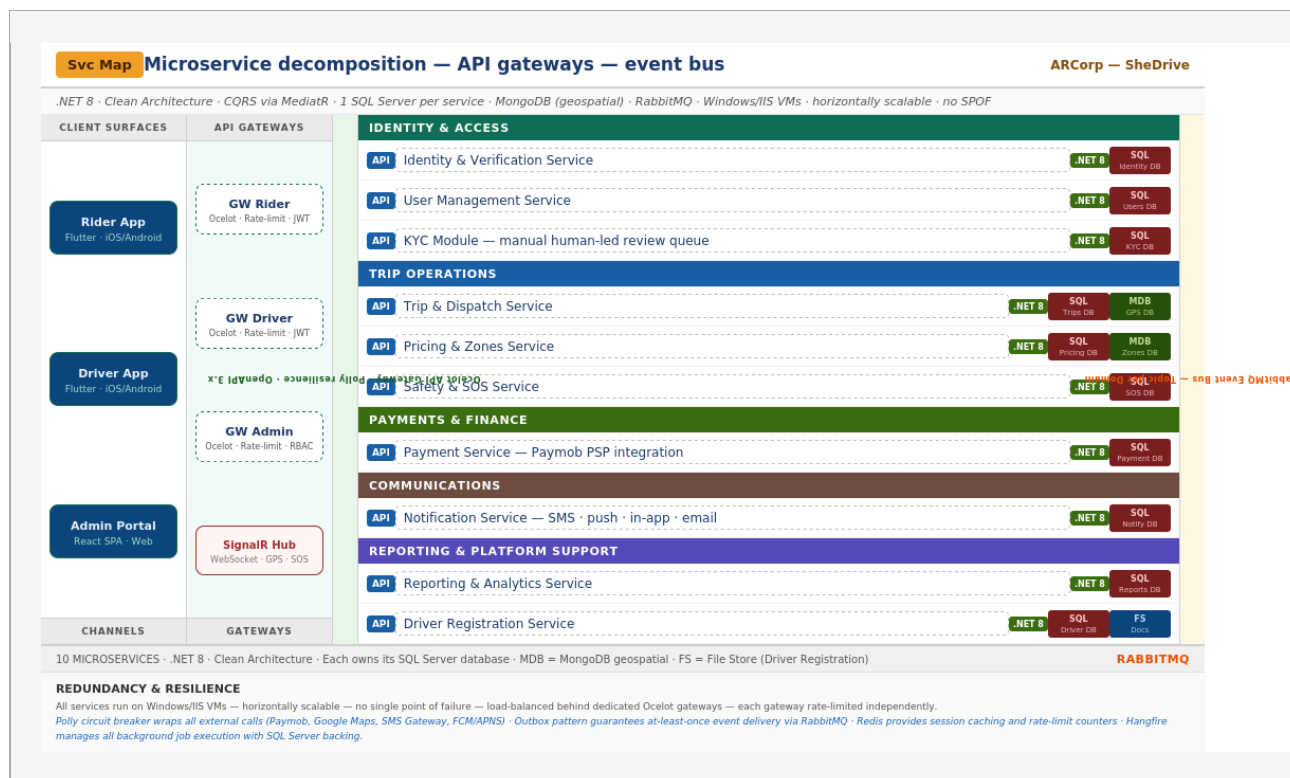


Figure 4. SheDrive Microservice Decomposition — API Gateways — Event Bus

4.2.1 Rider App

Purpose

The Rider App is the primary consumer-facing surface of the SheDrive platform. It provides female riders with the complete journey from account registration and identity verification through ride booking, real-time driver tracking, in-trip communication, payment, and post-trip feedback. The app is designed around the platform's safety-first positioning. SOS, live trip sharing, and trusted contact management are first-class features, not additions. The Rider App is built in Flutter, targeting iOS and Android from a single codebase, with full Arabic (RTL) and English bilingual support.

Primary Users

- Female Riders. Registered, verified female users of any age.
- Children under 15, as declared accompanied passengers, not direct app users.

Key Responsibilities

- OTP-based mobile number registration and secure session management.
- National ID and live selfie submission for identity verification before first trip.
- Map-based ride booking: pickup selection via GPS or manual pin, destination search, fare estimate display.
- Child accompaniment declaration (count and approximate ages) before request submission.
- Payment method selection (cash or digital saved card) per ride.
- Real-time driver tracking on map with continuously updated ETA during matching and en-route states.
- Driver information card on assignment, photo, vehicle details, plate number, and rating.
- In-trip text chat and masked-number voice call to the assigned driver.
- SOS button accessible throughout the active trip lifecycle.
- Live trip sharing link generation and delivery to trusted contacts.
- Trusted contact management, add, edit, remove up to three contacts.

- Ride cancellation with reason selection and cancellation fee disclosure before confirmation.
- Fare breakdown and receipt display on trip completion.
- Cash payment dual-confirmation flow matched with driver confirmation.
- Post-trip driver rating (1–5 stars) and categorical feedback tags.
- Trip history and receipt access.
- Saved places management (home, work, custom).
- Language switching between Arabic (RTL) and English, persisting across sessions.
- In-app FAQs and customer support contact channel.

Cross-Cutting Dependencies

- Identity & Verification Service, for verification status (first-trip verification outcome).
- Trip & Dispatch Service, for ride request, matching status, and lifecycle updates via REST and WebSocket.
- Payment Service, for card tokenisation, payment capture, and receipt retrieval.
- Safety & SOS Service, for SOS trigger and live trip sharing.
- Notification Service, for push and in-app notification delivery.
- API Gateway, as the sole entry point for all REST and WebSocket communication.

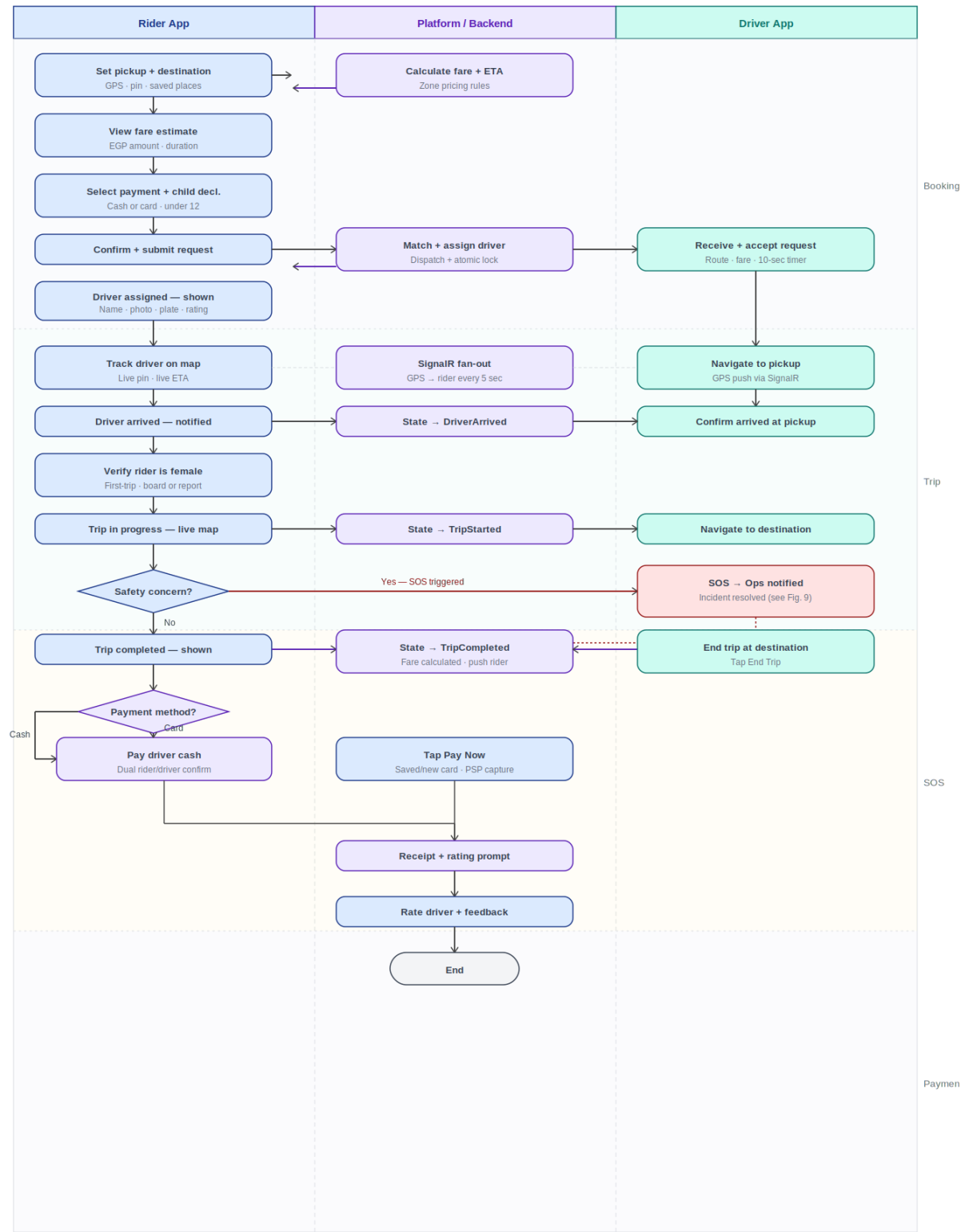


Figure — Rider Journey (Happy Path)

4.2.2 Driver App

Purpose

The Driver App enables verified and admin-approved female drivers to manage their availability, handle incoming ride requests, execute trips using their own eligible vehicles, and access earnings

and safety features. The onboarding journey: personal details (with background-check consent), vehicle details, vehicle photo, licence and vehicle registration documents, profile photo, and admin approval, is fully guided within the app. Once active, the app provides a streamlined ride-execution workflow: receive request, accept or decline, navigate to pickup, start trip, complete trip, and confirm cash payment where applicable. Like the Rider App, the Driver App is built in Flutter with full bilingual RTL support.

Primary Users

- Female Drivers. Licensed, admin-approved female users of any age, operating their own eligible vehicles.

Key Responsibilities

- OTP-based mobile number registration and secure session management.
- National ID submission and live selfie capture for identity verification.
- Driving licence upload (front and back) for format, expiry, and category validation.
- Vehicle registration document upload, front and back.
- Vehicle exterior photo upload for rider identification.
- Background check consent capture before document review.
- Onboarding status tracking across all stages, submitted, under review, approved, rejected.
- Online / offline availability toggle.
- Working zone selection from configured neighbourhoods.
- Incoming ride request popup with rider details, route summary, fare, and accept/decline action within timeout.
- Deep-link navigation to pickup location via device mapping application.
- Arrived at pickup confirmation triggering rider notification and wait window.
- Trip start action initiating real-time location streaming.
- Deep-link navigation to destination during active trip.
- Trip completion action triggering fare calculation and payment processing.
- Cash collection confirmation matched against rider confirmation.
- Digital payment outcome display post-completion.
- Ride cancellation with reason selection; repeated cancellations flagged for review.
- Earnings dashboard, daily, weekly, and per-trip, with platform commission breakdown.
- Cash owed to platform balance with settlement reminders.
- Post-trip rider rating and categorical feedback.
- In-trip text chat and masked-number voice call to the assigned rider.
- SOS button accessible throughout the active trip.
- Language switching between Arabic (RTL) and English.

Cross-Cutting Dependencies

- Identity & Verification Service, for application and document submission and the human verification decision.
- Trip & Dispatch Service, for ride request broadcast, acceptance, and lifecycle management.
- Payment Service, for cash confirmation, commission tracking, and earnings ledger.
- Safety & SOS Service, for SOS trigger.
- Notification Service, for push and in-app notifications.
- API Gateway, as the sole entry point for all REST and WebSocket communication.

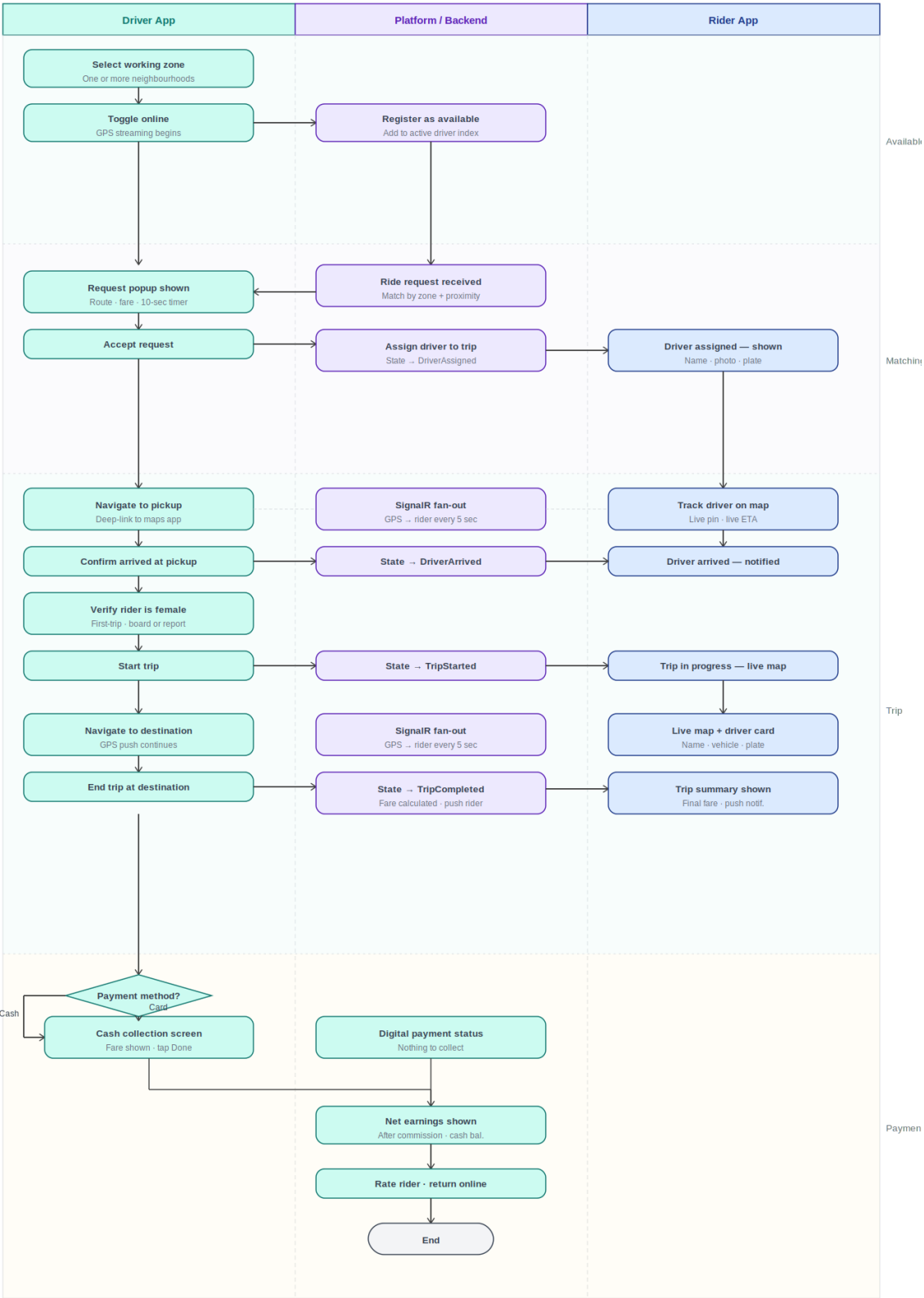


Figure — Driver Journey (Happy Path)

4.2.3 Admin & Operations Portal

Purpose

The Admin and Operations Portal is the unified back-office hub for all platform management activities. It provides differentiated, role-gated access to Operations, Customer Support, Finance, and Compliance teams through a role-based access control model aligned to the seven back-office roles defined in the BRD. The portal is the primary control surface for driver onboarding approval, account management, live ride monitoring, SOS response, pricing and zone configuration, cash reconciliation, and regulatory compliance management. It is implemented as an Angular single-page application, consuming backend APIs over HTTPS and receiving live operational updates via WebSocket.

Primary Users

- Administrator, full platform management access.
- Operations Supervisor, incident and escalation authority.
- Customer Support Agent, user inquiry and dispute handling.
- Finance User, payment reconciliation and cash settlement.
- Compliance User, verification auditing and regulatory reporting.
- Management User, read-only dashboard and KPI access.

Key Responsibilities

- Two-factor authenticated login (TOTP or SMS OTP) with enforced session timeout.
- Role-based access control enforced at both API Gateway and service layers.
- Live operational dashboard: active rides, online drivers, open SOS events, payment failure rate.
- Driver onboarding queue with document review, approval, and rejection workflow with captured reason.
- Driver account management: activation, suspension, deactivation with audit trail.
- Rider account management: search, view, suspension, and blocking.
- Live ride monitoring map and tabular active ride list with zone, status, and payment method filters.
- Complete ride detail view: event timeline, location trail, payment record, and communication log.
- Cancellation review queue with abnormal pattern flagging.
- Incident management workflow: logging, owner assignment, status tracking, and resolution with notes.
- SOS alert queue with real-time audible/visual notification and one-click response actions.
- SOS escalation workflow: initial responder → supervisor → external authority, with time-based triggers.
- Neighbourhood and zone management: polygon drawing, activation, boundary editing.
- Pricing configuration per zone: base fare, per-km rate, per-minute rate, minimum fare, zone-pair rules, peak-hour multiplier.
- Cancellation fee rule configuration per zone.
- Commission rate configuration with effective-date scheduling.
- Cash reconciliation dashboard with overdue settlement identification.
- Per-driver cash ledger and settlement history.
- Manual refund processing with reason capture and audit trail.
- Dispute management workflow.
- Audit log viewer, searchable by actor, action type, and date range.
- Notification template management per channel and language without code deployment.
- Static content management: Terms and Conditions, privacy policy, FAQs.
- Driver document expiry oversight, licence and registration expiry indicators per driver.
- Structured data export for stakeholder and governance reporting.

Cross-Cutting Dependencies

- All backend microservices, the portal is a consumer of every service's APIs.

- Safety & SOS Service, for live SOS queue and escalation actions.
- Reporting & Analytics Service, for all dashboard, report, and export views.
- Notification Service, for template management.
- Audit Trail, all administrative actions produce immutable audit records.
- API Gateway. JWT-validated entry point with RBAC enforcement.

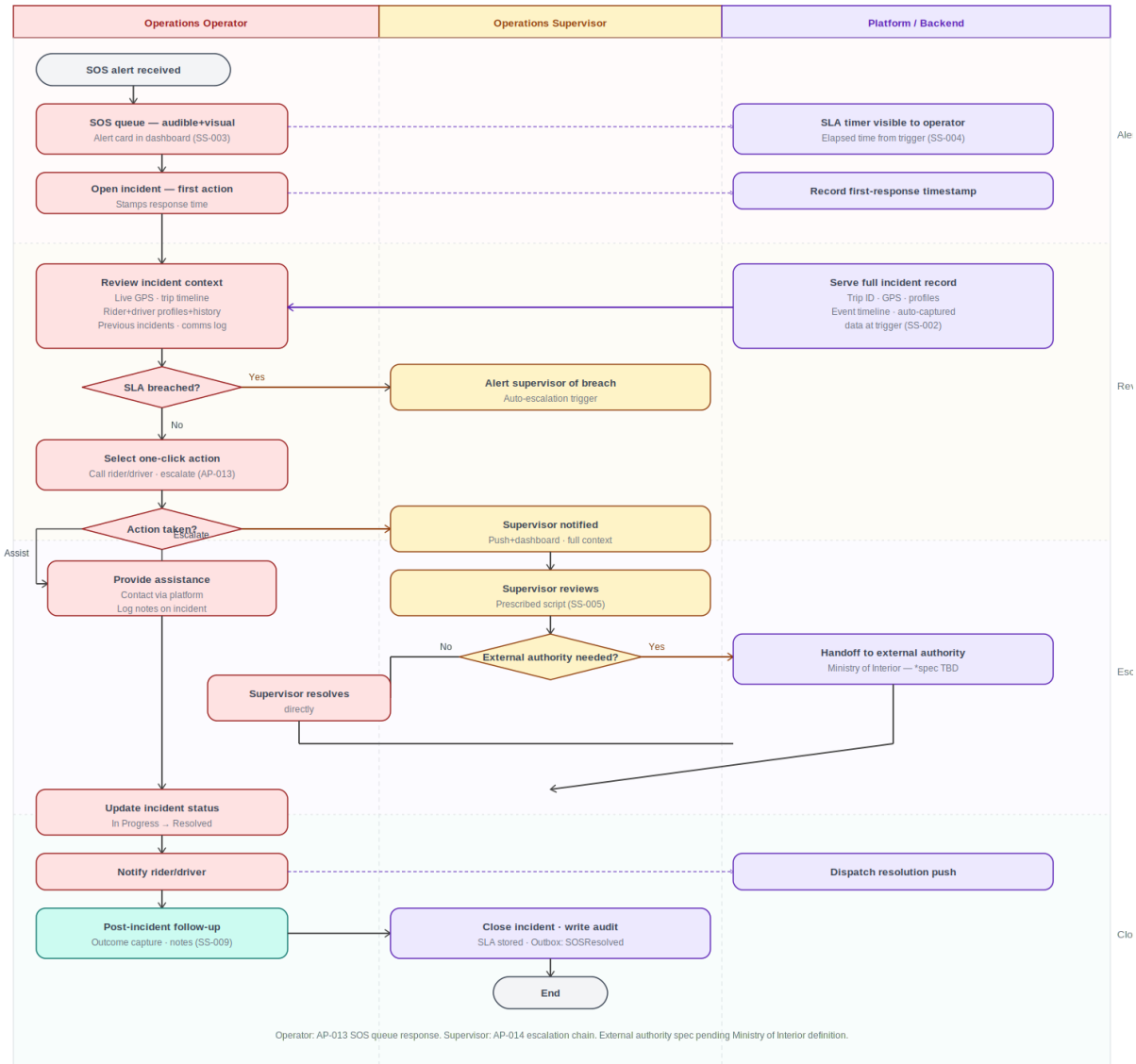


Figure — Admin SOS Handling Workflow

4.2.4 Trip & Dispatch Service

Purpose

The Trip and Dispatch Service is the matching and lifecycle core of the SheDrive platform. Every active ride flows through this service. It receives ride requests from the Rider App via the API Gateway, enforces the platform's operating-hours window, filters candidate drivers based on working zone and proximity, broadcasts requests with atomic acceptance locking, manages the trip state machine through all valid transitions, streams driver location during active trips, and detects geofence events for pickup and drop-off confirmation. The service is the single authoritative source of truth for all trip states.

Primary Users

- Female Riders, indirectly through ride booking interactions.
- Female Drivers, indirectly through ride acceptance and trip management interactions.
- Admin & Operations Portal, for live ride monitoring and trip detail views.

Key Responsibilities

- Ride request intake with operating-hours enforcement, new requests rejected outside the configured daytime window.
- Zone-aware driver eligibility filtering: only drivers whose working zones encompass the pickup location are included as candidates.
- Driver proximity and rating-based matching algorithm selecting the most suitable eligible driver.
- Ride request broadcasting to candidate drivers within a configurable timeout window. Delivery channel: if the Driver App has an active WebSocket connection at dispatch time, the trip offer is pushed directly over that connection (sub-second, no FCM latency). An FCM push notification is always dispatched in parallel as the mandatory fallback for drivers whose app is backgrounded or whose WebSocket session has lapsed. The driver pull-endpoint (#1648) remains the authoritative source of trip-offer details regardless of which channel woke the app.
- Atomic driver acceptance locking preventing double-assignment.
- Decline and timeout reassignment to the next eligible driver.
- No-driver-available retry queue with exponential backoff and rider notification after defined attempts.
- ETA calculation via Google Maps Platform, recalculated continuously as driver location updates.
- Trip lifecycle state machine enforcing valid transitions: RideRequested → SearchingForDriver → DriverAssigned → DriverEnRoute → DriverArrived → TripStarted → TripInProgress → TripCompleted / CancelledBeforeStart / CancelledDuringTrip.
- Real-time driver GPS location streaming to the Rider App and the Admin Operations portal via WebSocket.
- Pickup and drop-off geofence detection triggering relevant lifecycle events.
- Ride cancellation processing with cancellation reason capture and rule evaluation.

Cross-Cutting Dependencies

- Pricing & Zones Service, for fare estimation based on originating zone.
- Payment Service, fare trigger on TripCompleted state.
- Safety & SOS Service, receives SOS triggers during active trips.
- Notification Service, publishes ride lifecycle events for user notifications. Assignment notification delivery channel: when a DriverAssigned event is produced, the Ocelot gateway streaming route immediately pushes the assignment notification to any open Rider App WebSocket session (the matching screen keeps a connection open while polling). This gives sub-second assignment notification without waiting for the next 2–3 second poll cycle or relying on FCM delivery. FCM push is dispatched in parallel as the fallback. The Rider App always re-fetches authoritative trip state on receipt of the WebSocket message or push before rendering the driver card.
- MongoDB, driver live location indexed for geospatial proximity queries.
- SQL Server, trip records, state transitions, and cancellation records.
- Redis Cache, active driver location index and zone polygon configuration.
- Google Maps Platform, routing, ETA calculation.

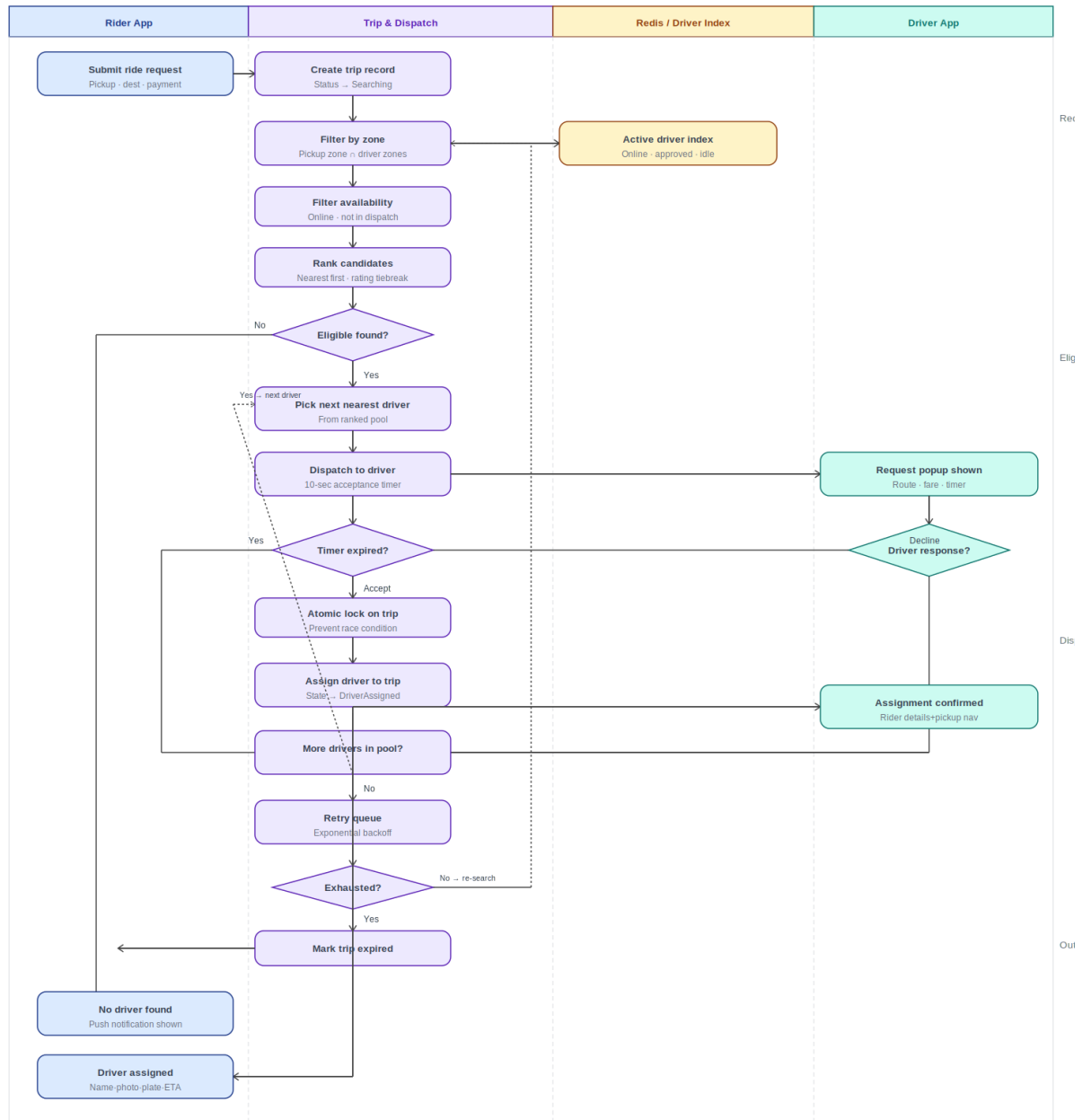


Figure — Ride Request and Driver Assignment

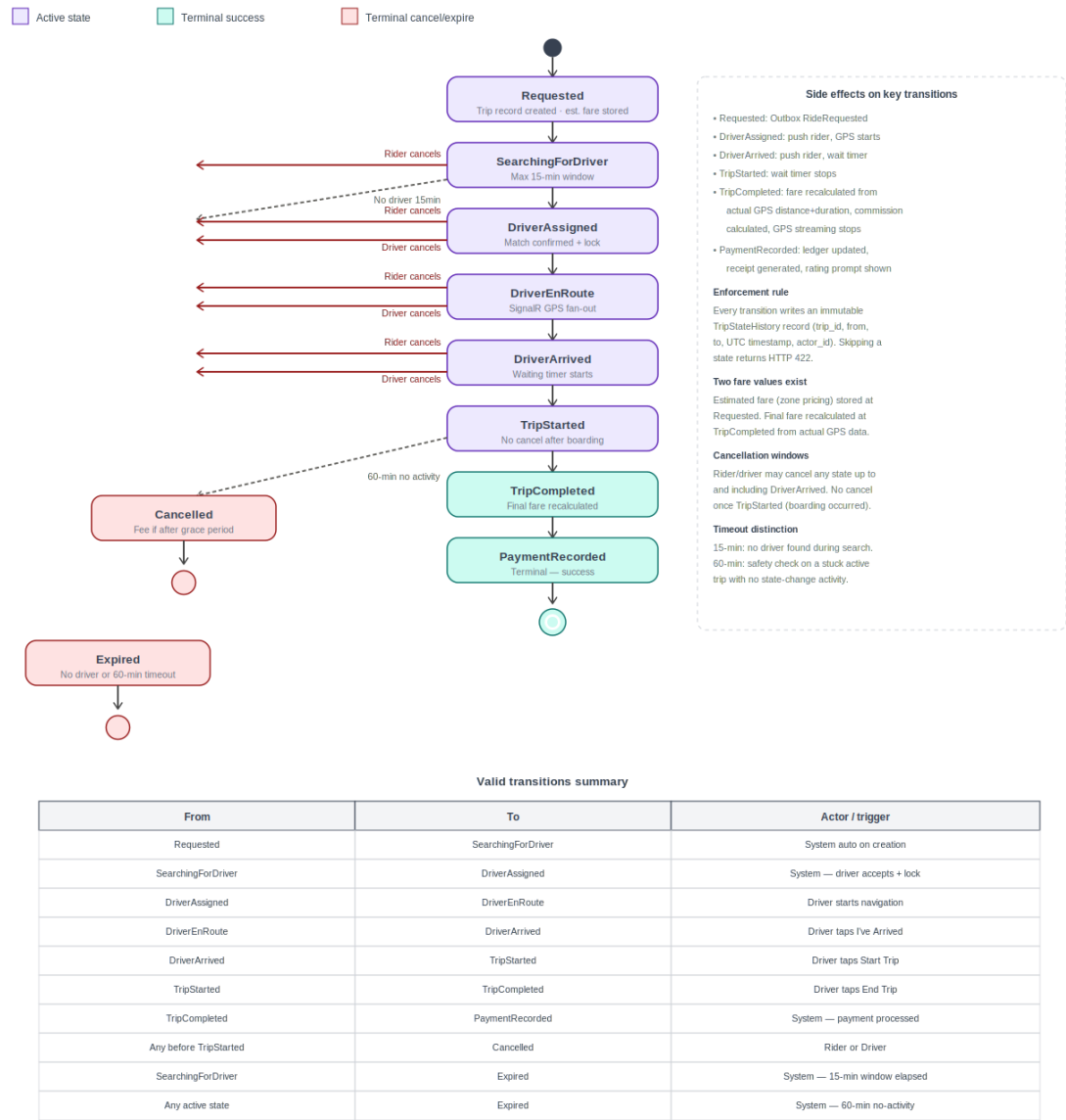


Figure — Trip Lifecycle State Machine

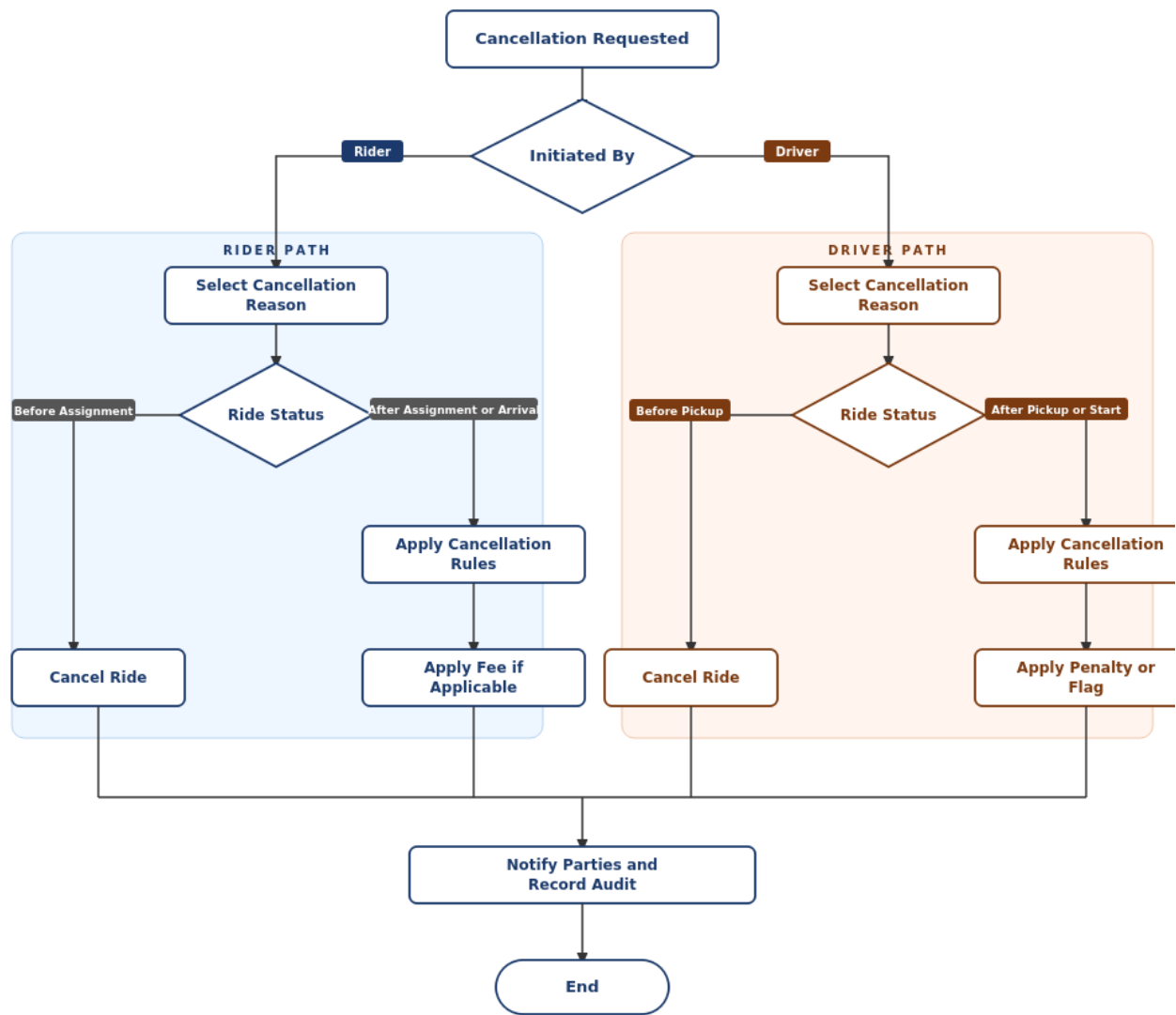


Figure — Ride Cancellation Flow

4.2.5 Identity & Verification Service

Purpose

The Identity and Verification Service is the platform’s gatekeeper for all user onboarding flows in Release 1, in which all verification is manual and human-led; no automated eKYC capability (OCR, liveness detection, or face matching) and no external identity verification provider is used. For Drivers, the service owns the application data model and the human review decision: it receives the submitted personal details (including the 14-digit national ID number with format validation and duplicate detection), documents, vehicle photo, and profile photo, and exposes them to the Admin Portal review workspace where an Administrator explicitly confirms the applicant is female before approval. For Riders, verification is deferred to the first trip: the service computes and owns the `is_first_trip` flag, supports the driver’s visual verification decision at pickup, records the verification outcome, and executes rider account suspension when a gender-mismatch report is submitted. Every decision including approval, rejection, gender-mismatch report, and suspension, is written to the immutable audit trail with reviewer identity and timestamp.

Primary Users

- Female Drivers, submitting application details, documents, and profile photo during onboarding.
- Female Riders, implicitly, through the first-trip verification flow executed by the assigned driver.
- Administrators, reviewing driver applications and making the approve / reject decision.
- Compliance Users, auditing verification decisions and gender-mismatch reports.

Key Responsibilities

- Driver application intake: personal details, 14-digit national ID number format validation, and duplicate registration detection.
- Background-check consent capture and persistence as part of the driver application.
- Document and photo reference management: driving licence (front/back), vehicle registration (front/back), vehicle photo, and profile photo stored in S3 with metadata in SQL Server.
- Admin review workspace backing APIs: pending application queue (search, date-range filter, oldest-first), full application detail with inline document viewing.
- Human eligibility decision recording: explicit female-confirmation flag, approval or rejection with mandatory reason, reviewer identity, and timestamp.
- Driver activation signalling to the User Management and Driver & Vehicle Registration services upon approval.
- is_first_trip flag computation at trip creation (derived from the rider trip history) and exposure to the Driver App via the trip detail API.
- First-trip verification outcome recording: rider verified, or gender-mismatch report with trip cancellation reason gender_mismatch_report.
- Rider account suspension for review on gender-mismatch report, with reinstatement workflow via the Admin Portal.
- Declared child passenger exception handling: per-trip child flag (passenger aged under 15) respected in the verification flow.
- Verification outcome notifications to users (approved, rejected with resubmission guidance).
- Future-readiness: the verification decision interface is provider-agnostic so an automated eKYC step can be attached in later phases without restructuring onboarding flows.

Cross-Cutting Dependencies

- Trip & Dispatch Service, trip creation computes is_first_trip; gender-mismatch cancellation transitions the trip state.
- User Management Service, verification status and suspension state updates.
- Driver & Vehicle Registration Service, application lifecycle coordination and driver activation.
- Notification Service, decision and status notifications to users (push with pull-based reconciliation).
- Admin & Operations Portal, review queue, application detail, and suspension management views.
- AWS S3, document, vehicle photo, and profile photo storage via presigned URLs.
- SQL Server, application records, decisions, and verification status.
- MongoDB, verification decision and gender-mismatch report audit log.

4.2.6 Payment Service

Purpose

The Payment Service handles all financial flows on the SheDrive platform, supporting both digital card and cash payment rails. It integrates with Paymob as the sole PSP for the MVP, manages card tokenisation and saved cards, pre-authorises fares on trip start, captures on completion, records cash payments through dual rider and driver confirmation, maintains the driver cash

balance ledger, calculates and applies platform commission, supports the cash settlement workflow, handles payment failures with retry management, and generates receipts. PCI scope is minimised by delegating all raw card data handling exclusively to Paymob's hosted payment fields.

Primary Users

- Female Riders, indirectly through fare payment and receipt access.
- Female Drivers, for cash confirmation and earnings / commission visibility.
- Finance Users, for cash reconciliation, settlement oversight, manual refunds, and financial reporting.

Key Responsibilities

- Paymob PSP integration for card tokenisation, pre-authorisation, capture, and refunds.
- Saved card management. Paymob tokens stored per rider for future trips.
- Payment pre-authorisation on TripStarted event, based on estimated fare.
- Payment capture on TripCompleted, supporting partial-capture for final fare variance.
- Cash payment recording on dual rider and driver confirmation.
- Commission calculation per completed trip and application to driver earnings or cash balance.
- Per-driver cash balance ledger: running total of cash trips, commission accrued, and amounts owed.
- Cash settlement workflow: settlement event recording and confirmation when driver clears balance.
- Settlement reminder triggers when cash balance exceeds configured threshold.
- Payment failure detection, rider notification, and retry management.

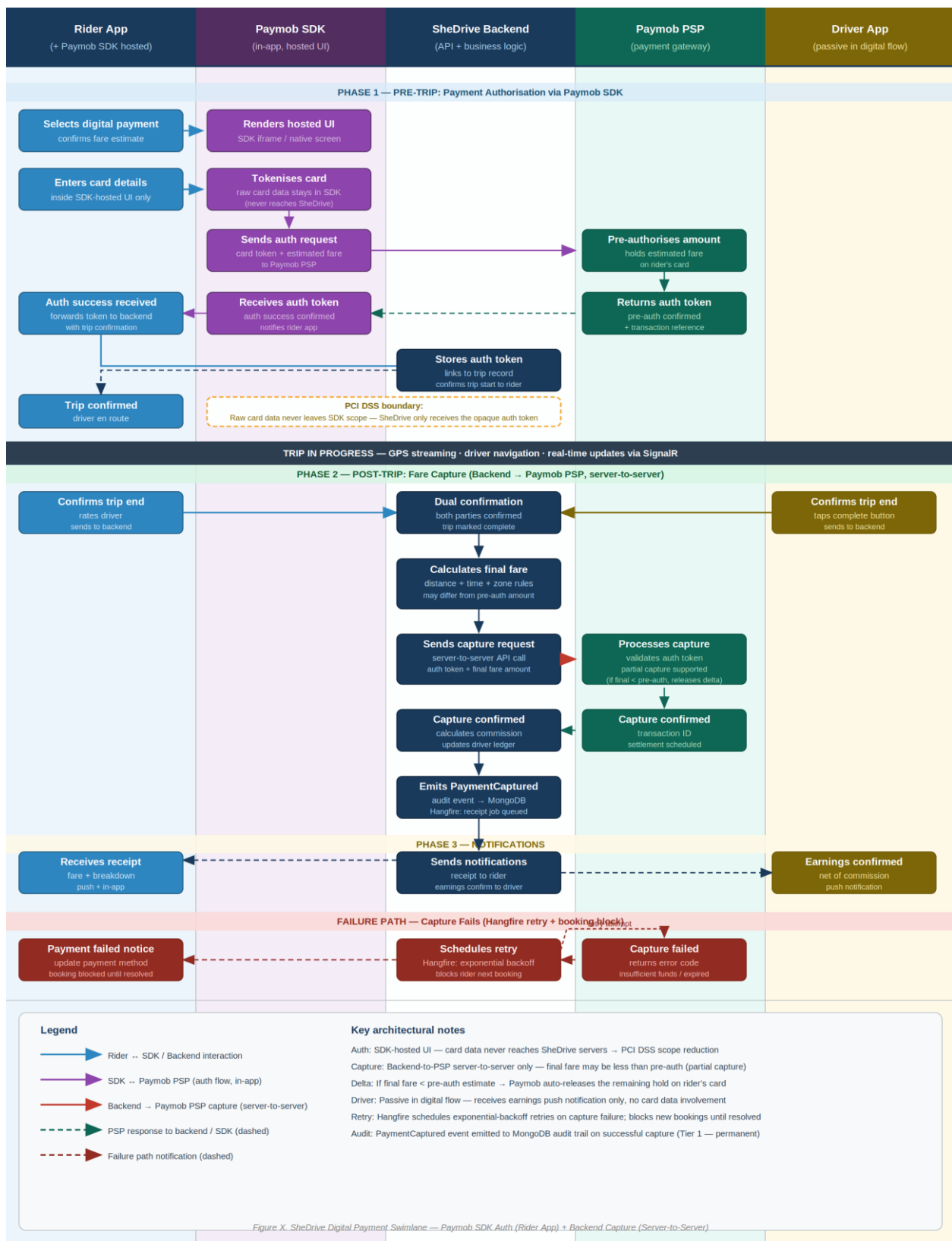


Figure — Paymob Digital Payment Swimlane: SDK Auth (Rider App) + Backend Capture (Server-to-Server)

4.2.7 Safety & SOS Service

Purpose

The Safety and SOS Service operationalises the platform's safety-first positioning as a first-class capability. It provides the SOS trigger surface in both mobile apps, automated data capture on activation, real-time alert delivery to the Operations SOS queue, response SLA tracking, a defined escalation workflow through to external authority handoff, live trip sharing link generation, trusted contact management, and post-incident follow-up support. It also enforces the platform's women-only and minimum-age policies across all platform interactions.

Primary Users

- Female Riders. SOS trigger and live trip sharing.
- Female Drivers. SOS trigger.
- Operations Supervisors. SOS queue management and escalation actions.
- Customer Support Agents, post-incident follow-up.

Key Responsibilities

- Single-tap SOS button accessible to both Rider and Driver during all active trip states.
- Automated capture on SOS activation: user location, trip identifier, Rider and Driver details.
- Real-time SOS alert delivery to the Admin Portal Operations queue with audible and visual notification.
- Response SLA tracking: elapsed time from SOS activation to first operator action, with threshold breach alerts.
- Defined escalation workflow: initial responder → Operations Supervisor → external authority, with time-based triggers and prescribed response scripts.
- Live trip sharing link generation for Rider App, a shareable URL showing real-time location, ETA, and driver information.
- Automatic delivery of live trip link to all configured Trusted Contacts on trip start.
- Trusted contact management: add, edit, remove up to three contacts per Rider.
- SOS incident record creation and status tracking through to resolution.
- Post-incident follow-up workflow: outcome capture and review notes by operations team.
- Women-only and age policy enforcement, all platform access points validated against verified identity.

Cross-Cutting Dependencies

- Trip & Dispatch Service, active trip context during SOS trigger.
- Notification Service. SOS alert to ops queue, trusted contact SMS and push notifications.
- Admin & Operations Portal. SOS queue display, response actions, and escalation workflow.
- SQL Server. SOS incident records and escalation history.
- MongoDB. SOS event detail log and incident timeline.
- Audit Trail, all SOS trigger and resolution events.

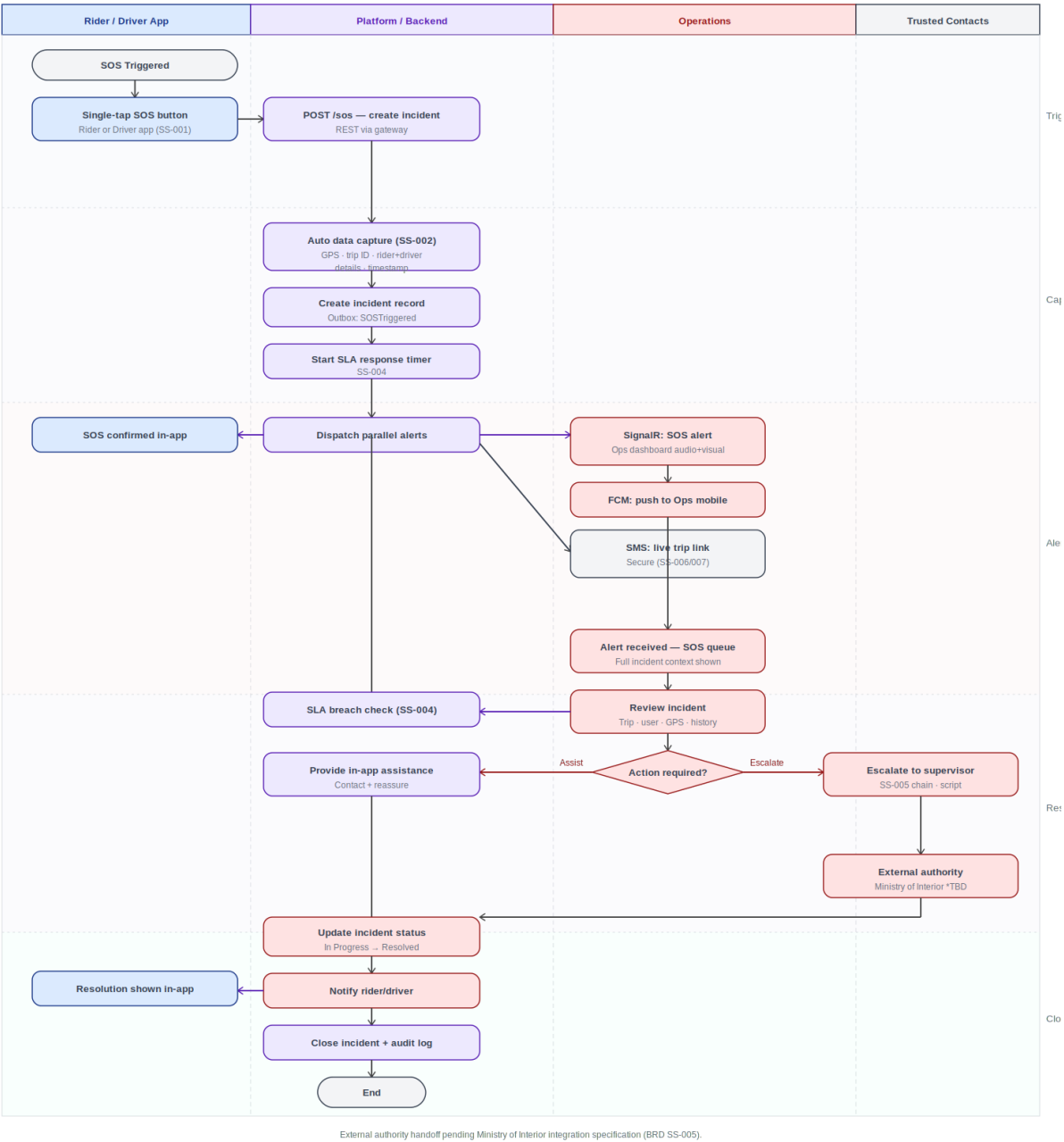


Figure — SOS and Safety Escalation Workflow

4.2.8 Pricing & Zones Service

Purpose

The Pricing and Zones Service enables the platform to configure ride pricing at the granularity of individual neighbourhoods within Greater Cairo. Administrators define zones as map polygons and configure base fare, per-kilometre and per-minute rates, minimum fare, zone-to-zone rules, cancellation fee rules, and optionally peak-hour multipliers per zone. All pricing rule changes are versioned with effective dates for audit and rollback. The service exposes fare estimate endpoints consumed by the Rider App before booking and is the authoritative source for all fare calculations used by the Trip and Dispatch Service.

Primary Users

- Administrators, zone definition and pricing configuration.
- Female Riders, indirectly through fare estimate display before booking.

- Finance Users, pricing rule audit for reconciliation.

Key Responsibilities

- Map-based neighbourhood polygon definition, activation, deactivation, and boundary editing.
- Per-zone base fare configuration.
- Per-zone per-kilometre and per-minute rate configuration.
- Per-zone minimum fare configuration for commercial viability of short trips.
- Zone-to-zone specific pricing rule support.
- Per-zone cancellation fee timing window and amount configuration.
- Optional peak-hour pricing multiplier per zone.
- Fare estimation endpoint: calculates estimated fare and trip duration from pickup zone, estimated distance, and estimated duration.
- Final fare calculation on trip completion using actual distance and duration.
- Pricing rule versioning with effective dates for audit and rollback.
- Cache publication of active pricing rules to Redis for low-latency fare estimates.

Cross-Cutting Dependencies

- Trip & Dispatch Service, fare estimates during ride request and final fare on completion.
- Admin & Operations Portal, zone management and pricing configuration UI.
- SQL Server, zone polygon definitions and pricing rule versions.
- Redis Cache, active pricing rules and zone polygons for low-latency access.
- Audit Trail, all pricing configuration changes.

4.2.9 User Management Service

Purpose

The User Management Service is responsible for the Rider and Driver account lifecycle and for all authentication and authorisation on the platform. It issues JWT tokens on successful authentication, manages session expiry, and enforces RBAC at the API Gateway and service layers. It handles account registration, profile management, account status transitions (active, suspended, deactivated), and trusted contact associations. It is the authoritative source of user identity status consumed by all other services.

Primary Users

- Female Riders, registration, login, profile management, trusted contacts.
- Female Drivers, registration, login, profile management.
- Administrators, account management actions including suspension and deactivation.
- All back-office roles. JWT-based authenticated portal access.

Key Responsibilities

- OTP-based mobile number registration for Riders and Drivers.
- Secure login with JWT issuance and persistent session management with defined expiry.
- Two-factor authentication enforcement for Admin Portal access (TOTP or SMS OTP).
- Profile management: name, photo, email, language preference.
- Account status lifecycle management: active, suspended, deactivated.
- RBAC enforcement: seven back-office roles plus Rider and Driver roles.
- Trusted contact management for Riders (up to three contacts).
- Verification status update linkage with Identity and Verification Service.
- Duplicate mobile number detection at registration.
- Session invalidation on logout and on account suspension.

Cross-Cutting Dependencies

- Identity & Verification Service, verification status linkage.
- API Gateway. JWT validation for all incoming requests.
- Notification Service. OTP delivery and account status notifications.
- SQL Server, user records, session data, RBAC assignments.
- Audit Trail, account status changes and access events.

4.2.10 Notification Service

Purpose

The Notification Service delivers all platform communications across SMS, push, and in-app channels. It manages bilingual templates (Arabic and English) per channel with variable substitution, tracks delivery status with retry and exponential backoff, respects user channel preferences for non-critical messages, and enforces critical-alert priority mode that bypasses device quiet hours for SOS and safety events. The service decouples notification logic from all other domain services, producers publish notification events to RabbitMQ exchanges and the Notification Service consumes them via MassTransit consumer topology and dispatches independently.

Primary Users

- All platform users. Rider, Driver, Administrator, Operations roles.
- Trusted Contacts, external individuals receiving live trip sharing links and SOS notifications via SMS.

Key Responsibilities

- SMS delivery via local Egyptian SMS gateway for OTP, critical alerts, and SOS notifications.
- Push notification delivery via Firebase FCM (Android) and APNS (iOS) for ride lifecycle events.
- In-app notification inbox for Rider, Driver, and Administrator applications.
- Email notifications for Admin and Finance events including reconciliation summaries.
- Bilingual notification template management: Arabic and English per channel with variable substitution.
- Delivery status tracking per notification with retry using exponential backoff.
- User-level channel preferences for non-critical notification types.
- Critical-alert priority mode: SOS and safety alerts bypass quiet hours and Do Not Disturb settings.
- Template management in the Admin Portal without code deployment.
- Dead-letter queue capture of persistently undeliverable notifications for operational review.

Cross-Cutting Dependencies

- Firebase FCM / APNS (external), push delivery.
- Local SMS Gateway (external). SMS delivery.
- All domain services, consume notification events via RabbitMQ.
- Admin & Operations Portal, template management.
- MongoDB, notification delivery log.
- Redis Cache, active notification templates.

4.2.11 Reporting & Analytics Service

Purpose

The Reporting and Analytics Service provides foundational visibility for both daily operations and management decision-making. It maintains read-optimised query models as pre-aggregated Materialised Views, implemented in SQL Server indexed views or a dedicated read schema, refreshed by Hangfire recurring jobs and event-driven consumers (see the Materialised View Pattern in Section 4.3.1). It produces live operational dashboards, periodic summaries, driver performance reports, cancellation analytics, payment reconciliation, safety incident reports, zone performance reports, and a management KPI view. No other service queries a view it did not project. All report data is exportable to CSV and Microsoft Excel. The service also produces the structured trip data export required by Egyptian regulatory authorities.

Primary Users

- Operations Supervisors, live dashboards and operational reports.
- Management Users. KPI dashboard and strategic reports.
- Finance Users, payment reconciliation and cash settlement reports.
- Compliance Users, safety incident reports and regulatory data exports.

Key Responsibilities

- Live operational dashboard: active rides, online drivers, payment success rate, open SOS events, cancellation rate.
- Daily and weekly ride summaries: trip count, GMV, completion rate, and average fare by zone.
- Driver performance reports: trips completed, earnings, cancellation rate, rating, and hours online.
- Cancellation analytics: reasons, timing patterns, and zone and driver correlation.
- Payment reconciliation report: cash and digital split, success and failure rates, outstanding cash balances.
- Safety incident reports: SOS counts, response times, escalation outcomes, and repeat-incident accounts.
- Zone performance reports: trips, GMV, supply versus demand balance, and cancellation rate.
- Management KPI dashboard: GMV, active users, retention, safety indicators, and financial health.
- CSV and Microsoft Excel export for all filtered report views.
- Structured trip data export: trip lengths, average speeds, timing, and geographic distribution, available for governance or stakeholder requests.

Cross-Cutting Dependencies

- All domain services, data consumed via RabbitMQ events and read-optimised query model population.
- SQL Server, source transactional data for financial and trip aggregations.
- MongoDB, source event log data for incident and operational reports.
- Admin & Operations Portal, primary consumption surface for all report views.
- Audit Trail, report access events logged.

4.2.12 Maps & Location Service

Purpose

The Maps and Location Service is the platform's centralised integration layer for all geospatial and routing capabilities. It abstracts the Google Maps Platform dependency behind a clean internal interface, ensuring that no other service depends directly on the external provider's API contract. The service exposes geocoding, routing, ETA calculation, and reverse-geocoding operations to consuming services, primarily Trip and Dispatch and the Rider and Driver Apps. It also maintains the geofence boundary logic for pickup and drop-off zone detection.

Primary Users

- Female Riders, indirectly through map display, pickup/destination search, and ETA.
- Female Drivers, indirectly through navigation deep-links and ETA display.
- Trip & Dispatch Service, for proximity-based matching, ETA, and geofence detection.

Key Responsibilities

- Geocoding and reverse-geocoding: address-to-coordinate and coordinate-to-address resolution.
- Route calculation: distance and estimated travel duration between two points.
- ETA calculation for driver-to-pickup and pickup-to-destination segments.
- Continuous ETA recalculation as driver location updates during active trips.
- Geofence boundary detection: entry into pickup zone triggers DriverArrived; entry into drop-off zone triggers completion prompt.
- Map tile and search API proxying for Rider and Driver App display.
- Anti-Corruption Layer: Google Maps Platform is accessed exclusively through this service's adapter. All other services use internal canonical location contracts.
- Circuit breaker protection on all Google Maps Platform API calls.
- Cache-aside for frequently requested routes and ETA calculations.

Data Store Responsibilities

The three data stores used by this service have distinct and non-overlapping responsibilities. MongoDB holds all live driver location documents, each written on every GPS update, indexed with a 2dsphere index for sub-millisecond nearest-neighbour queries, and automatically expired via a TTL index (24 hours) so stale location records are never retained. MongoDB does not store zone boundaries or pricing rules. SQL Server is the persistent master store for zone polygon definitions and all pricing rule versions, written by the Pricing & Zones Service and treated as configuration data: read infrequently, changed only through deliberate Admin Portal actions, versioned and auditable. Redis acts as the runtime cache for both: active driver location snapshots are pushed to Redis for the lowest-latency matching path, and zone polygon GeoJSON is loaded into Redis on first access (and on cache-invalidation after an Admin update) so that point-in-polygon checks and fare-estimate zone lookups do not touch SQL Server on the hot path.

Fare Calculation and Google Maps Platform

Google Maps Platform does not calculate fares. Its role is strictly to supply routing inputs: the Directions / Routes API returns the road distance in kilometres and the estimated travel duration in minutes between two coordinates. The Maps & Location Service adapter calls Google, normalises the response into a canonical RouteResult (distance, duration, polyline), and passes it to the Pricing & Zones Service. The Pricing & Zones Service owns the fare formula: $\text{BaseFare} + (\text{DistanceKm} \times \text{PerKmRate}) + (\text{DurationMin} \times \text{PerMinuteRate})$, subject to the zone MinimumFare and any active PeakMultiplier. All rate values are configured per zone by Administrators in the Admin Portal and cached in Redis. No fare logic exists inside the Maps & Location Service or within Google Maps Platform.

Client Interaction Model

Mobile (Rider App and Driver App)

Both mobile apps use the Google Maps SDK for Flutter for all client-side map rendering. Map tiles, camera control, live marker animation (driver pin moving toward the rider), and route polyline display all run entirely on-device at no per-call billing cost. The Google Maps API key used by the mobile SDK is a client-restricted key scoped to map tile rendering only; it is not the server-side key and cannot be used to call billable computation APIs. When a Rider types an address or drops a pin to set a pickup or destination, the Flutter app does not call Google directly: it calls the Maps & Location Service backend endpoint, which proxies the Geocoding or Places Autocomplete API call server-side, keeps the billable API key out of the client, and applies the Redis cache-aside before making an outbound call. Navigation for the Driver is provided via a deep-link hand-off to the device native maps application (Google Maps or Apple Maps), so turn-by-turn navigation cost and complexity sits outside the platform boundary.

Back Office (Admin Portal)

The Admin Portal uses the Google Maps JavaScript API (not the Flutter SDK) only for the zone polygon editor in the Pricing & Zones configuration screens, where Administrators draw, resize, and label neighbourhood boundary polygons that are then stored as GeoJSON in SQL Server. Outside the polygon editor, the Admin Portal does not render Google map tiles. The live operations map on the Admin dashboard (showing active driver positions and active trip routes) is served entirely from platform data: driver coordinates are sourced from MongoDB via the backend API, and trip route polylines are stored in SQL Server trip records. This means the live operations view carries no Google Maps billing cost and remains operational even if the Google Maps circuit breaker is open.

Cross-Cutting Dependencies

- Google Maps Platform (external), via integration adapter.
- Trip & Dispatch Service, primary consumer for matching, ETA, and geofence events.
- Redis Cache, cached route and ETA results for high-frequency pickup corridors.
- Rider App and Driver App, map display and navigation deep-links via API Gateway.

4.2.13 Driver & Vehicle Registration Service

Purpose

The Driver and Vehicle Registration Service manages the structured onboarding and registration of drivers and their vehicles on the SheDrive platform. It acts as the authoritative record for driver profiles, vehicle eligibility, document submission status, and registration lifecycle state. While the human verification decision is owned by the Identity and Verification Service, the registration service owns the end-to-end onboarding workflow, coordinating document collection, eligibility rule enforcement (supported vehicle types, registration validity), and the admin approval queue that activates or rejects a driver for platform operation. This separation keeps fleet and registration concerns cleanly decoupled from identity verification concerns.

Primary Users

- Female Drivers, submitting documents and tracking onboarding status.
- Administrators, reviewing the onboarding queue and approving or rejecting driver applications.
- Compliance Users, auditing vehicle eligibility and registration records.

Key Responsibilities

- Driver profile creation on registration: personal details, contact information, and language preference.
- Vehicle record creation: make, model, colour, year of manufacture, plate number.
- Vehicle eligibility enforcement: supported vehicle types (Sedan, SUV, Minivan), model year capture, and valid vehicle registration.
- Document upload orchestration: driving licence (front and back), vehicle registration (front and back), vehicle exterior photo, and driver profile photo.
- Background check consent recording and submission coordination with the Identity and Verification Service.
- Onboarding status tracking across all stages: draft, documents submitted, under review, approved, rejected.
- Admin onboarding queue: filterable by status, zone, and submission date; document viewer per applicant.
- Approval workflow: admin approves or rejects with mandatory reason captured and persisted.
- Rejection notification with reason, allowing driver to resubmit corrected documents.
- Driver activation on approval: profile status set to active, driver becomes eligible to go online.

- Vehicle re-verification triggers: registration and licence expiry indicators surfaced for operational follow-up.
- Working zone assignment: driver selects one or more configured neighbourhoods as their eligible operating zones.
- On-ground check record attachment per driver (optional operations checklist), surfaced to the Compliance role.

Cross-Cutting Dependencies

- Identity & Verification Service, coordinates the human verification decision and consent records, and signals approval outcomes to update registration status.
- User Management Service, driver account creation and status linkage.
- Admin & Operations Portal, onboarding queue, document review, and approval workflow.
- Notification Service, onboarding status notifications to drivers.
- SQL Server, driver profiles, vehicle records, document submission records, and approval history.
- MongoDB, onboarding event audit log.
- Audit Trail, all approval and rejection decisions.

Manual Verification Workflow and Tooling

All driver verification in Release 1 is manual and human-led. There is no electronic KYC (eKYC): no OCR, no liveness detection, no face matching, and no external identity verification provider. The registration service operationalises this through purpose-built Admin Portal tooling that makes the human review process consistent, queue-managed, and fully audited. The following responsibilities cover the complete verification workflow, which was previously described as a separate KYC module but is consolidated here as there is no eKYC and no separate automated verification component to justify a standalone service.

- Pending driver application queue: name, phone, and submission date per row, oldest first, with name/phone search and date-range filtering.
- Application detail review screen: personal details, vehicle details, all four documents viewable inline (image render and PDF preview with lightbox), vehicle photo, and profile photo.
- Mandatory human gender confirmation: the Approve dialog requires the explicit confirmation “I confirm this applicant is female” before approval can proceed.
- Rejection with mandatory written reason, delivered to the driver with resubmission guidance.
- On-ground check support: optional operations checklist record per driver (vehicle inspected, identity sighted) attachable to the application record.
- Decision audit records: every approval, rejection, and confirmation checkbox is written to the immutable audit trail with actor identity and timestamp.
- Push notification of decisions with pull-based status reconciliation on next app open (approval is never dependent on push delivery).
- Upgrade-readiness: the verification decision interface is provider-agnostic so an automated eKYC step (OCR, liveness detection, face matching) can be attached in a future release without restructuring the onboarding flow or the Admin Portal review screens.

4.2.14 Platform / Support Capabilities

Purpose

4.2.14 Platform / Support Capabilities

The following capabilities are not business modules in the BRD sense but are essential platform concerns that cut across all functional blocks. They are implemented as shared infrastructure components within the backend, enforced consistently across every microservice, and operated as first-class platform assets with their own configuration, monitoring, and ownership.

4.2.14.1 API Gateway & Communication Layer

The API Gateway tier is implemented as three independent Ocelot instances, co-hosted on IIS on the same Presentation & API Gateway VMs, each serving a distinct client surface: Rider Gateway (rider-api.shedrive.com), Driver Gateway (driver-api.shedrive.com), and Admin Gateway (admin-api.shedrive.com). All three instances sit behind the shared load balancer, routed by subdomain. Each instance has its own Ocelot JSON configuration file, its own IIS application pool, and its own deployment pipeline. No client contacts a backend service directly. The three gateways handle REST API traffic only. WebSocket real-time communication is handled by a separate dedicated SignalR Real-Time Hub (Section 4.2.14.6); the Ocelot WebSocket proxy is not used.

Rider Gateway (REST only)

- JWT authentication enforced on every request, scoped to the Rider role. OTP issuance and login endpoints subject to strict per-IP and per-device rate limits hardened against credential-stuffing and OTP-farming.
- Mobile-optimised routing: paginated and minimal-payload API responses designed for variable Egyptian mobile network conditions.
- Idempotency-key propagation: the gateway stamps and forwards idempotency keys on all state-modifying requests so mobile client retries are safe.
- Operating-window enforcement on ride-booking endpoints: requests outside the configured daytime window are rejected at the gateway before reaching any service.

Driver Gateway (REST only)

- JWT authentication enforced on every request, scoped to the Driver role. OTP and login endpoints carry the same per-IP and per-device rate limits as the Rider Gateway.
- Separate rate-limit profile tuned for driver trip-action frequency: trip acceptance, arrival confirmation, trip start, and completion endpoints each carry individually configured limits independent of the Rider Gateway profile.
- Operating-window enforcement on the driver availability-toggle endpoint: drivers cannot go online outside the configured operating window.
- Idempotency-key propagation on all state-modifying driver requests, identical mechanism to the Rider Gateway.

Admin Gateway (REST only)

- JWT authentication with role-scope assertion: the gateway validates that the token carries one of the eight recognised back-office roles before routing. Requests from unknown or mobile roles are rejected at the perimeter.
- IP-allowlist or VPN-enforcement can be applied to this instance in Production without affecting the Rider or Driver Gateways.
- Session-based rate limits tuned for human operator throughput rather than mobile burst patterns.

Cross-Cutting Responsibilities (all three instances)

- Correlation ID issuance: a unique correlation identifier is generated per inbound request and propagated through all downstream calls, log events, audit records, and distributed traces.
- Response compression (gzip) on all routes to minimise payload size.
- TLS enforcement on all inbound connections. Each instance exposes its own health endpoint scraped by Prometheus.

Architecture Decision: Three Instances on Shared VMs

Rider, Driver, and Admin gateways are intentionally realised as three independent Ocelot instances rather than a single instance with logical routes. The primary motivation is independent configuration and independent deployment: a rate-limit change on the Driver Gateway is deployed without touching the Rider or Admin configuration, and a misconfiguration in one instance cannot affect the others. The instances share the same IIS host and VM layer at MVP, preserving infrastructure simplicity. VM-level extraction per gateway instance is the defined scaling path and requires no backend service contract changes. WebSocket communication is explicitly excluded from all three gateway instances; it is handled solely by the SignalR Real-Time Hub (Section 4.2.14.6), keeping the two channels architecturally separate.

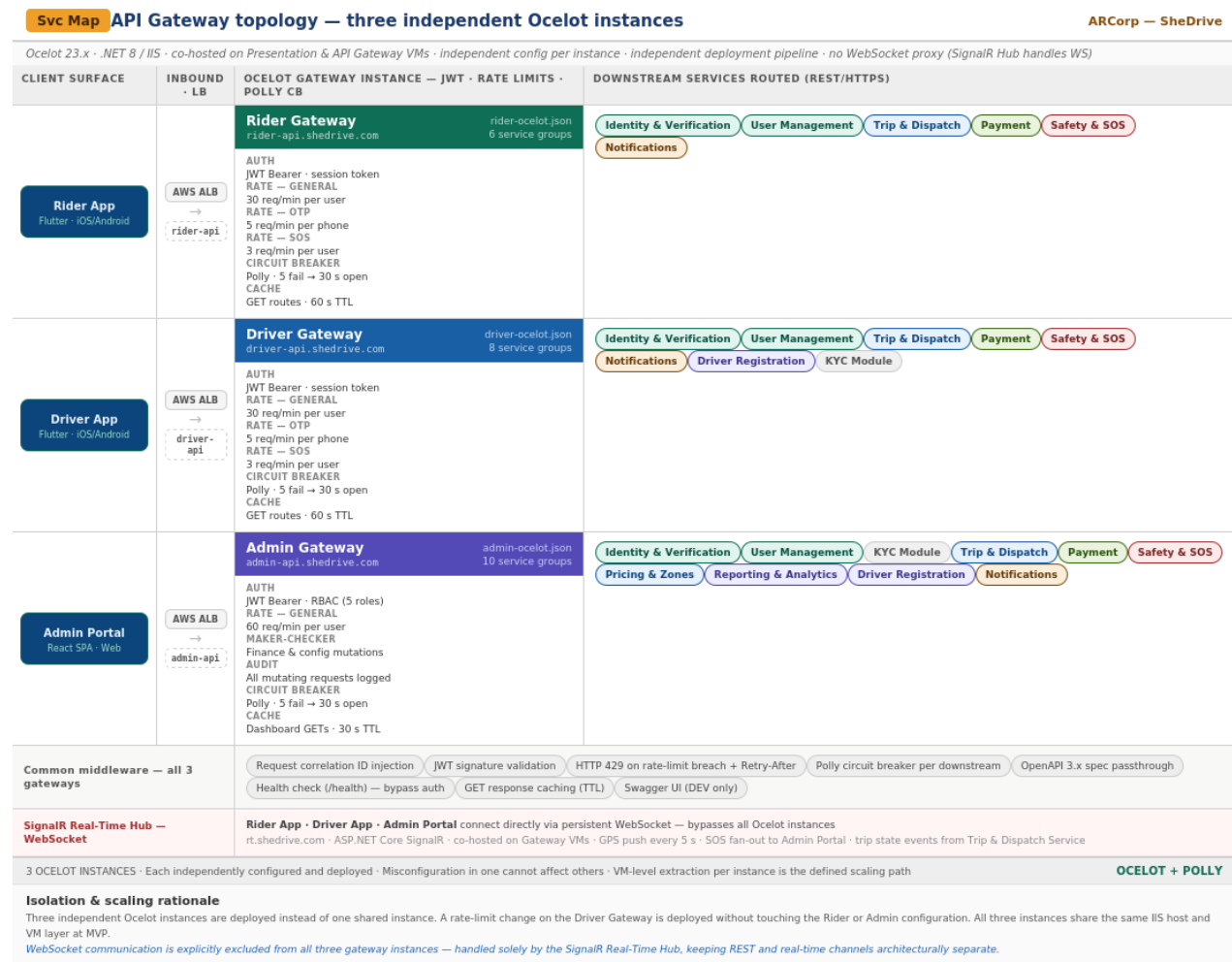


Figure 5. SheDrive API Gateway Topology — Three Independent Ocelot Instances

4.2.14.2 Background Jobs & Scheduling (Hangfire)

Hangfire is the platform-wide background processing framework for all .NET services, backed by SQL Server job storage and surfaced through the built-in Hangfire dashboard (access-restricted to engineering and operations roles). It executes all time-driven and deferred platform behaviour, cleanly separated from request-driven behaviour:

- Recurring (cron-style) jobs: cash settlement reminder sweeps, driver document expiry scans (driving licence, vehicle registration), report pre-aggregation, stale session and token cleanup, and notification retry sweeps.
- Note: Transactional Outbox draining is not a Hangfire responsibility. MassTransit’s built-in delivery service (IHostedService) polls the outbox, publishes to RabbitMQ, and handles

retry with backoff natively. Hangfire job scope is limited to the application-level scheduled jobs listed above.

- Delayed jobs: no-driver-available retry scheduling, payment retry windows, and OTP cooldown expiry handling.
- Fire-and-forget jobs: receipt generation, audit fan-out, and non-blocking post-trip processing.
- Reliability: automatic retries with configurable policies, job persistence across service restarts and deployments, and per-queue worker isolation so a slow job class cannot starve critical jobs.

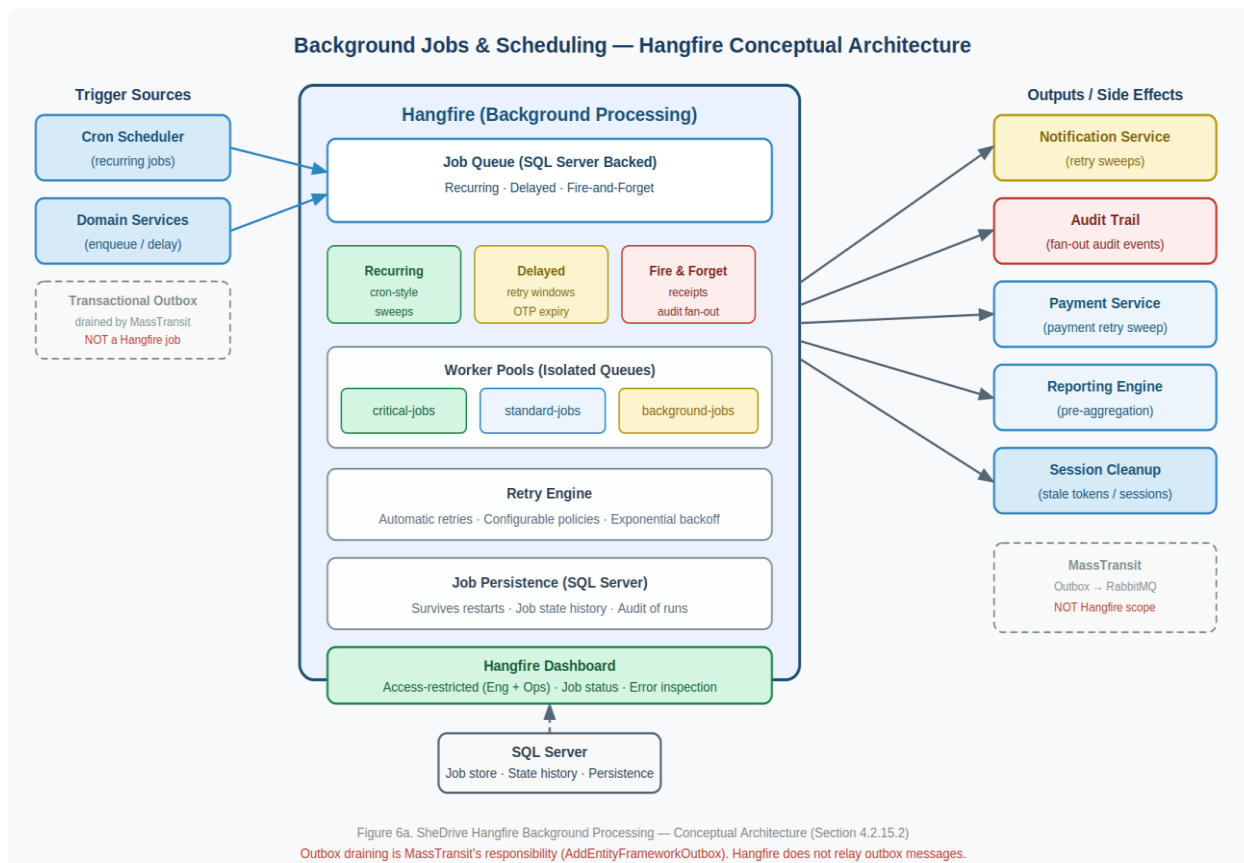


Figure 6a. SheDrive Hangfire Background Processing — Conceptual Architecture (Section 4.2.14.2)

4.2.14.3 Audit Trail

The Audit Trail is the platform's primary accountability mechanism, an append-only, access-controlled record of every significant business and security decision. It is distinct from the operational log (Seq): the operational log captures all technical activity at high volume with a 180-day rolling retention; the Audit Trail captures a small, curated set of high-value business events, retained indefinitely, and queryable by authorised roles through the Admin Portal Audit Log Viewer. An audit event is produced for every sensitive action including financial, identity-verification, safety, administrative, and account-status, regardless of which service originates it.

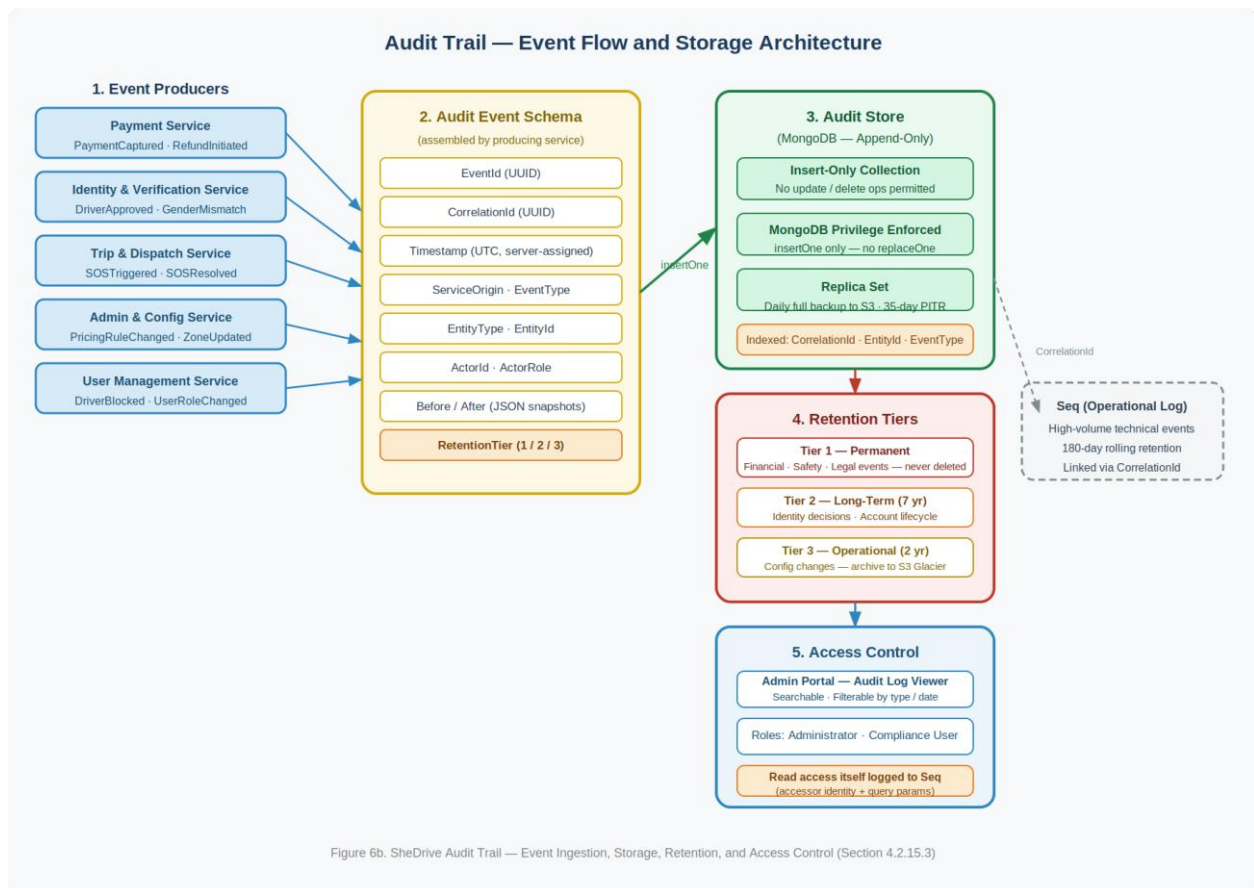


Figure 6b. SheDrive Audit Trail — Event Ingestion, Storage, Retention, and Access Control (Section 4.2.14.3)

Audit Event Schema

Every audit event written to the MongoDB audit collection carries the following mandatory fields:

Field	Type	Purpose
EventId	UUID	Globally unique event identifier.
CorrelationId	UUID	Shared with the technical log for cross-referencing the full request trace.
Timestamp	UTC datetime	Server-assigned at the point of event creation; never client-supplied.
ServiceOrigin	string	The microservice that produced the event (e.g., PaymentService, IdentityService).
EventType	enum	Categorised event code (e.g., PaymentCaptured, DriverApproved, GenderMismatchReport, SOSTriggered, ConfigurationChanged).
EntityType	string	The type of business entity affected (Trip, Driver, Rider, PricingRule, Zone, SOSIncident).
EntityId	string	The identifier of the affected entity.
ActorId	string	The authenticated user or system identity that performed the action.
ActorRole	string	The RBAC role of the actor at the time of the action.
Before	JSON (nullable)	Previous state snapshot for configuration-change and account-status events. Null for creation events.
After	JSON (nullable)	New state snapshot. Null for deletion or revocation events.

Field	Type	Purpose
AdditionalContext	JSON (nullable)	Event-specific payload (e.g., rejection reason text, GPS coordinates for SOS trigger, mismatch report note).
RetentionTier	enum (1 / 2 / 3)	Retention tier assigned at event creation time, determines the archival and purge policy (see Section 4.4.4): Tier 1 = Permanent (financial, safety, legal events); Tier 2 = Long-term configurable retention (identity decisions, account changes); Tier 3 = Operational configurable retention (admin configuration changes).

Audit Event Categories

Category	Event Types	Key Content
Financial	PaymentCaptured, PaymentFailed, RefundInitiated, RefundCompleted, CommissionCalculated, CashConfirmed, CancellationFeeCharged, CashSettlementRecorded.	Every financial transaction produces an audit event with the Before/After payment state, amount, currency, method, and actor.
Identity Verification	DriverApplicationSubmitted, DriverApproved, DriverRejected, FirstTripVerificationPassed, GenderMismatchReported, RiderAccountSuspended, RiderAccountReinstated.	Decision events include the FemaleConfirmedFlag, actor identity, rejection reason, and all document references. Gender-mismatch events include the trip reference and the reporting driver.
Safety & SOS	SOSTriggered, SOSAcknowledged, SOSEscalated, SOSResolved.	Each event carries the GPS snapshot, trip reference, escalation level, actor identity, and elapsed time from trigger. The full SOS timeline is queryable from the audit store alone.
Administrative Configuration	PricingRuleChanged, ZoneBoundaryUpdated, OperatingHoursChanged, CommissionRateChanged, NotificationTemplateUpdated.	Before and After snapshots of the full rule/zone/template object. Actor identity and timestamp are mandatory. These events are the source of truth for configuration rollback.
Account Status	DriverActivated, DriverDeactivated, DriverBlocked, RiderBlocked, UserRoleChanged.	Includes the reason (human-entered for blocks), actor identity, and prior status.

Storage, Access, and Retention

- The audit store is an append-only MongoDB collection. No update or delete operation is permitted by any role or service, this is enforced at the application layer (no update/delete code paths exist) and at the MongoDB user-privileges level (the audit write role has insertOne only; no updateOne, replaceOne, or deleteOne).
- Access to the audit store is restricted to Administrator and Compliance User roles through the Admin Portal Audit Log Viewer. All read access to audit records is itself logged to the operational log with the accessor identity and query parameters.
- Audit events are retained indefinitely. The rationale: payment disputes, safety incidents, and accountability for verification decisions can surface months or years after the originating event; the append-only collection grows at a bounded, predictable rate (only

significant business decisions; not API calls); and indefinite retention is achievable at negligible cost on MongoDB with the event schema defined above.

- The audit MongoDB replica set is backed up daily (full) to S3 with a 35-day rolling window and point-in-time recovery. The backup set is separate from the operational data backup.
- Audit data is excluded from data-erasure requests where the record is the only evidence of a financial, safety, or verification event, the record is retained in anonymised or pseudonymised form in that case, with the data-subject linkage removed.

Retention Tiers

Not all audit events carry equal long-term value. Each event is stamped with a `RetentionTier` field at creation time. The tier determines the archival and purge policy applied to that event independently of other events in the same collection. The three tiers are defined in Section 4.4.4 and summarised below:

Tier	Retention	Event Types	Rationale
Tier 1	Permanent (never deleted)	PaymentCaptured, RefundInitiated, CommissionCalculated, CancellationFeeCharged, CashSettlementRecorded, SOSTriggered, SOSResolved, GenderMismatchReported, RiderSuspended	Financial liability, safety accountability, and legal evidence have no bounded statute of limitations on this platform. Loss or alteration of these records is unacceptable under any circumstance.
Tier 2	Long-term (7 years, configurable)	DriverApproved, DriverRejected, DriverActivated, DriverDeactivated, DriverBlocked, RiderAccountBlocked, UserRoleChanged	Identity decisions and account lifecycle changes must be traceable beyond the lifetime of an active account for dispute resolution and governance review. 7 years is the default; configurable without code change.
Tier 3	Operational (2 years, configurable)	PricingRuleChanged, ZoneBoundaryUpdated, OperatingHoursChanged, CommissionRateChanged, NotificationTemplateUpdated	Configuration change history supports audit and rollback but its operational value diminishes after 2 years. Archival to S3 Glacier after 2 years; deletion after configurable cold-storage period.

4.2.14.4 Logging & Tracing

Structured logging and distributed tracing are the operational visibility layer for day-to-day investigation and post-incident analysis. They complement the Audit Trail (Section 4.2.14.3). While the Audit Trail records what business decision was made and by whom, logs and traces record how the platform processed the request, service calls, timings, exceptions, and infrastructure events, at a technical level.

Structured Logging. Mandatory Fields

All .NET services emit structured JSON log events via Serilog. Every log event carries the following mandatory fields regardless of severity:

Field	Purpose
CorrelationId	UUID shared across the full request chain from gateway to all downstream calls. The primary key for cross-service log correlation.
RideId	Populated for all events within an active ride context. Null for non-ride operations.
ServiceName	The originating microservice (e.g., TripDispatchService, PaymentService).
Environment	DEV / TEST / SIT / UAT / PROD, prevents cross-environment log confusion in Seq.
Severity	Verbose Debug Information Warning Error Fatal.
Timestamp	UTC, server-assigned; client timestamps are not used.
UserId (nullable)	Populated where an authenticated actor is present. Absent for system/background job events.

Log Levels. Usage Policy

Level	Usage
Verbose / Debug	Development and test environments only. Never enabled in Production. Used for detailed request/response tracing during feature development.
Information	Normal operational events: ride state transitions, payment stage changes, driver status changes, scheduled job executions. The default level for Production.
Warning	Recoverable unexpected conditions: a retry was needed, a circuit breaker is in half-open state, a queue consumer is lagging, a health check dependency is slow.
Error	A request failed due to an unhandled exception or an integration call that exhausted all retries. Always paired with a CorrelationId and a stack trace.
Fatal	Service cannot continue, database connection lost, Vault unreachable at startup. Triggers an immediate Grafana alert and an on-call page.

Strict Data Exclusions

The following categories of data must never appear in any log event at any severity level:

- PII: full mobile numbers (last 4 digits only where needed for operational reference), national ID numbers, full names in searchable log fields.
- Financial: Primary Account Numbers (card numbers), raw PSP authentication tokens.
- Identity documents and photos: S3 object keys are acceptable; document content is never logged.
- Passwords, JWT signing keys, Vault tokens, or any credential material.
- Free-text user input fields (e.g., rejection reason text, SOS notes), these are in the Audit Trail, not logs.

These exclusions are enforced by Serilog destructuring policies configured globally in the service bootstrap, sensitive properties are automatically masked before the log event is serialised.

Distributed Tracing. OpenTelemetry

Every inbound request to the Ocelot gateway starts a trace. The trace is propagated through all downstream service calls using the W3C TraceContext header, so every span in the trace shares the same TraceId. The CorrelationId (used in logs) is embedded in the root span so that a log event and its trace are always cross-referenceable:

- Trace structure: Root span (Ocelot gateway) → child spans per microservice call → grandchild spans for database queries, Redis operations, RabbitMQ publishes, and external HTTP calls (Paymob, Google Maps).
- Each span records: service name, operation name, start/end time, status (OK / Error), and key attributes (rideld, paymentId, tripStatus where relevant).
- Error spans carry the exception type and message (not stack trace, stack traces go to the Error log). This keeps trace storage lean.
- Trace data is collected by the self-hosted OpenTelemetry Collector on the monitoring VM and retained for 30 days. Grafana Tempo or a lightweight OTLP backend is used for trace search.
- Sampled traces: all Error-status traces are retained (100% sampling); successful traces are sampled at a configurable rate (default 10% in Production) to manage storage volume.

4.2.14.5 Health Check & Service Status

Every microservice exposes two ASP.NET Core Health Check endpoints that serve different purposes and are consumed by different systems. Health checks are the integration point between the application layer, the load-balancing layer, and the Prometheus monitoring stack.

Liveness vs Readiness. Distinction and Design

Check Type / Endpoint	What It Confirms	Consumed By	Implementation Detail
Liveness /health/live	The service process is running and not deadlocked.	Load-balancing layer (long interval: 30 s). If liveness fails repeatedly, the VM is restarted.	Returns 200 OK unconditionally if the process is alive. Does not check dependencies, a dependency failure should never cause a liveness failure, as it would trigger an unnecessary VM restart.
Readiness /health/ready	The service is healthy and ready to accept requests.	Load-balancing layer (short interval: 10 s). If readiness fails, the VM is removed from rotation but not restarted.	Checks all critical dependencies: SQL Server (connection open + SELECT 1), MongoDB (ping), Redis (PING), RabbitMQ (management API or channel open), Vault (seal status). Any dependency failure → 503 Service Unavailable.

Aggregated Platform Health

- An aggregated health endpoint on the Ocelot gateway polls the readiness endpoint of each registered microservice and returns a single combined health response (JSON) listing each service and its status. This is the source for the Admin Portal health dashboard widget visible to the Operations Supervisor role.
- Prometheus scrapes the readiness endpoints of all services via the blackbox exporter on a 10-second interval. A sustained readiness failure (≥ 2 consecutive failures) fires a Grafana alert to the Operations notification channel and, for Safety & SOS Service or Payment Service failures, additionally pages the engineering on-call rotation.
- The Safety & SOS Service is the highest-priority service on the platform: it is mandated to be active on both Application VMs at all times (active-active). Its readiness probe is checked every 5 seconds, and a single failure immediately fires an on-call page; not a Slack notification.

Health Check Coverage per Dependency

Dependency	Health Check Implementation
SQL Server	Connection open + SELECT 1 on the service's own database. Timeout: 3 s.
MongoDB	db.runCommand({ping: 1}) on the replica set primary. Timeout: 3 s.
Redis	PING command. Timeout: 1 s. Failure means cache is unavailable, service is marked not-ready to prevent cache-miss storms on a cold restart.
RabbitMQ	Channel.IsOpen check or management API /api/healthchecks/node. Timeout: 3 s.
HashiCorp Vault	Vault /v1/sys/health endpoint. Sealed or standby → not-ready (services cannot retrieve secrets). Timeout: 3 s.
Hangfire (Background VMs only)	Hangfire server heartbeat check, confirms the background processing host is registered and accepting jobs. Timeout: 5 s.

4.2.14.6 Real-Time Hub (SignalR)

The Real-Time Hub is the single, dedicated WebSocket channel for all real-time streaming on the SheDrive platform. It is implemented as an ASP.NET Core SignalR hub, self-hosted on IIS on the same Presentation & API Gateway VMs as the three Ocelot gateway instances, in its own IIS application pool. All three client surfaces (Rider App, Driver App, and Admin Portal) connect exclusively to this hub for real-time communication. The three Ocelot API Gateway instances handle REST traffic only and have no WebSocket responsibility.

Streaming Responsibilities

- Driver GPS location fan-out: the Driver App pushes location coordinates to the hub at a maximum 3-second interval. The hub fans these out to the Rider App session subscribed to that trip and to any Admin Operations dashboard session monitoring that trip simultaneously. Fan-out is hub-native; the Trip & Dispatch Service does not call multiple endpoints.
- SOS alert delivery: when the Safety & SOS Service publishes an SOSTriggered event, the hub pushes the alert immediately to all connected Admin Operations dashboard sessions. This push is parallel to the FCM push notification and is not dependent on it.

Connection Model and Authentication

- JWT validated on connection establishment. The same token issued by the User Management Service is used; no separate authentication mechanism is introduced for the hub.
- Active trips are modelled as SignalR groups. On driver assignment, the Rider App session and any active Admin dashboard session join the trip group. The Driver App pushes GPS coordinates to the group; the hub fans them to all group members in a single operation.
- Connection lifecycle: hub connections are established on app foreground and terminated cleanly on trip completion, cancellation, or app background. Clients reconnect automatically with exponential backoff on disconnect.
- Prometheus monitors the concurrent SignalR connection count via a dedicated gauge. A sustained breach of the defined threshold triggers a Grafana alert and initiates the scaling review defined in Section 7.5.

Integration with Backend Services

- The Trip & Dispatch Service and the Safety & SOS Service publish events to the hub via the server-side SignalR IHubContext. No Ocelot gateway is involved in this path. The hub is a push-only fan-out component from the service perspective; it does not own business logic or state.

Scalability Path

The Real-Time Hub is co-hosted on the Presentation & API Gateway VMs at MVP. When the sustained concurrent SignalR connection count exceeds the defined Prometheus threshold, the hub is extracted to dedicated VMs. Extraction requires no changes to backend service contracts or mobile client configuration; only the hub endpoint URL is updated in the client and gateway configuration.

Svc Map Real-Time Hub flow — SignalR swimlane · section 4.2.15.6

ARCorp — SheDrive

ASP.NET Core SignalR · WebSocket · persistent connections · co-hosted on Gateway VMs · IHubContext<T> for server-side publish · Group-based fan-out · JWT-authenticated

#	Driver App Flutter · iOS/Android	Rider App Flutter · iOS/Android	Admin Portal React SPA · Web	SignalR Real-Time Hub ASP.NET Core · IHubContext · Group routing	Backend Services Trip & Dispatch · Safety & SOS
A — Connection lifecycle & group assignment					
A1	SEND WS connect JWT Bearer in handshake · persistent conn			HUB Validate JWT · assign group → adds to driver:{userId} group	
A2		SEND WS connect JWT · joins trip group on assignment		HUB Validate JWT · assign group → adds to trip:{tripId} group on assignment	
A3			SEND WS connect JWT · admin role · shared ops group	HUB Validate JWT · assign group → adds to admin:ops + sos:queue groups	
B — GPS location streaming (continuous · every 5 s · while driver online or on active trip)					
B1	PUSH GPS update {lat, lng, accuracy, heading} every 5 s · stops on trip_ended			HUB Receive · cache to Redis · fan-out → broadcasts to trip:{tripId} group	
B2		RECV Driver position update Driver marker moves on live map · ETA recalculated		BROADCAST Push GPS to rider connection Target: trip:{tripId} group → rider conn	
B3			RECV All active driver positions Live map · all trips in motion visible to ops	BROADCAST Push GPS to admin group Target: admin:ops → all active admin conns	
C — Trip state events (backend → Hub → apps via IHubContext)					
C1				HUB Receive TripStateChanged · resolve groups Fan-out per state to relevant connections	Trip & Dispatch PUBLISH TripStateChanged Via IHubContext<RTHub> · fires on every state transition
C2	RECV New ride request Accept/decline shown · 30 s timeout			PUSH TripAssigned → driver conn Target: driver:{driverId} group	
C3		RECV Trip lifecycle updates Assigned · en-route · arrived · started · ended		PUSH TripState → rider conn Target: trip:{tripId} → rider member	
C4			RECV Live trip list refresh State column changes · ETA live · no polling	BROADCAST TripUpdate → admin group Target: admin:ops → all active admin conns	
D — SOS alert fan-out (P1 safety · sub-second delivery · all ops notified simultaneously)					
D1	SOS Tap SOS button REST POST → Driver Gateway → Safety & SOS Svc	SOS Tap SOS button REST POST → Rider Gateway → Safety & SOS Svc			Safety & SOS PERSIST Create incident Capture: userId · tripId · location · context
D2				HUB Receive SOSTriggered · fan-out to ALL admin conns Target: ALL connections in sos:queue group	Safety & SOS PUBLISH SOSTriggered via IHubContext Also publishes to RabbitMQ → Notification Svc → SMS → trusted contacts
D3			RECV SOS ALERT Audible + visual · SOS queue opens · full context · SLA countdown starts	BROADCAST SOSAlert → sos:queue group Sub-second · all active ops agents notified simultaneously	
D4				Trusted contacts notified via SMS with live tracking link → via RabbitMQ → Notification Service → SMS Gateway — NOT via SignalR	Notification Svc SMS Live tracking link Time-limited secure link · shows live location until trip ends

4 event flows · A: Connection · B: GPS streaming every 5 s · C: Trip state via IHubContext · D: SOS fan-out sub-second · All WS connections JWT-authenticated · Group routing prevents cross-user data leakage

SIGNALR HUB

Figure 6. SheDrive Real-Time Hub Flow — SignalR Swimlane (section 4.2.14.6)

4.3 Build View

The Build View describes the concrete implementation pattern applied to realise the SheDrive architecture, specifies the technology stack for each layer, and defines the responsibility boundaries between the delivery teams and stakeholders involved in implementing and operating the solution.

4.3.1 Implementation Patterns

The following implementation patterns are applied across the SheDrive platform. They are grouped by the tier in which they are primarily applied: backend microservices, mobile applications, and the Admin Portal SPA. Patterns listed under a tier may still have implications for other tiers, cross-tier references are noted where relevant.

A. Backend. .NET Microservices

Microservices Architecture

Each of the ten business modules is realised as an independently deployable .NET 8 service with its own data store and versioned API contract. A change to one service is deployed in isolation without affecting any other.

Clean Architecture — Jason Taylor Template (per service)

Each microservice is structured using the Jason Taylor Clean Architecture template as its canonical starting point. The template enforces Domain, Application, Infrastructure, and Presentation as four distinct projects per service, with strict inward dependency direction enforced at the project-reference level by the compiler. The Domain project has no references to any other project. The Application project references only Domain. The Infrastructure project references Application and Domain. The Presentation project (Minimal API endpoints) references Application only. Business logic has no dependency on infrastructure concerns, and the Domain layer is unit-testable without databases or external services.

All new services are created from a SheDrive-derived dotnet new shedrive-service template, built on the JT base and pre-configured with: RabbitMQ and MassTransit consumer registration, Serilog structured logging with mandatory CorrelationId propagation, Prometheus instrumentation, the Outbox table scaffold, role-scoped JWT validation, and ASP.NET Core Health Check endpoints. No service may be created from scratch or directly from the upstream JT template; all must start from the SheDrive template to guarantee consistency across the service estate. Infrastructure adapters (Paymob, Google Maps Platform, SMS gateway) are replaceable without touching business logic, as each adapter sits exclusively in the Infrastructure project behind an Application-layer interface.

Module-Aligned Service Boundaries

Each service owns exactly one business module and its data. Cross-module interactions are mediated through published API contracts or integration events; never through direct data coupling. Formal DDD tactical patterns (aggregates, ubiquitous-language workshops, event storming) are not adopted in Release 1.

Event-Driven Communication (RabbitMQ + MassTransit)

Domain events (RideRequested, TripCompleted, SOSTriggered, PaymentCaptured) are published to RabbitMQ exchanges and consumed asynchronously by downstream services through MassTransit consumer topology. Dead-letter queues capture persistently unprocessable messages.

Outbox Pattern

Domain events are written to a transactional outbox table in the same database transaction as the business operation. MassTransit's built-in delivery service (IHostedService, configured via AddEntityFrameworkOutbox) polls the outbox on a configurable interval and publishes events to RabbitMQ, guaranteeing at-least-once delivery without distributed transaction coupling. The delivery service manages retry and backoff natively. Hangfire is not involved in this flow. Outbox is reserved for critical, must-not-lose state-change events only, specifically: RideRequested, DriverAssigned, TripCompleted, PaymentCaptured, PaymentFailed, SOSTriggered, GenderMismatchReported, and DriverApproved. These are the events where loss or duplication would corrupt a financial record, a safety audit trail, or a core trip state. Non-critical fan-out events including notification dispatch signals, analytics update hints, and report-refresh triggers, are published directly to RabbitMQ by MassTransit without an Outbox write, because their loss is recoverable (the Notification Service retries independently; the Reporting Service re-projects from source events). This boundary is enforced at code-review level: a pull request that routes a non-critical event through the Outbox is rejected.

Saga Pattern. Orchestration via MassTransit

Multi-service flows including ride lifecycle, payment pre-auth/capture, and cash-settlement, are coordinated by a central Saga state machine (MassTransit StateMachine) that issues commands to participants and handles compensating transactions on failure. Why orchestration over choreography: the ride lifecycle and payment settlement flows have strict sequencing requirements, mandatory compensation logic on partial failure (e.g., release pre-auth if trip cannot start), and SLA-governed timeout handling. If these were left to implicit event choreography, where each service reacts to events from others without a coordinator, the overall flow state would be invisible, debugging failures would require tracing events across multiple services, and adding compensation paths would require all participants to be aware of all failure modes. An orchestrating Saga state machine holds all flow state in one place, makes every transition explicit, and gives operations full visibility of in-flight saga instances and their current step.

Selective CQRS — MediatR Pipeline

Commands (writes) and queries (reads) are separated at the application layer where a clear distinction exists. Write operations including ride requests, payment captures, SOS triggers, and admin configuration changes use transactional command paths with full consistency guarantees. Dashboards, trip history, and reporting consume read-optimised query models. Full event sourcing is not enforced globally.

MediatR is the in-process dispatcher for all commands, queries, and domain notifications within each service. Every inbound request is dispatched through a MediatR pipeline before reaching its handler. The pipeline behaviour chain executes in the following fixed order: (1) Logging behaviour — records the command or query name, caller identity, and correlation ID at the start and end of every dispatch; (2) Validation behaviour — invokes all registered FluentValidation validators for the request type before the handler is reached; a validation failure short-circuits the pipeline and returns a structured error response without entering the handler; (3) Exception Handling behaviour — catches unhandled exceptions from the handler, logs them with full context, and translates them into consistent API error responses; (4) Performance Tracking behaviour — measures handler execution time and logs a warning when a handler exceeds the configured threshold, providing the first signal of a slow query or command.

Cross-boundary rule: MediatR handlers must never dispatch to other MediatR handlers across service boundaries. A handler in the Trip & Dispatch Service must not send a MediatR command to a handler in the Payment Service. All cross-service communication is mediated exclusively through RabbitMQ domain events (via MassTransit) or versioned REST API contracts. This rule is enforced at code-review level; a pull request that dispatches a MediatR command to a handler in a different service boundary is rejected.

Materialised View Pattern

The Reporting & Analytics Service maintains pre-aggregated materialised views, implemented as indexed views in SQL Server or a dedicated read-schema populated by background projection, refreshed by Hangfire recurring jobs and event-driven consumers. Materialised views serve dashboard queries, financial reconciliation, and export requests without touching the transactional tables of origin services. Each view is owned exclusively by the Reporting Service; no other service queries a view it did not project. The projection lag (the window between a source event and its appearance in a view) is bounded and monitored.

State Machine Enforcement

The trip lifecycle is a strict server-side state machine: Requested → DriverAssigned → DriverEnRoute → DriverArrived → TripStarted → TripCompleted (or Cancelled at valid states). Transitions that skip states are rejected. Every accepted transition writes an immutable TripStateHistory record (tripId, fromState, toState, UTC timestamp, actor).

Circuit Breaker. Polly

All calls to external providers (Paymob, Google Maps Platform, SMS gateway, FCM/APNS) are wrapped in Polly circuit breakers. Why: without a circuit breaker, a slow or failing external provider blocks .NET HttpClient thread-pool threads waiting for a response that may never arrive. Under load, blocked threads accumulate until the thread pool is exhausted, making the calling service unresponsive, a cascading failure that takes an otherwise healthy service down because of a third-

party outage. The circuit breaker fails requests fast once the failure threshold is crossed, keeping threads free and allowing the rest of the platform to operate normally. Why Polly specifically: Polly is the de-facto .NET resilience library, a transitive dependency of many ASP.NET packages, and integrates natively with `IHttpClientFactory`, so every `HttpClient` registered in DI automatically benefits from circuit breaker and retry policies. The per-integration circuit breaker configuration (open threshold, half-open probe interval) is defined in each integration adapter in Section 4.6, keeping resilience policy and integration contract co-located.

Idempotent Operations

All state-modifying operations carry idempotency keys enforced at the API Gateway and re-asserted at the service layer. Retry attempts from mobile clients, queue redelivery, and Hangfire job retries produce the same deterministic outcome without duplicate side effects.

Integration Adapters and Anti-Corruption Layer

Each external service is accessed through a dedicated adapter that encapsulates the provider's protocol, authentication, payload mapping, timeout enforcement, circuit breaker, and error normalisation. Core domain logic never depends on the external service's data model or contract, replacing a provider requires only a new adapter.

Versioned API Contracts

All service APIs exposed through Ocelot are versioned. Breaking changes require a new version path. Mobile clients and the Admin Portal can evolve at different release cadences from backend services.

B. Mobile Applications. Flutter (Rider App & Driver App)

The Flutter mobile apps apply a distinct set of patterns suited to the mobile delivery context: consumer-grade devices, variable Egyptian mobile network conditions, aggressive battery and background-execution management, and the best-effort nature of push notification delivery.

Offline-Tolerant Design

Reference data (zone polygons, pricing rules, language packs) is cached locally on first load and served from cache on subsequent app opens. State-modifying requests (ride booking, trip actions, cash confirmation) carry idempotency keys and are retried with exponential backoff on network failure. The app surfaces a connectivity banner rather than failing silently.

Push + State-Reconciliation Hybrid

Push notifications (FCM/APNS) are treated as best-effort hints, they wake the app and suggest a state refresh. The app never trusts push payload content as the system of record. On every app foreground, reconnect, or push receipt, the app calls the authoritative trip-state API to fetch current state. This guarantees correct behaviour on devices with disabled notifications, aggressive battery optimisation, or missed deliveries.

Server-Timestamp-Derived Timers

All countdowns and elapsed timers visible in the app (driver waiting time, OTP cooldown, trip duration) are computed from server-provided timestamps rather than local device clocks. This makes timers resilient to app backgrounding, device restarts, and clock drift across devices.

BLoC / Provider State Management

The Flutter apps use BLoC (Business Logic Component) or Provider for state management. UI widgets observe state streams; business logic and API calls are separated from widget code. This makes the apps testable (BLoC units are testable without the UI), predictable (state is a function of events), and maintainable across the release cadence.

Repository Pattern (Data Access Abstraction)

Each feature area (trips, payments, profile, notifications) has a Repository class that abstracts the choice between a local SQLite/Hive cache and a remote API call. The app layer calls the repository; the repository decides whether the cached data is fresh enough or a network call is needed. This centralises caching strategy and makes the data layer swappable in tests.

Masked Number Proxy (Client Side)

When a rider or driver initiates an in-app call, the app requests a proxy session from the backend, it never dials the other party's real number directly. The proxy number and session token are displayed in the app for the duration of the active trip. The app terminates the proxy session on trip completion or cancellation.

Deep Link and Notification Tap Handling

Every push notification carries a structured deep-link payload (e.g., trip ID, action type). On notification tap, the app navigates directly to the relevant screen (trip detail, SOS status, payment outcome) regardless of whether the app was in foreground, background, or terminated. The navigation target always re-fetches authoritative state before rendering.

C. Admin & Operations Portal. Angular SPA

The Angular SPA applies standard enterprise frontend patterns that support the multi-role, data-dense nature of the Admin Portal and the security constraints of a back-office surface.

Feature-Module Architecture

Each functional area of the Admin Portal (Driver Management, Ride Operations, Finance, Safety & SOS, Reporting, System Configuration) is an Angular feature module with its own routing, components, services, and lazy-loaded bundle. This reduces initial load time and limits the blast radius of a change to one module.

Lazy Loading

Feature modules are loaded on demand when their route is first activated. The initial app bundle contains only the authentication and shell modules, making the portal fast to load even for roles that only use one or two modules.

Role-Guard-Based Route Protection

Each Angular route carries a role guard that checks the authenticated user's role claims before activating the route. A Finance User who navigates to the Driver Management route is redirected to an unauthorised page rather than seeing an empty view. Role guards are the frontend complement to the backend RBAC enforcement, both must pass independently.

Reactive Forms and Validation

All data-entry forms in the Admin Portal including approval/rejection decisions, pricing rule configuration, zone polygon editor, operating hours, commission rates, and cancellation policy, are built with Angular Reactive Forms. Why Reactive Forms reduce delivery time: the form model is defined once in the component class as a typed FormGroup; the template only binds to it. This means a developer building a new configuration form writes the field definitions, validators, and submission logic in one place and never touches the template to change business rules. Validators including EgyptianPhoneValidator, numeric range checks, and required-if-active-zone rules, are plain TypeScript functions written once and reused across every form that needs them.

Configuration forms in the Admin Portal are data-driven: the pricing rule editor and zone boundary form are built programmatically at runtime from the backend configuration schema, so adding a new rule parameter requires only a backend change and a config update, not a new Angular component. While a save call is in-flight, form.disable() prevents duplicate submissions in one line; re-enabling on response is equally trivial. The form is a plain TypeScript object testable with new FormGroup(...) in a unit test, with no browser or component lifecycle needed. The net effect is that each new Admin Portal feature form takes one developer a day to build, validate, and unit-test rather than two or three.

HTTP Interceptor. Token Refresh and Correlation ID

A global Angular HTTP interceptor attaches the JWT access token to every API request and handles token refresh transparently (using the refresh token when the access token expires). A second interceptor stamps a client-generated correlation ID on every request header, which the Ocelot gateway adopts if present, enabling Admin Portal requests to be correlated in Seq from the browser action through to the database query.

Real-Time SOS Queue (SignalR)

The Admin Operations dashboard maintains a SignalR connection to the Real-Time Hub (Section 4.2.14.6) for the duration of the operator's session. Incoming SOS events are pushed directly to

the SOS queue component without polling, with an audio alert and a visual badge. The connection is managed by an Angular service using the @microsoft/signalr client library, with automatic reconnection on disconnect.

D. Development Tooling — Internal CQRS Code Generator

To eliminate hand-crafting of structural boilerplate across ten services and enforce architectural convention at creation time rather than code-review time, the SheDrive platform introduces a lightweight internal CLI code generator. The generator produces a complete CQRS feature slice — all standard artifacts for a new entity or feature area — from a single YAML entity descriptor file. Convention is enforced at the point of file creation; structural divergence is detected before a pull request exists rather than during review.

Tool and Packaging

The generator is implemented as a .NET console application using Scriban as the templating engine (MIT licence, zero runtime footprint). It is packaged as a dotnet global tool and installed from the internal NuGet feed: dotnet tool install shedrive-gen. Invocation: dotnet shedrive-gen feature --descriptor <entity-descriptor.yaml>. The generator and its Scriban templates are version-controlled in the platform mono-repository alongside the service projects they serve. A convention change — for example, adopting a new handler base class — is applied by updating the template and reflected in all subsequent generation runs.

Artifacts Generated per Feature Invocation

- Domain: Entity class, domain event stub.
- Application: DTO class, Create command + handler, Update command + handler, FluentValidation validator for each command, GetById query + handler, GetList query + handler.
- Infrastructure: EF Core entity configuration class.
- Presentation: Minimal API endpoint registration file with CRUD route stubs.
- Tests: xUnit test class stubs for each command and query handler, correctly namespaced and cross-referenced.

What the Generator Does Not Produce

The generator produces structural scaffolding only. Business logic, domain rules, saga state machines, integration adapters, and domain events are always hand-authored. A generated handler contains a clearly marked TODO region; the handler is not functional until a developer completes the business logic. Deviating from the generated structure in a hand-written service or handler requires explicit justification at code review.

The entity descriptor YAML schema that drives the generator is documented in Section 10.5, Code Generator: Entity Descriptor Schema.

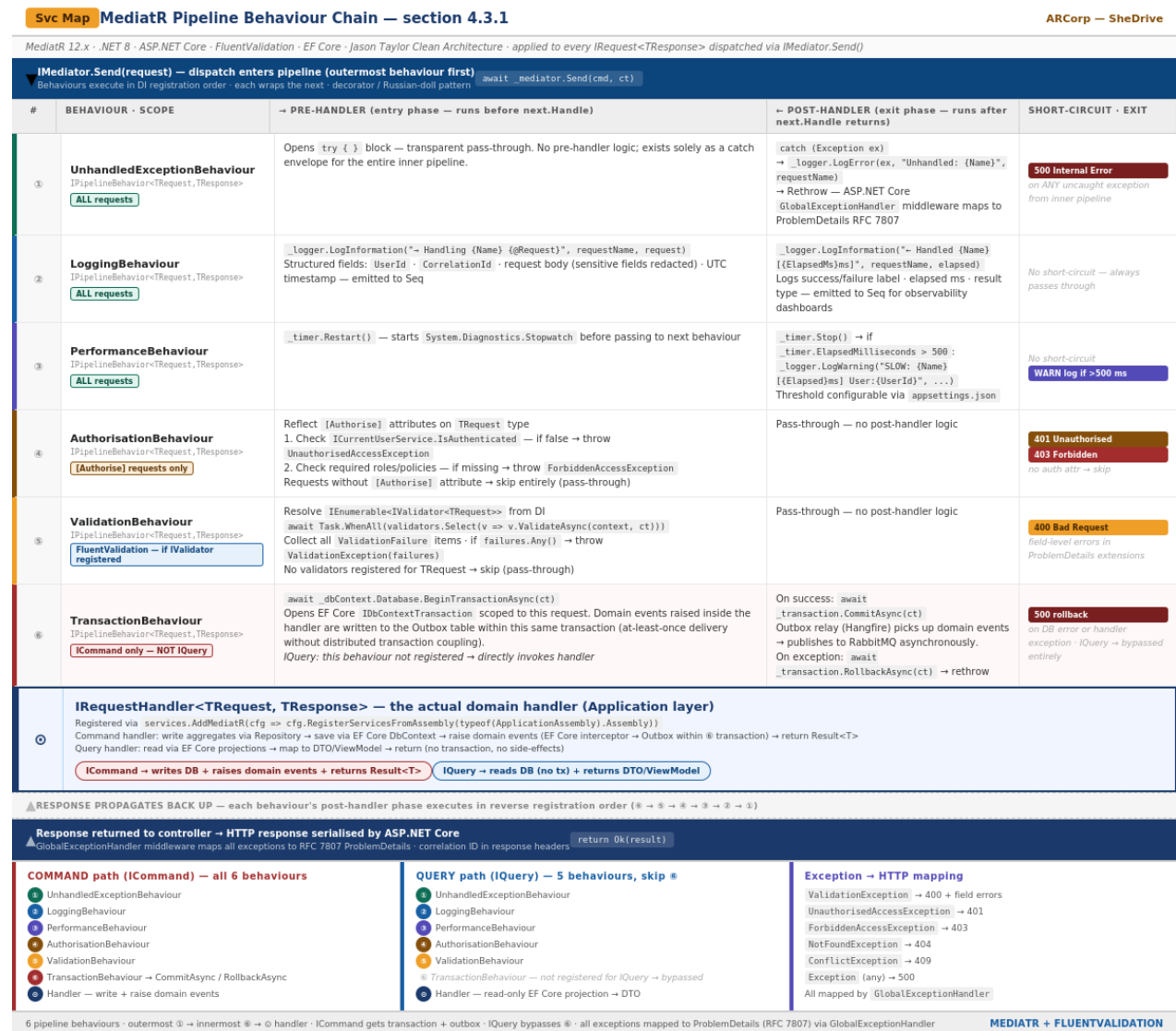


Figure 8. SheDrive MediatR Pipeline Behaviour Chain — six behaviours (section 4.3.1)

4.3.2 Technology Stack

The table below defines the complete technology stack for the SheDrive MVP, covering all platform layers from client surfaces through backend services, data stores, infrastructure, and external integrations. In line with the hosting constraint (Section 3.3), AWS is used for managed servers (EC2) and object storage (S3) only, all platform software is self-hosted and portable.

Layer / Concern	Technology	Role in SheDrive
Rider & Driver Apps	Flutter (Dart), iOS & Android	Single codebase cross-platform mobile apps for Riders and Drivers. Native plugin support for Google Maps, Firebase FCM, and camera capture. Bilingual Arabic (RTL) and English.
Admin & Ops Portal	Angular 17+. Web SPA	Role-based single-page application for all back-office functions. Statically served via IIS co-hosted on the Presentation & API Gateway VMs. Bilingual (AR/EN) with RTL layout support.
Backend Services	.NET 8 / ASP.NET Core	Independently deployable domain microservices hosted on IIS / .NET 8 Hosting Bundle on EC2

Layer / Concern	Technology	Role in SheDrive
		Windows Server VMs. Clean Architecture layering per service (Jason Taylor Template). REST APIs consumed via the three Ocelot gateway instances. Real-time push delivered via the SignalR Real-Time Hub.
API Gateway	Ocelot (.NET)	Three independent Ocelot instances (Rider Gateway, Driver Gateway, Admin Gateway) co-hosted on IIS on the Presentation & API Gateway VMs, each in its own IIS application pool. Each instance serves one client surface exclusively via REST. Provides JWT authentication enforcement (role-scoped per instance), rate limiting, request routing, response compression, correlation ID issuance, and operating-window enforcement. No WebSocket handling — real-time streaming is the sole responsibility of the SignalR Real-Time Hub (see below). Co-hosted with the Angular SPA on Presentation & API Gateway VMs.
Real-Time Hub	ASP.NET Core SignalR (self-hosted on IIS)	Dedicated WebSocket hub for all real-time streaming: driver GPS location fan-out to the Rider App and Admin Operations dashboard, and SOS alert delivery to the Admin Operations queue. JWT-authenticated on connection establishment. All three client surfaces (Rider App, Driver App, Admin Portal) connect to a single hub endpoint. Co-hosted on the Presentation & API Gateway VMs in its own IIS application pool. Trip groups managed natively by SignalR for per-trip fan-out.
Development Tooling	Internal CLI — Scriban (.NET Tool)	Internal CQRS code generator for all .NET microservices. Produces entity, DTOs, commands, queries, FluentValidation validators, Minimal API endpoints, and xUnit test stubs from an entity descriptor YAML file (see Section 10.5). Packaged as a dotnet global tool installed from the internal NuGet feed. Zero runtime footprint — build-time development tooling only. Schema documented in Section 10.5.
Compute / Hosting	AWS EC2. Windows Server (latest)	All backend layers deployed on AWS EC2 Windows Server VMs used purely as managed servers. IIS hosts .NET 8 microservices, the Ocelot gateway, and the Angular SPA. HA achieved through 2× VMs per layer behind the load-balancing layer.
Authentication & Authorisation	JWT / OAuth 2.0 (ASP.NET Core Identity)	Token-based authentication issued by the User Management service. JWT validated at the Ocelot gateway and re-enforced at the service layer. RBAC aligned to the 8 platform roles.
Async Messaging. Broker	RabbitMQ (latest stable LTS)	Self-hosted RabbitMQ 2-node quorum queue cluster on EC2 VMs. Handles ride matching, payment events, SOS alerts, notification dispatch, and report aggregation. Outbox draining is performed by the MassTransit delivery service within each microservice host.
Async Messaging. Client	MassTransit (.NET)	.NET message bus abstraction over RabbitMQ. Provides consumer topology, retry policies, dead-

Layer / Concern	Technology	Role in SheDrive
		letter queue routing, and saga orchestration across all services.
Background Jobs & Scheduling	Hangfire (.NET)	Platform-wide background job framework with SQL Server job storage. Recurring (cron), delayed, and fire-and-forget jobs: settlement reminders, document expiry sweeps, payment retry sweeps, report pre-aggregation, and stale-session cleanup. Built-in monitoring dashboard, access-restricted. Outbox draining is handled by MassTransit, not Hangfire.
Relational Database	SQL Server (self-hosted on EC2)	Microsoft SQL Server (latest LTS. Enterprise Edition) self-hosted on EC2 Windows Server VMs. AlwaysOn AG or Failover Cluster for HA. Stores trips, payments, commissions, user accounts, zone config, pricing rules, verification records, and Hangfire job storage.
Document & Event Store	MongoDB (self-hosted on EC2)	MongoDB Community/Enterprise self-hosted on EC2 VMs. 2-node replica set plus arbiter. 2dsphere geospatial index for driver live location. TTL index on location documents. Stores event logs, audit records, and SOS incident data.
Cache Layer	Redis (self-hosted on EC2)	Redis self-hosted on EC2 VMs with Redis Sentinel for HA. allkeys-lru eviction. Stores pricing rules, zone polygons, active driver index, session tokens. Never the source of truth.
Structured Logging. Client	Serilog (.NET)	Structured logging library in all .NET services. Emits structured JSON with mandatory CorrelationId, RidId, ServiceName, and Severity fields. No sensitive data in log events.
Centralised Log Server	Seq (self-hosted)	Self-hosted structured log server on the dedicated monitoring VM. Receives Serilog output from all services. Provides searchable log explorer for operational investigation and post-incident analysis.
Metrics & Alerting	Prometheus + Grafana (self-hosted)	Prometheus scrapes service, IIS, VM, SQL Server, MongoDB, Redis, and RabbitMQ exporters on the monitoring VM. Grafana provides operational dashboards and alert rules routed to the Operations notification channel and engineering on-call.
Distributed Telemetry	OpenTelemetry	OpenTelemetry SDK in all .NET services exports distributed traces and metrics to the self-hosted collector on the monitoring VM. End-to-end request tracing with correlation IDs across all microservices.
Health Monitoring	ASP.NET Core Health Checks	Liveness and readiness endpoints on every microservice. Aggregated health dashboard for the Operations team. Integrated with the load-balancing layer health probe configuration.
Push Notifications	Firebase Cloud Messaging (FCM) / APNS	Push delivery to Rider and Driver Flutter apps on Android (FCM) and iOS (APNS). Device token management per user. Push is best-effort: every push-driven flow has a pull-based state reconciliation path. Critical SOS alerts bypass quiet hours and have SMS fallback.

Layer / Concern	Technology	Role in SheDrive
SMS Delivery	Local Egyptian SMS Gateway	OTP delivery during registration and login. Critical alert SMS for SOS events and payment failures. Bilingual template support.
Maps & Location	Google Maps Platform	Geocoding, routing, ETA calculation, and map rendering. Client-side Flutter Maps SDK for rendering; all billable server-side APIs centralised in the Maps & Location Service with Redis caching (see Section 4.6.2).
Payment Processing	Paymob (Egypt)	Single PSP integration for digital card payments. Hosted payment fields for PCI-compliant card capture and tokenisation in the Rider App; server-side intention, capture, refund, and webhook handling in the Payment Service (see Section 4.6.1).
Document Storage	AWS S3 Standard (managed)	Driver onboarding documents, vehicle photos, and profile photos stored in a private S3 bucket with SSE-S3 encryption. Presigned URLs for time-limited access.
Secrets Management	HashiCorp Vault (self-hosted)	All database passwords, PSP API keys, SMS tokens, and JWT signing keys stored in a self-hosted HashiCorp Vault instance on the monitoring/management layer, with DPAPI-protected configuration as the bootstrap mechanism. No secrets in config files or source code.
TLS Certificates	TLS certificate provider	TLS certificates for all internet-facing endpoints, automated issuance and renewal via TLS automation tooling on the gateway VMs (or a commercial wildcard certificate). TLS 1.2 minimum everywhere.

4.3.3 Team Responsibilities

The table below defines the responsibility boundaries between the ARCorp delivery teams, SheDrive Team stakeholders, and external service providers involved in implementing and operating the SheDrive platform.

Team / Party	Scope	Key Responsibilities
ARCorp. Solution Architect	Architecture & design	HLD ownership, architectural decision authority, cross-service design governance, integration boundary definition, and alignment with BRD requirements.
ARCorp. Backend Technical Leads	Microservice implementation	Service-level design, Clean Architecture enforcement, API contract definition, code quality, unit and integration test coverage, and domain model integrity.
ARCorp. Mobile Technical Leads	Flutter app implementation	Rider and Driver app development, Google Maps and FCM plugin integration, offline-tolerant API consumption, bilingual RTL layout, and platform-specific build management.
ARCorp. Frontend Technical Lead	Admin Portal implementation	Angular SPA development, RBAC enforcement at UI layer, bilingual RTL layout, real-time WebSocket dashboard integration, and Admin Portal API consumption.

Team / Party	Scope	Key Responsibilities
ARCorp. DevOps / Release Engineers	Infrastructure & pipeline	Terraform and Ansible/DSC infrastructure definition, Azure DevOps pipeline configuration, IIS deployment automation to EC2 VMs, environment management, and observability stack setup (Seq, Prometheus, Grafana, OpenTelemetry).
ARCorp. Quality Control	Testing & acceptance	Test plan authorship, functional and regression test execution, performance and integration test coordination, defect management, and acceptance evidence collection.
SheDrive Team. Product Owner	Scope & priorities	Requirement decisions, acceptance criteria confirmation, change request governance, and MVP scope boundary enforcement.
SheDrive Team. Operations Management	Operational readiness	SOS workflow procedure definition, driver onboarding process sign-off, cash settlement process confirmation, and operations team training.
SheDrive Team. Finance & Compliance	Regulatory & financial rules	Commission rate and cancellation rule approval, cash settlement workflow confirmation, structured data export requirements, and data-protection policy review.
External Providers (Paymob, Google Maps, SMS Gateway, FCM/APNS)	External service delivery	Maintaining availability and stability of their exposed APIs and sandboxes, providing integration documentation, and supporting integration testing with appropriate test environments.

4.4 Information View

This section defines the information architecture of the SheDrive platform, the major business entities, data ownership boundaries, data classification tiers, data store assignment rationale, and data lifecycle policies. The Information View is the primary reference for data governance and the platform data-retention policy.

4.4.1 Major Business Entities

The following table defines the major business entities on the SheDrive platform, their key attributes, and their relationships to other entities. This is a conceptual entity model, it is not an entity-relationship diagram and does not define physical database schema.

Entity	Key Attributes	Relationships
Rider	RiderId, MobileNumber, FullName, Email, VerificationStatus (first-trip verification), AccountStatus (incl. SuspendedForReview), LanguagePreference, CreatedAt	Has many Trips. Has many TrustedContacts. Has at most one FirstTripVerification outcome. Has many PaymentCards (via PSP tokens).
Driver	DriverId, MobileNumber, FullName, Gender, LicenceNumber, VerificationStatus, OnboardingStatus, AccountStatus, ActiveWorkingZones, CashBalance, CreatedAt	Has many Trips. Has one DriverApplication (with decision record). Has one Vehicle. Has many DriverDocuments. Has one ProfilePhoto. Has many CashSettlements.
Vehicle	VehicleId, DriverId, Make, Model, Year, PlateNumber, Colour, VehicleType (Sedan/SUV/Minivan), RegistrationExpiry, EligibilityStatus	Belongs to one Driver. Has many VehicleDocuments.
Trip	TripId, RiderId, DriverId, VehicleId, PickupLocation, DropoffLocation, Status (lifecycle state), PaymentMethod, EstimatedFare, FinalFare, ChildPassengerDeclared, IsFirstTrip, StartedAt, CompletedAt, ZoneId	Belongs to one Rider and one Driver. Has one Payment. Has one Cancellation (if cancelled, incl. gender_mismatch_report reason). Has many TripStateHistory records (immutable). Has one SOSIncident (if triggered).
Payment	PaymentId, TripId, RiderId, Method (Cash/Digital), Amount, Commission, Status, PspTransactionId, CapturedAt	Belongs to one Trip. Has many PaymentAttempts (for retries). Has one Receipt. Referenced by DriverCashLedger (cash trips).
DriverCashLedger	LedgerId, DriverId, TripId, CashAmount, CommissionOwed, SettledAt, SettlementStatus	Belongs to one Driver. Referenced by CashSettlement. Updated on every completed cash Trip.
Zone / Neighbourhood	ZoneId, Name, PolygonGeoJSON, IsActive, CreatedAt, UpdatedAt	Has many PricingRules. Referenced by Trip (pickup zone). Referenced by Driver

Entity	Key Attributes	Relationships
		(working zones). Has many CancellationRules.
PricingRule	RuleId, ZoneId, BaseFare, PerKmRate, PerMinuteRate, MinimumFare, PeakMultiplier, EffectiveFrom, EffectiveTo, Version	Belongs to one Zone. Versioned, previous rules retained for audit. Consumed by Trip (fare calculation).
DriverApplication	ApplicationId, DriverId, NationalIdNumber (14-digit), DateOfBirth, BackgroundCheckConsent, Status (Pending/Approved/Rejected), FemaleConfirmedFlag, RejectionReason, ReviewedBy, ReviewedAt	Belongs to one Driver. Immutable after decision. Has associated DocumentReferences and ProfilePhotoReference (S3 keys).
SOSIncident	IncidentId, TripId, TriggeredBy (Rider/Driver), TriggerTime, Location, Status, FirstResponseTime, EscalationLevel, ResolvedAt, ResolutionNotes	Belongs to one Trip. Has many IncidentEvents (timeline). Has one Operator assignment. Linked to AuditEvent log.
AuditEvent	EventId, EntityType, EntityId, Action, ActorId, ActorRole, PreviousValue, NewValue, CorrelationId, Timestamp	Referenced by all entities that produce audit records. Append-only, never updated or deleted. Stored in MongoDB audit collection.
Notification	NotificationId, RecipientId, Channel (SMS/Push/Email/InApp), TemplateId, Language, Status, SentAt, DeliveryConfirmedAt, RetryCount	Associated with a triggering event (TripId, IncidentId, etc.). References NotificationTemplate. Stored in MongoDB delivery log.

4.4.2 Data Ownership Matrix

Each data set on the platform has a single owning service that is the sole writer and authoritative source of truth for that data. All other services are read-only consumers through defined API contracts; never through direct database access. The following table defines data ownership and read access across all services.

Entity / Data Set	Owning Service	Primary Store	Read Consumers
Rider profile & account	User Management	SQL Server	Trip & Dispatch, Safety & SOS, Notification, Admin Portal, Reporting
Driver profile & account	Driver & Vehicle Registration	SQL Server	Trip & Dispatch, Payment, Safety & SOS, Notification, Admin Portal, Reporting
Vehicle record	Driver & Vehicle Registration	SQL Server	Trip & Dispatch, Admin Portal

Entity / Data Set	Owning Service	Primary Store	Read Consumers
Driver application & verification decision	Identity & Verification	SQL Server + MongoDB (audit)	Admin Portal (review queue), User Management, Driver & Vehicle Registration
Driver live location	Trip & Dispatch	MongoDB (2dsphere)	Trip & Dispatch (matching), Maps & Location, Rider App (tracking), Admin Portal (live map)
Trip record (lifecycle state)	Trip & Dispatch	SQL Server	Payment, Safety & SOS, Notification, Admin Portal, Reporting
Trip event log	Trip & Dispatch	MongoDB	Reporting, Admin Portal (ride detail view)
Zone / polygon definition	Pricing & Zones	SQL Server	Trip & Dispatch (eligibility), Driver App (zone selection), Redis (cached)
Pricing rules	Pricing & Zones	SQL Server	Trip & Dispatch (fare estimate), Payment (fare calculation), Redis (cached)
Payment record	Payment	SQL Server	Admin Portal (reconciliation), Reporting, Driver Cash Ledger
Driver cash ledger	Payment	SQL Server	Admin Portal (cash reconciliation), Driver App (earnings view), Reporting
Cash settlement	Payment	SQL Server	Admin Portal (settlement dashboard), Reporting
SOS incident record	Safety & SOS	SQL Server + MongoDB (detail)	Admin Portal (SOS queue), Reporting, Audit Trail
Audit event log	All services (producers)	MongoDB (append-only)	Admin Portal (audit viewer. Compliance/Admin roles only)
Notification delivery log	Notification	MongoDB	Admin Portal (template management), Reporting
Notification templates	Notification	SQL Server	All services (consumers), Redis (cached)
Driver onboarding documents & photos	Identity & Verification	AWS S3	Admin Portal (document review), Identity & Verification (decision support)
Cancellation record	Trip & Dispatch	SQL Server	Admin Portal (cancellation review), Reporting
Session tokens (JWT)	User Management	Redis	Ocelot API Gateway (validation), all services (re-validation)
Active driver index (geospatial)	Trip & Dispatch	Redis	Trip & Dispatch (matching candidate set)
Report aggregations	Reporting & Analytics	SQL Server (read model)	Admin Portal (dashboard, reports, exports)

Svc Map Data Architecture — ownership, classification & retention · section 4.4					ARCorp — SheDrive
Per-service data isolation · no cross-service database access · SQL Server (transactional) · MongoDB (document + geospatial + audit WORM) · Redis (cache/session) · AWS S3 (file store) · each service is sole writer of its own database					
#	OWNING SERVICE	DATABASE / STORE	TYPE	KEY ENTITIES & DATA STORED	RETENTION
IDENTITY & ACCESS — SQL Server					
1	Identity & Verification sole writer · KYC results	identity_db SQL Server	SQL Server	NID OCR results (confidence score, extracted fields, raw data ref) Liveness outcomes · face match scores · verification decisions Document metadata refs (S3 object keys — blobs not stored here)	5 years regulatory
2	User Management sole writer · accounts	users_db SQL Server	SQL Server	User accounts (userid, phone, name, role, status, reg. date) Session tokens (hashed) · device FCM tokens · revocation list Trusted contacts · profile photo ref (S3) · RBAC assignments	Acct lifetime + 2 yr operational
3	KYC Module sole writer · review queue	kyc_db SQL Server	SQL Server	Review queue entries · reviewer assignments Human decisions + written rejection reasons Review audit trail (reviewer ID, timestamp, decision)	5 years regulatory
TRIP OPERATIONS — SQL Server					
4	Trip & Dispatch sole writer · trip lifecycle	trips_db SQL Server	SQL Server	Trip records (tripId, riderId, driverId, zoneId, fare, status) TripStateHistory — immutable log of every state transition + actor Assignment log · GPS event trail (sampled, in-trip) · cancellation records	3 years operational
PRICING & ZONES — SQL Server					
5	Pricing & Zones sole writer · rate config	pricing_db SQL Server	SQL Server	Zone definitions (name, active flag, effective dates) Rate cards (base fare, per-km, per-min, min fare per zone) Zone-pair overrides · cancellation rules · rate card version history	Indefinite configuration
PAYMENTS & FINANCE — SQL Server					
6	Payment Service sole writer · financial records	payment_db SQL Server	SQL Server	Payment records (tripId, method, amount EGP, status, paymobRef) Paymob card tokens (PCI-scoped, AES-256 field-level encryption) Cash ledger · commission records · receipts · reconciliation entries	7 years financial reg.
SAFETY & SOS — SQL Server					
7	Safety & SOS sole writer · incidents	sos_db SQL Server	SQL Server	SOS incidents (userid, tripId, location snapshot, timestamp) Escalation log · responder assignments · SLA timestamps Resolution records · trusted-contact dispatch log per incident	5 years regulatory
COMMUNICATIONS — SQL Server					
8	Notification sole writer · delivery logs	notify_db SQL Server	SQL Server	Notification templates (AR/EN per channel, versioned) Delivery log (channel, status, attempt count, failure reasons) Retry queue state · user channel preferences	Templates: indefinite Logs: 180 days operational
REPORTING & ANALYTICS — SQL Server (read models)					
9	Reporting & Analytics sole writer · aggregated views	reports_db SQL Server	SQL Server	Pre-aggregated ride summaries (daily/weekly/zone granularity) Payment reconciliation projections · driver performance snapshots Materialised views refreshed by Hangfire recurring jobs from domain events	3 years operational
DRIVER REGISTRATION — SQL Server					
10	Driver Registration sole writer · onboarding	driver_db SQL Server	SQL Server	Onboarding applications · status (pending/approved/rejected) Vehicle details · document metadata (S3 keys, blobs not here) Background check results · admin approval records	Acct lifetime + 2 yr operational
DOCUMENT STORE — MongoDB (geospatial · audit WORM · flexible schema)					
11	Trip & Dispatch sole writer · real-time GPS	gps_db MongoDB	MongoDB	Active driver locations (driverId, lat, lng, accuracy, heading) 2dsphere index for \$near proximity queries (driver matching) Updated every 5 s via SignalR Hub · TTL auto-expire on inactivity	Active + 24 h history TTL-based
12	Pricing & Zones sole writer · zone polygons	zones_db MongoDB	MongoDB	Zone polygon GeoJSON documents (Feature per zone) 2dsphere index for \$geoWithin pickup-to-zone lookup Zone versions archived (never deleted) · effectiveFrom timestamps	Indefinite configuration
13	Audit Trail Service sole writer · append-only · WORM	audit_db MongoDB	MongoDB	Audit event documents — flexible schema, one document per event (eventId, timestamp UTC+2, actorId, actorRole, action, entityType, entityId) (beforeState, afterState, correlationId, serviceOrigin, ipAddress) Covers: admin mutations · config changes · account state changes · SOS events · financial transitions · auth events No updates · no deletes · append-only · Change Stream for real-time audit alerts	5 years min WORM · regulatory
CACHE & SESSION — Redis (volatile · TTL-governed · rebuilds from SQL on restart)					
14a	User Management session cache	sessions:{userid} Redis key-space	Redis	Hashed session token → (userid, role, issuedAt) · revoked token set · device token lookup	TTL: 30 days TTL-based
14b	Trip & Dispatch driver location cache	location:{driverId} Redis key-space	Redis	Latest GPS position per online driver (low-latency matching read) · online flag · active tripId ref	TTL: 10 min inactivity TTL-based
14c	Ocelot Gateways rate-limit counters	ratelimit:{userid} Redis key-space	Redis	Per-user sliding-window request counters per endpoint type (general / OTP / SOS)	TTL: 60 s / 300 s TTL-based
14d	Pricing & Zones pricing cache	pricing:active Redis key-space	Redis	Active rate cards per zone (serialised) · invalidated on ZoneUpdated event via RabbitMQ	TTL: 15 min + event-invalidated TTL-based
FILE STORE — AWS S3 (immutable after upload · pre-signed URL access only · no direct client access)					
15	Driver Registration sole writer · document blobs	shedrive-documents/ AWS S3 bucket	AWS S3	National ID scans (front/back) · live selfie · driving licence · vehicle docs · profile photos Access via time-limited pre-signed URLs only · S3 Object Lock for regulatory docs	Acct lifetime + 2 yr regulatory
Isolation rule — no cross-service DB access No service reads another service's database directly. Violations rejected at code-review. Cross-service data access exclusively via: • REST APIs via the appropriate Ocelot gateway • Domain events via RabbitMQ (async, eventual consistency) • Pre-aggregated read models in reports_db • Audit trail (audit_db) is write-only from all services — never read back into service business logic Backup & recovery SQL Server: Always On AG (SQL-VM-01~SQL-VM-02) · EBS daily snapshot 30-day retention MongoDB (gps_db · zones_db · audit_db): EBS daily snapshot · replica set for HA at scale-out · audit_db Change Streams enable real-time audit monitoring Redis: Volatile at MVP — on restart: sessions require re-auth, cache rebuilds from SQL (~15 min) S3: Cross-region replication · versioning · Object Lock (WORM) for regulatory docs Encryption & classification PII: users_db · identity_db · sos_db · driver_db · audit_db · S3 — SQL Server TDE · MongoDB WiredTiger · S3 SSE-S3 Financial (PCI): payment_db — Paymob tokens field-level AES-256 Audit WORM: audit_db append-only · Change Stream monitored by Prometheus · delete attempt → immediate alert All transmission TLS 1.2+ enforced at gateway and IIS					
15 data stores · 10 SQL Server per-service · 3 MongoDB (GPS + Zones + Audit WORM) · 4 Redis key-spaces (volatile TTL) · 1 AWS S3 bucket · sole-writer ownership enforced across all stores					

Figure 9. SheDrive Data Architecture — Ownership, Classification & Retention (section 4.4)

4.4.3 Data Classification

All data on the SheDrive platform is classified into one of seven sensitivity tiers. Classification determines the access controls, encryption requirements, logging restrictions, and retention policies applicable to each data category.

Classification	Data Categories	Examples	Controls Applied
Identity Documents & Photos (Highest sensitivity)	Driving licence images, vehicle registration images, driver profile photos, vehicle photos	Driving licence front/back, vehicle registration front/back, driver profile photo	Stored in AWS S3 (encrypted). Access via presigned URL (time-limited). Never included in logs. Retained only as long as required for the verification purpose. Never transmitted to any third party.
Personal Identifiable Information (PII)	Full name, mobile number, email, date of birth, national ID number, gender	Rider/Driver name and mobile, driver-entered 14-digit national ID number	Encrypted at rest (SQL Server TDE). TLS 1.2 in transit. Access restricted by RBAC. Not included in structured logs. Data subject access and erasure handled as platform policy.
Financial	Payment card tokens, fare amounts, commission records, cash settlement amounts	Paymob PSP token (never raw PAN), trip fare, driver commission owed	Raw card data (PAN) never stored on platform. Paymob hosted fields only. PSP tokens encrypted at rest. Financial records immutable. Audit trail mandatory. 180-day minimum retention baseline.
Safety & Incident	SOS trigger events, incident records, escalation logs, location at SOS trigger	SOS trigger timestamp, GPS coordinates, incident timeline, escalation notes	Immutable audit record per incident. Access restricted to Operations Supervisor and Administrator roles. 180-day minimum retention baseline. Location data at SOS trigger retained as incident evidence.
Operational	Trip state, ride events, driver location (active trips), zone config, pricing rules	Trip lifecycle state transitions, driver GPS coordinates during trips	Driver live location expires via MongoDB TTL index (24h for live location; trip event log retained indefinitely). Zone and pricing config versioned and audited.
Audit & Compliance	Immutable audit events for all financial, identity, configuration, and SOS actions	Admin approval action, pricing rule change, payment capture event	MongoDB append-only collection. No delete or update permitted. Access restricted to Administrator and Compliance User roles. All access events themselves logged. 180-day minimum retention baseline (platform policy).
System & Operational Logs	Structured service logs, distributed traces, health check records	API request logs, exception logs, RabbitMQ consumer event logs	No PII, PAN, or sensitive data in any log event. Stored in Seq with 180-day retention. OpenTelemetry traces retained 30 days. Access restricted to Engineering and Operations roles.

4.4.4 Data Lifecycle and Retention

The following table defines the lifecycle of each major data category. All retention policies satisfy the 180-day minimum retention baseline defined as platform data-governance policy (Section 3.3). Data subject erasure requests are honoured as platform policy, except where an open dispute, incident, or financial reconciliation requires the record to be kept.

Data Category	Retention Period	Policy	Notes
Trip records	Indefinite	Never deleted. Created on ride request submission; retained as the source for reporting, dispute resolution, and analytics.	Includes the immutable trip_state_history timeline per trip.
Payment records	Indefinite	Never deleted. Created on trip completion or cancellation with fee; financial audit requirement.	180-day minimum baseline always exceeded.
Driver live location	Active trip duration; 24 h TTL	MongoDB TTL index expires live location documents 24 hours after creation.	Trip-time location snapshots are preserved in the trip event log.
Trip event log	Indefinite	Retained indefinitely as operational evidence and reporting source.	—
Audit events. Tier 1 (Financial, Safety, Legal)	Permanent (never deleted)	Append-only MongoDB collection; no delete or update permitted by any role or service. Tier 1 events: PaymentCaptured, RefundInitiated, CommissionCalculated, CancellationFeeCharged, CashSettlementRecorded, SOSTriggered, SOSResolved, GenderMismatchReported, RiderSuspended. Permanently retained, financial liability, safety accountability, and legal evidence have no bounded statute of limitations.	RetentionTier = 1 stamped at event creation. Erasure requests: record retained in pseudonymised form; data-subject linkage removed.
Audit events. Tier 2 (Identity, Account Lifecycle)	7 years (configurable)	Tier 2 events: DriverApproved, DriverRejected, DriverActivated, DriverDeactivated, DriverBlocked, RiderAccountBlocked, UserRoleChanged. Identity decisions and account lifecycle changes traceable beyond the active account lifetime for dispute resolution and governance.	RetentionTier = 2. Archived to S3 Glacier after 7 years. Retention period configurable in Admin Portal without code deployment.
Audit events. Tier 3 (Admin Configuration)	2 years (configurable)	Tier 3 events: PricingRuleChanged, ZoneBoundaryUpdated, OperatingHoursChanged, CommissionRateChanged, NotificationTemplateUpdated. Configuration change history supports rollback and audit;	RetentionTier = 3. Archived to S3 Glacier after active retention period; cold-storage deletion after configurable extension window.

Data Category	Retention Period	Policy	Notes
		operational value diminishes after 2 years.	
Onboarding documents and photos (S3)	Duration of account + 90 days	Retained while the driver account is active; moved to S3 Glacier 90 days after account deactivation.	Deleted on erasure request as platform policy unless an open dispute or incident requires retention.
Session tokens (Redis)	Session TTL	Auto-expired by Redis on TTL; explicitly invalidated on logout and account suspension.	Volatile data; never the source of truth.
Notification logs	90 days	Delivery logs retained for operational SLA review, then archived or purged.	No PII retained beyond delivery confirmation.
System logs (Seq)	180 days	Purged on rolling 180-day window aligned to the data-governance baseline.	No PII or sensitive data is present in logs.
Distributed traces (OpenTelemetry)	30 days	Trace data retained for performance analysis and incident investigation, then purged.	Traces carry correlation IDs only; no payload data.
Pricing rule versions	Indefinite	All versions retained with effective dates for audit and rollback.	Versioning is mandatory for every change.
Hangfire job history	30 days	Succeeded-job history trimmed on a rolling window; failed jobs retained until resolved.	Job storage resides in SQL Server.

4.4.5 Data Residency

All primary platform data (SQL Server, MongoDB, Redis, RabbitMQ) resides on EC2 VMs provisioned in the AWS Middle East (UAE) or Europe (Frankfurt) region, selected to minimise latency from Egypt. AWS S3 buckets containing onboarding documents and photos are provisioned in the same region. The table below defines every data category that crosses the platform boundary to an external provider, what is shared, and how it is handled.

Data Category	External Destination	Data Shared	Residency / Handling
Card payment data	Paymob (PSP)	Card number, expiry, CVV, entered by the Rider directly into Paymob's hosted payment fields.	Card data never transits SheDrive infrastructure; only the PSP-issued token is returned and stored. Paymob is a licensed Egyptian PSP.
Payment transaction data	Paymob (PSP)	Amount, currency, order reference, and token reference for pre-authorisation, capture, and refund calls.	Server-to-server over TLS from the Payment Service; webhook callbacks verified by HMAC signature.

Data Category	External Destination	Data Shared	Residency / Handling
Routing and geocoding requests	Google Maps Platform	Pickup/drop-off coordinates and route waypoints (no user identity attached).	All billable server-side calls routed through the Maps & Location Service; requests carry no PII. Client-side SDK renders map tiles on-device.
Push notification payloads	Firebase FCM / Apple APNS	Device token, ride reference identifiers, and status codes.	No PII in any push payload. Push is best-effort; authoritative state is always pulled from the platform.
SMS messages	Local Egyptian SMS Gateway	Recipient mobile number plus OTP code or brief bilingual alert text.	No sensitive personal data beyond the recipient number. SOS SMS uses a dedicated dispatch path.

4.5 Security View

This section defines the security architecture of the SheDrive platform, the authentication and authorisation model, role-based access control matrix, encryption controls, PCI DSS scope boundary, secrets management approach, and network security configuration. Security is a foundational architectural concern on SheDrive, given the sensitivity of identity documents and photos, financial transaction records, and the safety-critical SOS workflow.

4.5.1 Authentication and Authorisation Flow

The platform uses JWT-based authentication issued by the User Management Service and validated at the Ocelot API Gateway and re-enforced at the service layer. The following table describes the full authentication lifecycle.

Step	Description
1. Registration (OTP)	User submits mobile number. User Management Service sends OTP via SMS gateway. User submits OTP. On validation, account created. JWT not issued yet, account is unverified.
2. Login (OTP)	Registered user submits mobile number. OTP sent via SMS gateway. On valid OTP submission, User Management Service issues a signed JWT (HS256 / RS256) containing: UserId, Role, VerificationStatus, IssuedAt, ExpiresAt.
3. Admin 2FA Login	Admin Portal user submits credentials. User Management Service validates credentials, then issues a TOTP or SMS OTP challenge. On successful second factor, JWT issued with AdminRole claim and a shorter session expiry.
4. Ocelot Gateway Validation	Every inbound request to the Ocelot API Gateway carries the JWT in the Authorization Bearer header. Ocelot validates the JWT signature, expiry, and issuer before routing. Requests without a valid JWT are rejected with HTTP 401 before reaching any microservice.
5. Service-Level RBAC	Each microservice re-validates the JWT claims on every request. Role claims are mapped to permitted operations at the service layer. For example, a Finance User JWT can access the cash reconciliation endpoint but not the driver approval endpoint. This double-enforcement prevents any bypass if the gateway is misconfigured.

Step	Description
6. Session Expiry & Renewal	JWT tokens have a defined expiry (mobile: longer session, Admin Portal: shorter session). Refresh tokens are issued alongside JWTs for mobile clients. On expiry, the client presents the refresh token to obtain a new JWT without re-authenticating. Refresh tokens are stored in Redis and invalidated on logout or account suspension.
7. Logout & Invalidation	On explicit logout, the JWT is added to a short-lived Redis denylist for the remainder of its TTL. The refresh token is deleted from Redis. On account suspension, all active refresh tokens for that user are purged from Redis immediately.

4.5.2 Role-Based Access Control Matrix

The following matrix defines the capabilities available to each platform role. A tick (✓) indicates the role has access; a cross (✗) indicates access is denied at both the API Gateway and service layers. The matrix is enforced in code; not by configuration, and is validated by automated RBAC test coverage.

Capability / Action	Admin	Ops Sup.	Support	Finance	Compliance	Mgmt	Driver / Rider
Driver onboarding review & approval	✓	✓	✗	✗	✓	✗	✗
Driver account suspend / deactivate	✓	✓	✗	✗	✗	✗	✗
Rider account suspend / block	✓	✓	✓	✗	✗	✗	✗
Zone & pricing configuration	✓	✗	✗	✗	✗	✗	✗
Commission rate configuration	✓	✗	✗	✗	✗	✗	✗
Live ride monitoring	✓	✓	✓	✗	✗	✗	✗
SOS queue & escalation	✓	✓	✗	✗	✗	✗	✗
Cash reconciliation dashboard	✓	✗	✗	✓	✗	✗	✗
Manual refund processing	✓	✗	✗	✓	✗	✗	✗
Audit log viewer	✓	✗	✗	✗	✓	✗	✗
Verification document review	✓	✗	✗	✗	✓	✗	✗

Capability / Action	Admin	Ops Sup.	Support	Finance	Compliance	Mgmt	Driver / Rider
Rider suspension review (gender-mismatch reports)	✓	✗	✗	✗	✓	✗	✗
Structured data export	✓	✗	✗	✗	✓	✗	✗
Notification template management	✓	✗	✗	✗	✗	✗	✗
KPI & management dashboard	✓	✓	✗	✓	✗	✓	✗
Own profile & preferences	✗	✗	✗	✗	✗	✗	✓
Ride booking & tracking	✗	✗	✗	✗	✗	✗	Rider only
Trip acceptance & execution	✗	✗	✗	✗	✗	✗	Driver only

4.5.3 Encryption, Network, and Data Security Controls

The following table defines the full set of security controls applied to the SheDrive platform, covering encryption, PCI scope, secrets, network isolation, web application protection, IAM, and identity document and photo handling.

Security Control	Implementation
Encryption in Transit	TLS 1.2 minimum enforced on all connections: mobile clients → Ocelot gateway, Admin Portal → Ocelot gateway, Ocelot → microservices (internal VPC), microservices → SQL Server, microservices → MongoDB, microservices → Redis, all outbound calls to external providers. Certificates for internet-facing endpoints are issued and auto-renewed via a TLS certificate provider.
Encryption at Rest. SQL Server	Transparent Data Encryption (TDE) enabled on all SQL Server databases. Encryption keys protected via Windows DPAPI with the master key material held in the self-hosted Vault. Covers all data files, log files, and backups.
Encryption at Rest. MongoDB	MongoDB Encrypted Storage Engine (WiredTiger) enabled. Encryption keys managed locally with key material held in the self-hosted Vault. All document collections, index files, and journal files encrypted.
Encryption at Rest. Redis	Redis data is volatile (no persistence), encryption at rest is not applicable. Encryption in transit (TLS) between services and Redis is enforced. Sensitive data (PAN, identity documents, photos) is never stored in Redis.
Encryption at Rest. AWS S3	Server-side encryption (SSE-S3) enabled on all S3 buckets containing onboarding documents and photos. Bucket policies enforce HTTPS-only access. Presigned URLs are time-limited (maximum 15 minutes) and scoped to the specific object key.

Security Control	Implementation
PCI DSS Scope Reduction	Raw payment card numbers (PAN) are never transmitted to or stored on SheDrive platform infrastructure at any point. All card data entry is handled exclusively through Paymob's hosted payment fields rendered in the Rider App. Only Paymob-issued tokenised references are stored in the SheDrive Payment Service database. The platform operates outside PCI DSS merchant scope for card data storage and processing.
Secrets Management	All secrets (database passwords, PSP API keys, SMS gateway tokens, JWT signing keys) are stored in a self-hosted HashiCorp Vault instance. Services authenticate to Vault at startup (AppRole) and retrieve secrets into memory only; no secrets in application configuration files, environment variable files, or source code repositories. DPAPI-protected bootstrap configuration secures the Vault access credentials on each VM. Secret rotation is operated through Vault policies.
Network Security. VPC	All EC2 VMs (application, database, cache, broker, monitoring) are deployed in private VPC subnets with no direct internet access. Only the Ocelot API Gateway layer VMs are internet-facing (via AWS ELB). All outbound internet calls (to external providers) egress via NAT Gateway from private subnets. VPC Flow Logs enabled for network traffic monitoring and forensic investigation.
Network Security. Security Groups	Each EC2 VM layer has a dedicated Security Group. Inbound rules are least-privilege: Application VMs accept traffic only from the ELB Security Group. Database VMs accept traffic only from Application VM Security Group. RabbitMQ VMs accept traffic only from Application and Background Processing Security Groups. Monitoring VM accepts traffic only from Application VMs and authorised engineer IPs. No 0.0.0.0/0 inbound rules on any private-layer Security Group.
WAF (Web Application Firewall)	Web application protection is applied at the gateway layer: IIS request filtering and dynamic IP restriction modules on the gateway VMs, Ocelot per-client and per-endpoint rate limiting, and strict input validation at every service boundary. An external edge WAF/CDN service (e.g., Cloudflare) can optionally be placed in front of the platform for managed OWASP rule sets and DDoS absorption without any AWS service dependency. Security events are streamed to Seq for correlation alongside application logs.
IAM Least Privilege	Each EC2 VM instance profile grants access only to the AWS resources that VM requires, which, under the hosting constraint, is limited to scoped S3 bucket access for the VMs that serve document upload and review flows. No wildcard permissions. Cross-VM IAM role sharing is prohibited. All other credentials (database, broker, Vault) are platform-managed, not AWS-managed.
Biometric & Sensitive Data Handling	Driver documents (driving licence, vehicle registration), vehicle photos, and profile photos are uploaded directly from the Driver App to AWS S3 via presigned upload URLs, the images are never routed through the application tier. References (S3 object keys) are stored on the SQL Server DriverApplication record. Photos and documents are not retained beyond the purpose for which they were collected; on account deactivation a data erasure review is initiated as platform policy.

4.6 Integration View

This section defines all external service integrations on the SheDrive platform. Each integration is accessed exclusively through a dedicated adapter in the Maps & Location Service, Payment Service, or Notification Service; no other service or layer calls an external provider directly. Each

adapter encapsulates the provider's protocol, authentication, payload mapping, error normalisation, timeout enforcement, and circuit breaker configuration. In Release 1 there are four external integrations: Paymob (payments), Google Maps Platform (maps and location), FCM/APNS (push notifications), and the local Egyptian SMS gateway. No identity verification or background check provider is integrated; verification is manual and human-led (Section 4.2.5).

4.6.1 Paymob. Payment Service Provider

Role on the Platform	Processes all digital card payments on the SheDrive platform. Provides hosted payment fields for PCI-compliant card capture, card tokenisation, payment pre-authorisation on trip start, final fare capture on trip completion, and manual refund initiation from the Admin Portal.
Protocol / API Style	HTTPS REST API (JSON). Paymob Hosted Payment Fields rendered in the Rider App for card entry; no card data transits SheDrive servers.
Authentication	API key authentication. API key stored in the self-hosted Vault, retrieved at startup by the Payment Service. All API calls include the key in the request header. Webhook callbacks verified by HMAC signature.
Timeout Policy	Payment pre-authorisation: 10-second timeout. Payment capture: 15-second timeout. Refund: 30-second timeout. Timeouts enforced by the Polly resilience library within the Payment Service adapter.
Circuit Breaker	Circuit breaker opens after 5 consecutive failures or a 50% failure rate within a 60-second window. Half-open state after 30 seconds, a single probe request tests recovery before full traffic is restored.
Failure Behaviour	Pre-authorisation failure: ride is not started; Rider is notified with retry option. Capture failure after trip completion: trip is marked as payment-pending; Rider is notified; retry is offered; persistent failure blocks new bookings. Refund failure: logged and flagged in the Admin Portal for manual follow-up.
Fallback / Degradation	No automated fallback to an alternative PSP in the MVP (single PSP). On persistent circuit-open, cash payment is not affected; only digital payment trips are impacted. Finance User is alerted via Admin Portal dashboard alarm.

Integration Responsibility and Scope

The Paymob integration is split across two clearly bounded sides so that each team knows exactly what it builds. The Rider App (mobile) is the only surface that integrates with Paymob client-side; the Payment Service is the only component that integrates with Paymob server-side. The Admin Portal and Driver App never communicate with Paymob, they consume Payment Service APIs only.

Rider App (Flutter). ARCorp Mobile Team	Renders Paymob hosted payment fields (Paymob SDK / hosted checkout) for card entry and saved-card selection. Receives the tokenisation result and passes only the token reference to the Payment Service. Displays payment outcome states delivered by the platform (paid, retry offered, blocked). Never handles, stores, or logs raw card data, card entry happens inside the Paymob-hosted component.
Payment Service (.NET). ARCorp Backend Team	Owns the full server-side cycle: merchant authentication, payment intention creation, pre-authorisation at trip start, capture on TripCompleted, refunds initiated from the Admin Portal, and webhook endpoint for asynchronous payment status callbacks (HMAC-verified, idempotent). Persists tokens, attempts, and terminal payment states; publishes payment events to RabbitMQ.

Admin & Operations Portal (Angular)	No direct Paymob integration. Initiates refunds and views reconciliation exclusively through Payment Service APIs.
Driver App (Flutter)	No Paymob integration of any kind. Cash flows are confirmed in-app and recorded by the Payment Service ledger.
Paymob (provider side)	Provides sandbox and production merchant accounts, hosted payment fields, tokenisation, pre-auth/capture/refund APIs, webhook delivery, and integration documentation and test cards for SIT.

4.6.2 Google Maps Platform. Maps & Location Service

Role on the Platform	Provides geocoding, reverse-geocoding, routing, distance calculation, ETA computation, and map rendering for the Rider App, Driver App, and the Maps & Location Service. All billable server-side Maps API calls are routed exclusively through the Maps & Location Service (anti-corruption layer); no other service calls Google Maps directly.
Protocol / API Style	HTTPS REST API (JSON) for server-side calls (Geocoding, Directions/Routes, Distance Matrix). Google Maps SDK for Flutter for client-side map rendering and marker tracking in the Rider and Driver Apps.
Authentication	API key authentication. Separate, individually restricted API keys for server-side and client-side usage. Server key stored in the self-hosted Vault; client keys embedded in app bundles with application-ID and API-scope restrictions applied in the Google Cloud console.
Timeout Policy	Geocoding / Directions: 5-second timeout. Distance Matrix: 8-second timeout. ETA recalculation (active trips): 3-second timeout. Enforced by Polly within the Maps & Location Service adapter.
Circuit Breaker	Circuit breaker opens after 5 consecutive failures or 50% failure rate in 60 seconds. Half-open after 30 seconds. Separate circuit breaker instance per API endpoint type (Geocoding, Directions, Distance Matrix).
Failure Behaviour	Geocoding failure: ride booking fails with user-facing error; Rider prompted to retry or adjust the pin manually. ETA calculation failure during active trip: last known ETA displayed; silent retry on next location update. Matching failure: ride request is queued for retry.
Fallback / Degradation	Cache-aside in Redis for frequently requested routes and ETAs, a cache hit bypasses the Maps API call entirely. On persistent circuit-open, the Maps & Location Service returns cached estimates where available; matching falls back to straight-line distance approximation until the circuit recovers.

Recommended Employment Approach (Technical and Commercial)

The platform employs Google Maps in a deliberate hybrid model that optimises both cost and reliability:

- Client-side rendering, server-side computation, map tiles, camera control, and live marker animation run on-device through the Maps SDK for Flutter (high-volume, low unit cost), while every billable computation call (geocoding, routing, distance matrix) is centralised in the Maps & Location Service. This concentrates spend where it can be cached, throttled, and monitored.
- Aggressive server-side caching. Redis cache-aside for repeated route combinations, zone-anchored geocoding results, and short-lived ETA results. Cached responses cut billable call volume substantially at MVP traffic patterns and keep the platform inside or near the monthly free usage credit at launch volumes.

- Key hygiene and blast-radius control, server and client API keys are separate and individually restricted (application ID, IP, API scope), so a leaked client key cannot be used to drive server-side billable APIs.
- Commercial controls, the Google Cloud billing account is owned by the SheDrive Team (not ARCorp), with budget alerts and a hard billing cap configured from day one, and weekly usage review against the free-credit threshold during the launch period.
- Portability, because no service except Maps & Location knows the provider contract, an alternative provider (e.g., Mapbox, HERE) can be introduced behind the same adapter in a later phase without touching business logic or mobile booking flows (client map rendering would be swapped at the app layer).

4.6.3 Firebase Cloud Messaging (FCM) / Apple Push Notification Service (APNS)

Role on the Platform	Delivers push notifications to the SheDrive Rider App and Driver App on Android (via FCM) and iOS (via APNS). Used for ride lifecycle events, SOS alerts, onboarding decision updates, and payment outcome notifications. SOS alert push notifications are configured with critical alert priority to bypass device quiet hours. Push is best-effort by design: every push-driven flow has a pull-based state reconciliation path on app open or resume.
Protocol / API Style	FCM: HTTPS REST API v1 (JSON), Firebase Admin SDK (.NET). APNS: HTTP/2 protocol via Apple's servers, managed through the Firebase Admin SDK (FCM acts as the APNS relay for iOS in this configuration). Device tokens managed per user by the Notification Service.
Authentication	FCM: OAuth 2.0 service account credentials (Google), stored in the self-hosted Vault. APNS: APNs Auth Key managed via the Firebase Console.
Timeout Policy	FCM dispatch: 5-second timeout per notification batch. SOS notifications: 3-second timeout with immediate retry on failure (higher priority path). Enforced by the Notification Service adapter.
Circuit Breaker	Circuit breaker opens after 10 consecutive FCM failures in 30 seconds. Half-open after 15 seconds. Push notification delivery is inherently best-effort. FCM/APNS do not guarantee delivery to offline devices.
Failure Behaviour	On FCM failure: notification is queued for retry with backoff (up to 3 retries). On persistent failure: notification event is logged to the dead-letter queue for operational review. For SOS alerts: SMS fallback is triggered immediately in parallel. SOS safety communication does not depend solely on push delivery. For onboarding decisions: approval state is always visible on the driver's next app open via the status check API even if the push never arrives.
Fallback / Degradation	SMS Gateway is the explicit fallback for all safety-critical SOS notifications. Push and SMS are dispatched in parallel for SOS events, push failure does not delay or block SMS delivery.

4.6.4 Local Egyptian SMS Gateway

Role on the Platform	Delivers SMS messages for OTP authentication (registration and login), SOS safety alerts to Riders and Drivers, live trip sharing links to Trusted Contacts, payment failure alerts, and onboarding status notifications. SMS is the foundational communication channel, it operates without requiring an internet connection or an installed app on the recipient device.
-----------------------------	--

Protocol / API Style	HTTPS REST API (JSON or form-encoded, provider-specific). Bilingual message templates (Arabic and English) submitted as pre-rendered message body strings. Long messages split into segments by the gateway.
Authentication	API key or username/password authentication (provider-specific). Credentials stored in the self-hosted Vault.
Timeout Policy	OTP delivery: 5-second timeout. SOS alert SMS: 3-second timeout with immediate retry on failure. Standard notification SMS: 8-second timeout. Enforced by the Notification Service adapter.
Circuit Breaker	Circuit breaker opens after 5 consecutive failures in 30 seconds. Half-open after 20 seconds. Separate circuit breaker for the SOS alert path to prevent SOS SMS from being affected by bulk notification failures.
Failure Behaviour	On OTP delivery failure: user is shown a retry option with a countdown timer. On SOS SMS failure: the failure is logged with immediate alert to the Operations team. SOS response is not blocked by SMS failure. On general notification failure: event is queued in dead-letter queue for review.
Fallback / Degradation	For OTP: the platform allows up to 3 OTP resend attempts before requiring the user to wait a cooldown period. For SOS: push notification is dispatched in parallel, push delivery is not dependent on SMS success. No automated gateway fallback at MVP (single gateway).

5. Deployment View

This section describes the deployment architecture for the SheDrive platform, the target environments, the AWS service topology, and the hosting configuration standards applied across all environments. The deployment model is designed to support controlled progressive validation from development through production, maintain regulatory compliance requirements, and provide operational resilience appropriate to the MVP scale target.

5.1 Environments

The SheDrive platform is deployed across five logical environments throughout its delivery and operational lifecycle. Multiple environments are required to enable parallel development and testing activities, progressive validation of functional and integration requirements, and risk reduction before changes are introduced into the live production system. All environments are managed through a centralised infrastructure-as-code base (Terraform for AWS provisioning, Ansible/PowerShell DSC for VM configuration) and deployed exclusively via Azure DevOps pipelines, ensuring environment parity and eliminating manual configuration drift.

The following table describes each environment, its intended audience, and its purpose and infrastructure characteristics.

Environment	Audience	Purpose and Characteristics
Development (DEV)	ARCorp Engineering team	Primary daily development environment. Each developer works against a shared environment or a local Docker Compose service set. DEV is the first integration point for new features and bug fixes. Infrastructure is minimal, single VM per layer, no high-availability configuration, and smaller database instances. Automated unit and component test suites run on every commit

Environment	Audience	Purpose and Characteristics
		via the Azure DevOps pipeline. DEV data is fully synthetic and contains no real user information. This environment is reset or refreshed freely as required by the engineering team.
Testing (TEST)	ARCorp QC team	Dedicated testing environment owned and operated by the ARCorp Quality Control team. Receives a stable, deployable build promoted from DEV after passing the pipeline quality gate. The QC team executes structured functional test cases, regression suites, exploratory testing, and defect verification cycles in this environment. TEST is isolated from DEV activity; no in-progress feature work is deployed here. External provider integrations use sandbox credentials. TEST data is synthetic. Successful TEST sign-off by the QC Lead gates promotion to SIT.
System Integration Testing (SIT)	ARCorp Engineering · QC	Integration validation environment where all microservices are deployed together at their latest tested build and exercised end-to-end. SIT validates cross-service interactions, message queue event flows, external provider integrations (Paymob sandbox, Google Maps test keys, SMS gateway test account), and the full ride lifecycle across all modules. Infrastructure mirrors the production topology at reduced scale, single-AZ, smaller database tier, single VM per layer. SIT data is synthetic. Successful SIT completion gates promotion to UAT / Pre-Production.
UAT / Pre-Production	SheDrive Team · ARCorp QC	Combined User Acceptance Testing and Pre-Production environment. This environment serves two sequential purposes within the same infrastructure tier. In the UAT phase, SheDrive Team stakeholders conduct scenario-based acceptance testing against acceptance criteria derived from the BRD, with the ARCorp QC team providing test coordination. In the Pre-Production phase, final smoke testing, infrastructure validation, and production-readiness checks are performed before go-live. This environment mirrors Production configuration, zone definitions, and pricing rules as closely as feasible. External integrations use production-equivalent sandbox or staging credentials. Anonymised or synthetic data only. Formal sign-off by the SheDrive Product Owner and ARCorp QC Lead is required before Production deployment is authorised.
Production (PROD)	Live users · SheDrive Operations	The live operating environment serving real Female Riders, Female Drivers, and Back-office Users. Multi-AZ deployment across two AWS Availability Zones for service resilience (2× VMs per layer). Self-hosted SQL Server (AlwaysOn) and MongoDB (replica set) are configured with automated backup, point-in-time recovery, and cross-AZ replication. Redis runs with Sentinel failover. All observability tooling (Seq, Prometheus, Grafana, OpenTelemetry) is fully active. Access is strictly restricted; no direct developer access to Production infrastructure; all changes arrive exclusively through the Azure DevOps pipeline following UAT sign-off. Log and data retention is set to a minimum of 180 days per the platform data-governance baseline.

Environment promotion follows a linear gate model: DEV → TEST → SIT → UAT/Pre-Production → Production. Promotion between environments is triggered only by a successful pipeline execution that passes all applicable quality gates, automated tests, security scans, and where applicable human approval gates from the ARCorp QC Lead or the SheDrive Product Owner. No direct deployment to SIT, UAT, or Production is permitted outside this pipeline flow.

[Figure 4. Deployment Environment Lifecycle and Promotion Flow]

Figure 4. SheDrive Environment Lifecycle (DEV → TEST → SIT → UAT/Pre-PROD → PROD)

5.2 Host Configuration

The SheDrive platform is hosted on Amazon Web Services using EC2 virtual machines as managed servers, plus AWS S3 for object storage, and nothing else from the AWS service catalogue. This is a deliberate hosting constraint (Section 3.3): all platform software including application services, databases, message broker, cache, secrets management, and the observability stack, is self-hosted on the VMs, keeping the platform fully portable to any VM-based hosting provider. All infrastructure is defined as code (Terraform for AWS provisioning; Ansible or PowerShell DSC for VM software configuration), versioned in the source repository, and deployed through the Azure DevOps pipeline, ensuring that infrastructure changes are subject to the same review, test, and promotion gates as application code.

The production topology comprises the following self-hosted layers, each deployed as 2× VMs for high availability (Section 7 defines the full sizing): Presentation & API Gateway (IIS: Ocelot + Angular SPA), Application Services (IIS: .NET 8 microservices), Background Processing (Hangfire workers and consumers), SQL Server (AlwaysOn), MongoDB (replica set), Redis (Sentinel), RabbitMQ (quorum cluster), and a dedicated Monitoring & Management VM (Seq, Prometheus, Grafana, OpenTelemetry Collector, Vault).

[Figure 7. Production Deployment Topology (EC2 VM-Based, Self-Hosted Stack)]

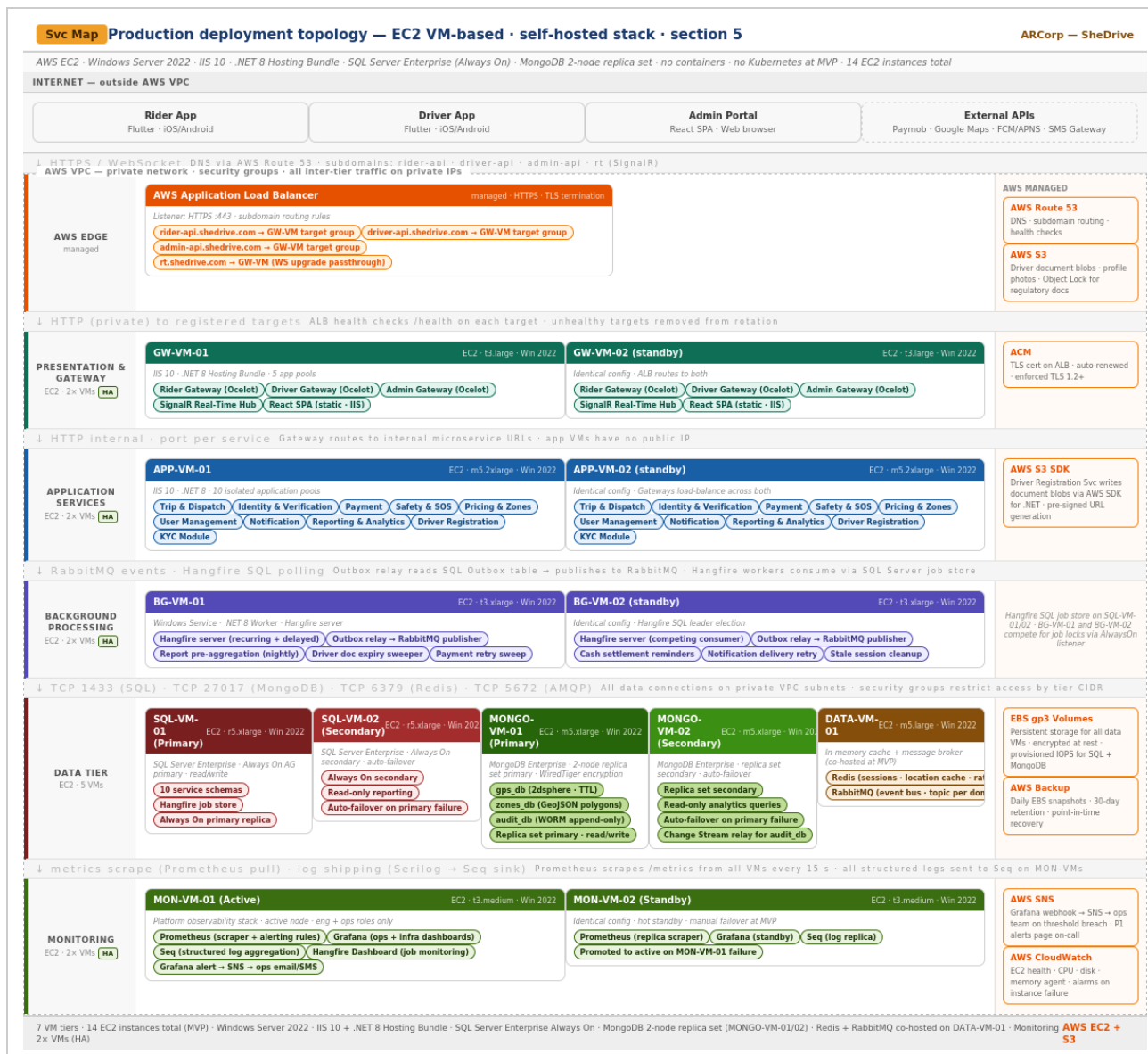


Figure 7. SheDrive Production Topology (AWS EC2 managed servers + S3)

5.2.1 Hosting Configuration Standards

The following configuration standards are applied consistently across all environments, with Production-specific stricter controls noted where applicable. These standards ensure that the platform operates on a secure and maintainable infrastructure and that the platform data-governance baseline, particularly the 180-day retention and access-control policies, is met by design.

Network Isolation	All compute, database, cache, and broker VMs are deployed in private VPC subnets with no direct internet exposure. Only the Presentation & API Gateway VM layer is internet-facing. All internal service-to-service communication stays within the VPC. Outbound calls to external providers egress via NAT from private subnets.
Transport Security	TLS 1.2 is the minimum enforced standard on all client-to-gateway, gateway-to-service, and service-to-external-provider connections. Certificates are issued and auto-renewed via a TLS certificate provider on the gateway VMs. No unencrypted HTTP traffic is permitted in SIT, UAT, or Production environments.

Secrets and Credentials Management	No secrets, API keys, database passwords, or signing keys are stored in VM images, configuration files, or source code repositories. All credentials are retrieved at runtime from the self-hosted HashiCorp Vault instance via AppRole authentication, with DPAPI-protected bootstrap configuration per VM. Secret rotation is operated through Vault policies.
Encryption at Rest	All SQL Server databases use Transparent Data Encryption (TDE). MongoDB uses the WiredTiger encrypted storage engine. S3 buckets use server-side encryption (SSE-S3). EBS volumes attached to database VMs are encrypted. Redis holds volatile data only; no persistence of sensitive data.
Identity and Least-Privilege Access	EC2 instance profiles grant only scoped S3 access where required; no other AWS-service permissions exist. Administrative access to Production VMs requires MFA-protected bastion access with full session logging; direct developer access to Production is prohibited. All changes flow through the Azure DevOps pipeline.
Operating System Hardening	Windows Server VMs are built from a hardened golden image: unused roles and features removed, local firewall enabled with least-privilege rules, automatic security updates within a defined maintenance window, and anti-malware enabled. All VMs run Windows Server (latest); a single hardening baseline applies across all roles.
Time Synchronisation	All VMs synchronise time against a common NTP source, ensuring consistent and accurate timestamps across structured logs, audit records, trip state history, and distributed traces. Time consistency is critical for audit records and SOS SLA measurement.
Log Retention and Data Residency	Seq log retention is configured to a minimum of 180 days, aligned to the platform data-governance baseline. All VMs and S3 buckets are provisioned in the AWS Middle East (UAE) or Europe (Frankfurt) region to minimise latency from Egypt.
Backup and Recovery	SQL Server native backups run daily (full) and hourly (log) to S3, with 7-day retention in TEST/SIT and a 35-day rolling window in Production. MongoDB replica-set backups support point-in-time recovery. S3 document buckets use versioning to recover accidentally deleted or overwritten files. Recovery procedures are documented and validated in Pre-Production before each Production release.
Web Application Protection	IIS request filtering and dynamic IP restriction on the gateway VMs, Ocelot per-client and per-endpoint rate limiting, and strict input validation at every service boundary. An external edge WAF/CDN (e.g., Cloudflare) may optionally front the platform for managed OWASP rule sets and DDoS absorption. Security events stream to Seq for correlation.

5.2.2 Availability and Resilience Configuration

The production deployment targets 99.5% availability of scheduled operating hours within the configured daytime window. The following resilience measures are applied:

- **Multi-Availability Zone (Multi-AZ)**, every VM layer is deployed as 2× VMs split across two AWS Availability Zones. SQL Server AlwaysOn maintains a synchronous secondary in the second AZ with automatic failover. The MongoDB replica set and Redis Sentinel topology span both AZs. RabbitMQ quorum queues replicate across the two AZs. A single AZ failure does not take the platform offline.
- **Load-Balanced Stateless Application Layer**, the gateway and application VMs are stateless (session state externalised to Redis) and sit behind the load-balancing layer with readiness-

probe-based rotation. A failed VM is removed from rotation automatically and replaced from the golden image via the IaC pipeline.

- **Database Automated Failover.** SQL Server AlwaysOn promotes the secondary replica automatically; the availability group listener endpoint remains unchanged and application services reconnect through connection retry logic built into the .NET data access layer. MongoDB elects a new primary automatically; Redis Sentinel promotes a replica on master failure.
- **Message Durability.** RabbitMQ quorum queues persist messages across both nodes; a single node outage does not result in message loss. Dead-letter queues capture messages that exceed the retry threshold for operational review.
- **S3 Durability.** Amazon S3 provides 11-nines durability for stored objects. Onboarding documents and photos uploaded to S3 are durably stored and protected against hardware failure by design.
- **Health Checks and VM Replacement,** the load-balancing layer probes each service's readiness endpoint; unhealthy VMs are removed from rotation. Hangfire and consumer hosts expose equivalent health endpoints monitored by Prometheus with Grafana alerting.
- **Graceful Shutdown,** deployments drain in-flight requests: IIS app pools are taken out of rotation, complete in-flight requests within a defined drain period, and recycle cleanly. This prevents request loss during deployments and VM replacement.

5.3 Mobile Application Rollout Plan

The SheDrive mobile apps follow a phased rollout model, geographic expansion and platform (Android / iOS) expansion are sequenced to concentrate supply and demand, validate operational readiness at each step, and manage App Store / Play Store review timelines. No fixed dates are attached to phases; progression between phases is governed by operational sign-off against the criteria in the table below. The Admin Portal and backend services are deployed in full from Phase 1 and serve all phases.

Phase	Audience	Geography	Platform	Criteria and Notes
Phase 1 (Go-Live)	Female Riders + Drivers in the launch city	Single launch city (Cairo)	Android only	Concentrate supply and demand. Validate real-world first-trip verification, SOS response SLA, and payment flows before geographic expansion. Android-first reduces App Store review risk on the critical path. Play Store submission occurs 4+ weeks before go-live.
Phase 2 (Zone Expansion)	Same audience	Additional Cairo governorates / districts	Android	Expand zone polygons via Admin Portal configuration (no code deployment). iOS submission to App Store takes place during this phase, targeting Phase 3 timing. Monitor matching-success-rate and driver supply per zone before each zone activation.
Phase 3 (iOS Launch)	Same audience	Cairo (all active zones)	Android + iOS	iOS Rider App and Driver App released once App Store approval is received. iOS push delivery (APNS) validated in Phase 2 SIT. Concurrent sessions expected to increase; gateway concurrency reviewed against Section 7.5 triggers.

Phase	Audience	Geography	Platform	Criteria and Notes
Phase 4 (National Expansion)	Same audience	Additional Egyptian cities	Android + iOS	City-by-city expansion, each requiring zone configuration, driver onboarding drive, and operations staffing confirmation. Each new city is treated as a mini-Phase 1 with a supply-readiness gate before rider access is opened.
Phase 5 (R2 Scope)	Same audience + potential male-driven shuttle (separate product decision)	National	Android + iOS	Release 2 features (night window, automated eKYC, multi-city routing, corporate accounts) considered for roadmap. Requires formal BRD amendment and HLD update.

6. Operation View

This section describes the operational model for the SheDrive platform, the DevOps enablement strategy, CI/CD pipeline, engineering inner-loop practices, observability and monitoring configuration, and the feedback loop that connects production operations back into the development lifecycle. The operation view ensures that the platform can be delivered continuously, monitored effectively, and evolved confidently over time.

6.1 DevOps Enablement and Operational Tooling

The SheDrive platform adopts an integrated DevOps model that treats delivery pipeline, infrastructure, quality validation, and operational monitoring as a unified operational capability rather than separate concerns. The outer loop of this model covers everything from source commit through production deployment and operational health monitoring.

6.1.1 CI/CD Pipeline Strategy

All build, test, and deployment activities are automated through Azure DevOps pipelines. Pipelines are defined as YAML in the source repository, versioned alongside the application code, and executed on Azure DevOps-hosted agents. Pipeline definitions are reviewed under the same pull request process as application code. The pipeline is structured as a sequence of progressive stages with explicit quality gates between environments, reflecting the five-environment promotion model defined in Section 5.1.

The following table defines the full CI/CD pipeline stage sequence for the SheDrive platform:

Pipeline Stage	Trigger	Actions and Quality Gates
1. Source & Build	Commit to any branch	Source checkout, dependency restore, .NET build (Release configuration), Flutter build, Angular build. Fails immediately on any compilation error. Versioned deployment package built and tagged with build number and Git commit SHA, published as a pipeline artifact.
2. Unit & Component Tests	Post-build (CI gate)	All .NET unit tests executed (xUnit). Code coverage report generated, minimum coverage threshold enforced (fails build if below configured %). Flutter

Pipeline Stage	Trigger	Actions and Quality Gates
		widget tests executed. Angular unit tests (Jest/Karma) executed. Test results published to Azure DevOps for trend tracking. Stage failure blocks all downstream stages.
3. Static Analysis & Security Scan	Post-unit tests	SonarQube (or equivalent) static analysis: code quality rules, duplication, complexity thresholds. OWASP dependency-check: known CVE scan on all NuGet, npm, and pub packages. License compliance check on open-source dependencies.
4. Deploy to TEST	Merge to develop branch	Terraform plan reviewed and applied for TEST environment (if infrastructure changes present). Rolling IIS deployment to the TEST VMs (one VM at a time, drained and verified). Smoke test suite executed post-deployment, verifies that all services are healthy and core API endpoints respond. QC team notified of deployment via Azure DevOps notification.
5. Functional & Regression Tests (TEST)	Post-TEST deployment	Full ARCorp QC test suite executed: functional test cases covering all BRD requirements, regression suite covering previously verified scenarios. API integration tests against internal service boundaries. External provider sandbox integration tests (Paymob, Google Maps, SMS). Test results published and reviewed by QC Lead. QC Lead approval required to proceed to SIT.
6. Deploy to SIT	QC Lead approval gate	Terraform apply for SIT environment. Rolling IIS deployment. Full end-to-end integration test suite: cross-service event flow validation, RabbitMQ message routing, WebSocket real-time location streaming, SOS alert delivery, payment capture flow, dual cash confirmation. SIT sign-off required to proceed to UAT.
7. Deploy to UAT / Pre-PROD	SIT sign-off gate	Terraform apply for UAT/Pre-PROD environment. Rolling IIS deployment with health-gated VM rotation (zero downtime). SheDrive Team acceptance testing period. ARCorp QC provides support and defect management. Production-readiness checks: infrastructure parity validation, data migration dry-run (if applicable), rollback procedure test. Product Owner and QC Lead sign-off gate.
8. Production Release	Product Owner sign-off	Terraform apply for Production (change-controlled, reviewed plan). Rolling IIS deployment to Production with health-gated VM rotation. Post-deployment smoke tests against Production endpoints. Grafana deployment annotation created for metric correlation. Rollback plan available, the previous deployment package is retained for immediate redeployment if required. Release notification issued to SheDrive Operations team.

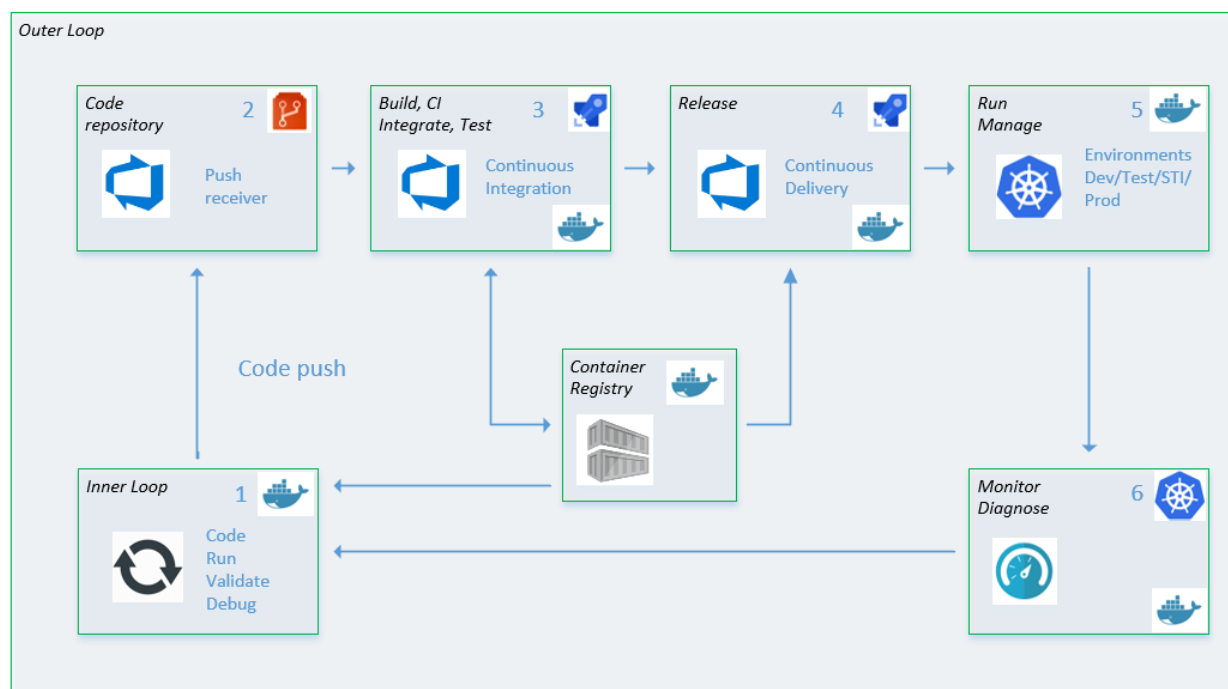


Figure 6. DevOps Outer Loop (source: ARCorp DevOps reference model)

6.1.2 Source Control and Branching Strategy

A disciplined Git branching strategy is applied across all SheDrive repositories, backend services, mobile apps, the Admin Portal, and the infrastructure-as-code repository. The strategy supports parallel feature development, stable release management, and emergency hotfix capability while preserving traceability between source changes and deployed builds.

Branch	Purpose and Rules
main / production	Protected branch. Represents the exact code running in Production at all times. Direct commits are prohibited. Updated only by merge from release/* branches after UAT sign-off and pipeline success.
release/*	Created from develop when a release candidate is ready for UAT. No new features; only bug fixes identified during UAT are applied here. Merged to main after Production deployment and back-merged into develop.
develop	Primary integration branch. All feature and fix branches are merged here after peer review and unit test success. Triggers the DEV and TEST deployment pipeline stages automatically on each merge.
feature/*	Short-lived branches for individual features or BRD requirements. Named by ticket reference (e.g., feature/DA-012-ride-request-popup). Merged to develop via pull request with mandatory peer review and CI gate.
fix/*	Short-lived branches for bug fixes. Follow the same PR and CI gate rules as feature branches. Applied to the relevant branch (develop or release/*) depending on severity and environment.
hotfix/*	Emergency production fixes only. Branched from main, applied directly to main after pipeline success, and back-merged into develop and any active release/* branch. Requires Product Owner notification.

6.1.3 Monitoring, Observability, and Alerting

Operational visibility is provided by a fully self-hosted observability stack on the dedicated Monitoring & Management VM: Seq (structured logs via Serilog), Prometheus (metrics scraped from service, IIS, OS, SQL Server, MongoDB, Redis, and RabbitMQ exporters), Grafana (dashboards and alert routing), and the OpenTelemetry Collector (distributed traces). The stack is configured from day one, monitoring is not added post-launch. All alert rules are defined as code alongside the infrastructure, and alerts are routed to the SheDrive Operations notification channel; critical alarms (SOS SLA breach, Production service failure, payment processing failure) additionally page the engineering on-call rotation.

Monitoring Area	Tool / Mechanism	What Is Monitored and Alert Conditions
Service Health	ASP.NET Core Health Checks + Prometheus	Liveness and readiness endpoints per microservice probed by the load-balancing layer and scraped by Prometheus (blackbox exporter). Unhealthy VMs are removed from rotation. Alert: ≥ 2 consecutive readiness failures on any service → Ops notification and on-call page.
API Performance	Prometheus + OpenTelemetry metrics	Gateway and per-service latency (p50, p95, p99) and 4xx/5xx error rates per endpoint. Alert: p95 latency > 2 s sustained for 5 minutes, or 5xx rate > 1% over 5 minutes → engineering on-call.
Real-Time Location Streaming	Custom service metrics → Prometheus	WebSocket connection count, message throughput, and connection error rate. Alert: connection error rate > 5% over 3 minutes, or location update lag > 5 seconds → Ops alert.
SOS Response SLA	Safety & SOS Service metrics + Grafana	Elapsed time from SOS trigger to first operator action tracked per incident. Alert: SLA threshold breach (configurable, default 3 minutes without operator response) → priority alert in the Admin Portal SOS queue and the Ops notification channel.
Payment Processing	Payment Service metrics + Grafana	PSP call success rate, payment capture failure rate, retry queue depth, and dead-letter queue depth. Alert: capture failure rate > 5% over 10 minutes → Finance and Engineering. Dead-letter queue depth > 0 → immediate alert.
Database Performance	SQL Server / MongoDB exporters	CPU, memory, free storage, connection count, read/write latency, AlwaysOn replica lag, and replica-set election events. Alert: CPU > 80% for 10 minutes, free storage < 20%, or replica lag beyond threshold → engineering on-call.
Message Queue Depth	RabbitMQ exporter → Prometheus	Queue depth per RabbitMQ queue, consumer utilisation, and unacked message counts. Alert: queue depth > 500 sustained for 5 minutes (consumer lag) → alert. Dead-letter queue depth > 0 → immediate engineering alert.
Cache Health	Redis exporter → Prometheus	Redis CPU, memory utilisation, eviction count, Sentinel failover events, and connection count. Alert: memory > 80% or eviction rate spike → investigate and consider scaling cache.
Background Jobs	Hangfire dashboard + metrics	Recurring job success/failure, queue lengths, and processing latency per job class. Alert: any failed recurring job (settlement reminders, expiry sweeps, payment retry sweeps) after retries → engineering

Monitoring Area	Tool / Mechanism	What Is Monitored and Alert Conditions
		alert. MassTransit outbox backlog growth (monitored via delivery service metrics) → immediate alert.
VM and OS Health	Windows/node exporters → Prometheus	CPU, memory, disk utilisation, and disk-queue length per VM; IIS app pool state and worker process recycling. Alert: disk > 80% (notably the Seq and database volumes), or app pool failure → Ops alert.
Security and Access	IIS logs + Vault audit + Seq	Authentication failure spikes, gateway rate-limit triggers, Vault access audit events, and bastion session logs streamed to Seq. Alert: abnormal authentication failure spike or unexpected Vault access pattern → security alert to operations and engineering leads.
Distributed Traces	OpenTelemetry Collector	End-to-end trace per request across the gateway and all microservices, annotated with correlation ID, ride ID, and user role. Used for post-incident root-cause analysis and latency profiling. Trace data retained 30 days.

6.1.4 Automated Quality Gates

Quality validation is enforced as a non-negotiable gate at each pipeline stage. The following gates apply:

- Build gate, compilation failure blocks all downstream stages immediately.
- Unit test gate, any test failure or coverage threshold breach blocks promotion to TEST.
- Static analysis gate, blocker or critical severity SonarQube findings block promotion.
- Dependency CVE gate, critical severity CVEs in NuGet, npm, or pub packages block promotion.
- Smoke test gate, any failing smoke test after deployment blocks progression to the next environment.
- QC sign-off gate. QC Lead approval in Azure DevOps required before SIT deployment.
- UAT sign-off gate. SheDrive Product Owner and ARCorp QC Lead approval required before Production deployment.
- Production deployment gate, infrastructure change plan reviewed and approved by Solution Architect before Terraform apply.

6.2 Inner Development Loop and Engineering Practices

The inner loop of the SheDrive DevOps model focuses on developer productivity, code quality, and early defect detection before changes reach shared environments. The following engineering practices are applied consistently across all ARCorp delivery teams working on the SheDrive platform.

Practice	Application on SheDrive
Peer Code Review	All pull requests to develop, release/*, and main branches require at least one peer reviewer approval before merge. Reviewers validate adherence to Clean Architecture layering, BRD requirement coverage, API contract consistency, and test coverage. The solution architect reviews all cross-service interface changes.

Practice	Application on SheDrive
Static Code Analysis	SonarQube (or equivalent) integrated into the Azure DevOps pipeline. Rules enforce code duplication thresholds, cyclomatic complexity limits, naming conventions, and .NET security hotspot detection. Analysis results are surfaced in the pipeline and block promotion on blocker or critical severity findings.
Automated Unit Testing	Each .NET microservice maintains an xUnit test project targeting the Domain and Application layers. Tests cover domain entity behaviour, business rule enforcement (women-only policy, age check, operating hours, cancellation fee logic), and application service orchestration. Mocks replace all infrastructure dependencies. Flutter widget tests cover Rider and Driver App UI components and navigation flows.
Coding Standards	A shared coding standards document is maintained in the repository. Standards cover: C# naming conventions (PascalCase, camelCase), async/await patterns, exception handling, logging format (structured JSON with mandatory CorrelationId field), and .NET Clean Architecture layer boundaries. Dart/Flutter standards cover widget structure, state management patterns, and API client conventions. Angular standards cover component responsibility boundaries and service injection patterns.
Dependency Vulnerability Scanning	OWASP Dependency-Check runs as part of every build. NuGet, npm (Angular), and pub (Flutter) packages are scanned against the NVD CVE database. Critical and high severity findings block the pipeline. A reviewed exception process exists for false positives with mandatory documentation.
Troubleshooting-Oriented Design	All .NET services use structured JSON logging (Serilog) with mandatory fields: CorrelationId, RideId (where applicable), UserId (where applicable), ServiceName, Environment, and Severity. Log statements at entry and exit of all service boundary calls. Exceptions are logged with full context before being translated to API error responses. No sensitive data (PAN, national ID numbers, identity documents or photos) is ever included in logs.
Local Development Environment	A Docker Compose configuration is maintained in the repository to enable developers to run all dependencies locally: SQL Server, MongoDB, Redis, and RabbitMQ. This eliminates dependency on shared DEV infrastructure for day-to-day development and accelerates the inner loop cycle time.

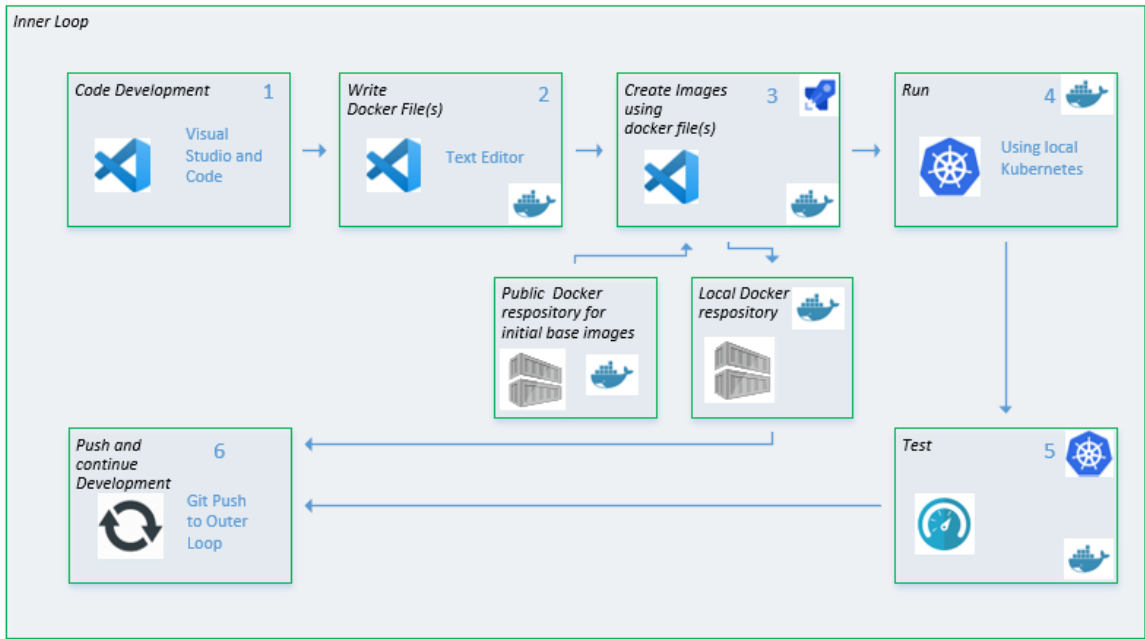


Figure 7. DevOps Inner Loop (source: ARCorp DevOps reference model)

6.3 Feedback Loop Between Operations and Development

The feedback loop is the mechanism that turns operational evidence into engineering and product action. It is deliberately lightweight: three cadences, one owner per signal, and a single backlog as the destination for every action. Findings that carry architectural implications are escalated to the Solution Architect and addressed through a formal architectural review before corrective implementation begins.

Cadences

Cadence	Signals Reviewed	Action Mechanism
Immediate (same day)	Production incidents, service degradation, SOS SLA breach, payment failure spike, data integrity issue.	Incident response now; post-incident review within 48 hours producing a root-cause finding and corrective backlog items. Critical incidents additionally produce an architectural finding reviewed by the Solution Architect and Product Owner.
Weekly (delivery stand-up)	Grafana alert trends, RabbitMQ consumer lag, SOS SLA compliance report, matching success and cancellation patterns, payment reconciliation report, QC defect trends.	Each signal has a named owner who presents a one-page trend view. Items crossing a defined threshold become backlog items with priority agreed in the meeting; zone, pricing, or driver-supply interventions are routed to SheDrive Operations.
Monthly (engineering review)	OpenTelemetry trace analysis of the slowest 1% of requests, Hangfire job performance, capacity utilisation against the Section 7.5 scaling triggers, accumulated architectural findings.	Produces performance improvement tasks for the next sprint cycle and a capacity recommendation. Architectural findings are dispositioned: accept, schedule, or redesign.

Signal Ownership

Signal Source	Owner	Feedback Mechanism and Action
Grafana alarms and anomaly trends	ARCorp DevOps Engineer	Threshold breaches that are not incidents are tracked over rolling 4-week windows; persistent patterns become engineering improvement tasks.
SOS response SLA report	SheDrive Operations Manager	Weekly distribution of first-response times, escalation rate, and resolution time. Systemic misses trigger operational procedure review and, where applicable, engineering changes to alert delivery or escalation logic.
Matching and cancellation analytics	SheDrive Operations + Product	No-driver-available patterns, zone-specific driver cancellation rates, and abnormal cancellation timing feed zone configuration, pricing, or driver-supply interventions.
Payment reconciliation report	SheDrive Finance User	Persistent payment failure patterns escalate to engineering for PSP integration, retry logic, or error-message investigation. Unresolved cash balances beyond the settlement threshold are escalated to drivers by Operations.
QC defect trends	ARCorp QC Lead	Recurring defect categories including regressions in a specific service, integration failures at a boundary, and mobile issues under poor network conditions, are flagged to the relevant technical lead for root-cause action.
Product feedback (UAT and post-launch)	SheDrive Product Owner	Usability issues, workflow gaps, and reporting deficiencies are triaged into the product backlog; anything revealing an HLD gap is escalated to the Solution Architect before implementation.

7. Hardware Sizing

This section defines the hardware and infrastructure sizing for the SheDrive platform across its production and pre-production environments. Sizing is calibrated to the confirmed MVP scale target of fewer than 5,000 registered users within Greater Cairo, with a daytime-only operating window. Specifications are organised by architectural layer. All production layers are provisioned with a minimum of 2 VMs to satisfy the platform-wide high-availability requirement. All values are indicative and will be validated and refined based on real usage metrics after go-live.

7.1 Assumptions

This section defines the quantitative assumptions used as the basis for estimating the initial hardware sizing of the solution for the first year of operation. All values are indicative and will be validated and refined based on real usage metrics after go-live. A concurrency model is defined below, translating the Release 1 registered-user baseline into the design ceiling used for VM sizing and load-balancing configuration.

Assumption	Value / Basis
Target registered user base (MVP)	< 5,000 registered users (Riders and Drivers combined). Confirmed MVP scale target for Greater Cairo launch.

Assumption	Value / Basis
Estimated daily active users	10–15% of registered base, approximately 500–750 daily active users at MVP peak.
Peak concurrent active rides	50–100 simultaneous active rides during the daytime peak hour.
Ride requests per operating day	300–600 ride requests per day, constrained to the daytime operating window.
Real-time location update frequency	1 GPS update per 3 seconds per active trip. At 100 concurrent trips: ~33 WebSocket messages per second at peak.
Operating window	Daytime only; no overnight compute load. Workload is predictable and time-bounded.
Geographic scope	Greater Cairo (Cairo and Giza) only. Zone and pricing data is small, bounded, and fully Redis-cacheable.
Payment events per day	300–600 payment events per day (one per completed or cancelled-with-fee ride).
Driver onboarding document storage	200–400 driver applications at MVP. 6–8 documents per application averaging 500 KB each. Total: ~1–2 GB S3 storage at launch.
Database growth rate (Year 1)	1,000–2,000 trip records per week. SQL Server expected to remain below 20 GB in Year 1. MongoDB event store grows faster due to location event volume.
Availability target (Production)	99.5% of scheduled daytime operating hours. All production layers provisioned with 2× VMs for HA.
Target API response time	< 1 second (p95) for standard REST calls. < 2 seconds for ride matching result. < 5 seconds for SOS alert delivery to Operations queue.
Sizing review cadence	End of first operational month, then quarterly. Planned resizing triggered by sustained utilisation trends confirmed over a 4-week rolling window.

7.2 Production Environment

The table below defines the production environment specification organised by architectural layer. The deployment model is EC2 VM-based on AWS, running Windows Server with IIS and .NET 8. All layers are provisioned with 2 × VMs in an active-active or active-passive configuration to satisfy the 99.5% availability target. AWS S3 is retained as a managed service for document storage.

Role / Layer	Servers	CPU	Memory	Storage	Software
Presentation & API Gateway Layer (Angular SPA + Ocelot Gateway)	2 x VMs	4 vCPUs	16 GB	150 GB (OS) + 100 GB (App / Web files) SSD	Windows Server (latest), IIS, .NET 8, Ocelot, Angular SPA
Application Layer (.NET Microservices. 9 domain services)	2 x VMs	8 vCPUs	16 GB	150 GB (OS) + 100 GB (App / Temp) SSD	Windows Server (latest), IIS, .NET 8, MassTransit, SQL Server client
Background Processing Layer	2 x VMs	4 vCPUs	16 GB	150 GB (OS) + 100 GB	Windows Server (latest), .NET 8,

Role / Layer	Servers	CPU	Memory	Storage	Software
(Async workers & event consumers)				(Jobs / Temp) SSD	MassTransit, Hangfire, RabbitMQ client
Database Layer (SQL Server. Relational / Transactional)	2 x VMs	8 vCPUs	32 GB	150 GB (OS) + 250 GB (Data) SSD + 100 GB (Logs) SSD	Windows Server (latest), SQL Server Enterprise, AlwaysOn AG, TDE, SQL Agent(sy). Stores: riders, drivers, trips, payments, commissions, zone config, pricing rules, verification records, SOS incident records.
Database Layer (MongoDB. Document / Event Store)	2 x VMs	4 vCPUs	16 GB	150 GB (OS) + 200 GB (Data) SSD	Windows Server (latest), MongoDB, 2-node replica set, 2dsphere index, WiredTiger encryption
Cache Layer (Redis. In-memory store)	2 x VMs	2 vCPUs	16 GB	150 GB (OS) + 50 GB (persistence buffer) SSD	Windows Server (latest), Redis, Redis Sentinel
Message Broker Layer (RabbitMQ. Async event backbone)	2 x VMs (2-node quorum cluster)	4 vCPUs	16 GB	150 GB (OS) + 100 GB (message persistence) SSD	Windows Server (latest), RabbitMQ, quorum queue cluster, MassTransit topology, DLQ
Object Storage Layer (Document & asset store)	Managed (AWS S3)	Managed	10 GB initial (pay-as-you-grow)	Managed	AWS S3 Standard (managed service, retained). Driver onboarding documents and photos (driving licence and vehicle registration images, vehicle photos, driver profile photos). SSE-S3 server-side encryption. Presigned URL access. S3 Glacier lifecycle policy for archived documents.
Logging & Monitoring Layer (Serilog + Seq + OpenTelemetry)	2 x VMs	4 vCPUs	16 GB	150 GB (OS) + 300 GB (Log store) SSD	Windows Server (latest), Seq, Prometheus, Grafana, OpenTelemetry Collector

Role / Layer	Servers	CPU	Memory	Storage	Software
Secrets Management Layer (HashiCorp Vault)	2 x VMs (active / standby)	2 vCPUs	4 GB	60 GB (OS) + 20 GB (Vault data + Raft storage) SSD	Ubuntu Server (LTS), HashiCorp Vault

7.2.1 Network and Communication Prerequisites

The following table defines the network traffic types, peak volumes, and connectivity requirements for the SheDrive Production environment.

Traffic Type	Peak Volume	Requirement
WebSocket. Driver Location Streaming	~33 messages/sec (100 concurrent trips × 1 msg per 3 sec)	Persistent WebSocket connections per active trip. Each message ~200 bytes. Fan-out to Rider App and Admin Portal per trip. Managed by Ocelot WebSocket proxy. TLS 1.2 minimum.
REST API. Client to Ocelot Gateway	~20–30 req/sec (peak booking window)	All Rider App, Driver App, and Admin Portal traffic routed through Ocelot over HTTPS. TLS 1.2 minimum. IIS load-balanced across 2 gateway VMs via AWS Elastic Load Balancer or NLB.
Internal. VM-to-VM (VPC)	Sub-millisecond latency (intra-AZ)	All microservice-to-microservice calls remain within the AWS VPC private subnet over the internal 1 Gbps interface. No internet egress for inter-service traffic.
Message Broker. RabbitMQ Cluster	Cluster inter-node sync (continuous)	2-node RabbitMQ quorum queue replication over private VPC. Quorum queue sync is continuous. Required: low-latency (< 1 ms), stable private link between both broker VMs.
Outbound. Google Maps Platform	~3–4 API calls/sec (100 concurrent trips)	Fare estimate + ETA recalculation per active trip. Outbound via AWS NAT Gateway from private subnet. Circuit breaker applied. Cache-aside in Redis reduces Maps API calls for repeated routes.
Outbound. Paymob PSP	~1–2 calls per completed ride	Pre-authorisation on trip start, capture on completion. Low sustained throughput (~10–20 calls/minute at MVP peak). Outbound via NAT Gateway. Idempotency keys enforced.
Outbound. Firebase FCM / APNS	Up to 600 push events/day	Push notifications for ride lifecycle events, SOS alerts, and onboarding updates. SOS alerts use critical priority. Outbound via NAT Gateway.
Outbound. SMS Gateway	Up to 5,000 SMS/day	OTP delivery (registration / login), SOS SMS to trusted contacts, critical alert SMS. Bilingual template support. Outbound via NAT Gateway.
S3. Document Upload & Retrieval	~1–2 GB total (MVP launch batch)	Driver onboarding document and photo uploads. Admin Portal and Identity & Verification service access via presigned URLs. S3 VPC Endpoint recommended to avoid NAT Gateway charges on S3 traffic.

7.3 Pre-Production Environment (UAT / Pre-PROD)

The Pre-Production environment mirrors the Production setup in full, same architectural layers, same VM counts, same software stack, and same AWS service tier selections. Its primary purpose is to validate system behaviour, deployment procedures (including rollback), and infrastructure-as-code application under production-equivalent conditions before any release is promoted to the live environment.

As confirmed in the Environments section (5.1), UAT and Pre-Production share a single environment tier. The hardware specification for this environment is identical to the Production specification defined in section 7.2 above. No separate sizing table is required.

7.4 System Integration Testing (SIT) Environment

The SIT environment is used to validate end-to-end system integration across all internal microservices and external provider integrations before promoting changes to Pre-Production or Production. All architectural layers are present, all services, queues, databases, and caches are deployed, to ensure reliable integration validation with consistent topology.

The SIT environment is intentionally sized at approximately 20–30% of the Production specification. Single-VM, reduced-compute instances are used throughout since only synthetic test volumes are expected. No HA configuration is applied in SIT, availability is not a SIT validation concern.

Role / Layer	Servers	CPU	Memory	Storage	Software (Top 5)
VM 1. Presentation, API Gateway & Application Layer (Ocelot Gateway, Angular SPA, all 9 microservices, Background Processing workers)	2 x VMs	4 vCPUs	16 GB	100 GB (OS) + 100 GB (App / Temp) SSD	Windows Server (latest), IIS, .NET 8, MassTransit, Hangfire
VM 2. Data Layer (SQL Server, MongoDB, Redis, RabbitMQ)	2 x VMs	4 vCPUs	16 GB	100 GB (OS) + 150 GB (SQL data) + 80 GB (MongoDB) + 30 GB (message store) SSD	Windows Server (latest), SQL Server, MongoDB, Redis, RabbitMQ
VM 3. Operations & Secrets Layer (Seq, Prometheus, HashiCorp Vault)	2 x VMs	2 vCPUs	16 GB	100 GB (OS) + 50 GB (Log store) SSD	Ubuntu Server (LTS), Seq, Prometheus, HashiCorp Vault, OpenTelemetry Collector
Object Storage (AWS S3)	Managed	Managed	Managed	Shared test bucket (SIT-prefixed paths)	AWS S3 Standard

7.5 Scaling Roadmap and Growth Triggers

The following table defines the business signals that trigger a capacity or architecture review, the observable metric that confirms the signal has been reached, the recommended technical response, and the decision owner. Scaling actions do not require code changes, they are infrastructure and configuration operations. No timelines are assumed; triggers are volume-driven.

Growth Trigger / Business Signal	Observable Metric	Recommended Technical Response	Decision Owner
User Growth. Registered users exceed 15,000	Monthly active user report in Admin Portal exceeds 15,000; new-user registration rate > 500/week sustained.	Add a third Application VM (scale-out). Increase Redis memory allocation. Review Hangfire worker-count configuration.	ARCorp DevOps + SheDrive Operations
Concurrency Ceiling. Peak concurrent sessions sustained ≥ 400	Prometheus gateway connection gauge sustained above 400 for > 30 minutes during a peak window.	Add a second Presentation & API Gateway VM behind the load balancer. Review Ocelot rate-limit thresholds.	ARCorp DevOps
Trip Volume. Daily completed trips exceed 2,000	Reporting dashboard daily-trip count > 2,000 on 5 consecutive operating days.	Review SQL Server query performance and index coverage. Add read-replica for Reporting Service queries. Increase Hangfire worker threads for settlement and report jobs.	ARCorp Backend Lead + SheDrive Finance
Real-Time Hub Load. Sustained concurrent SignalR connections exceed 300	Prometheus SignalR connection gauge on the Real-Time Hub sustained above 300 for more than 15 minutes during a peak window.	Extract the Real-Time Hub to dedicated VMs. Update the hub endpoint URL in client configuration and gateway settings. No backend service contract changes required; the Trip & Dispatch Service and Safety & SOS Service continue to publish to the hub via IHubContext regardless of deployment location.	ARCorp DevOps + ARCorp Solution Architect
Geographic Expansion. Platform extends to a second Egyptian city	SheDrive Product Owner formally approves second-city launch in the phased rollout plan (Section 5.3).	Provision zone polygons and city configuration in Admin Portal. Review driver-supply model. Assess whether an additional Application VM layer is needed for the new supply zone. Consider read-replica placement closer to the new city.	ARCorp Solution Architect + SheDrive Product Owner
iOS Launch, iOS app goes live (Phase 3 of rollout)	Rider App iOS build approved on the Apple App Store.	Validate FCM → APNS push delivery at scale. Monitor APNS-specific push failure rates in Prometheus. Increase gateway rate-limit headroom for the additional concurrent mobile connections.	ARCorp Mobile Lead + SheDrive Operations

Growth Trigger / Business Signal	Observable Metric	Recommended Technical Response	Decision Owner
Storage Growth. Seq log store or MongoDB data volume > 70% of disk	Prometheus disk-utilisation alert on monitoring or database VMs fires at the 70% threshold.	Expand EBS volume on the affected VM (online operation on AWS EC2). Review log retention policy. Archive aged MongoDB TTL-expired documents.	ARCorp DevOps
Payment Volume. Digital payment failure rate > 5% sustained over 10 minutes	Grafana payment-failure-rate alert fires (configured in 6.1.3).	Immediate: open circuit breaker and switch affected trips to cash. Short-term: investigate Paymob PSP status and retry logic. Long-term: evaluate second PSP behind the adapter.	ARCorp Backend Lead + SheDrive Finance
Driver Supply. No-driver-available rate > 30% of ride requests in a zone over 7 days	Reporting dashboard matching-success-rate metric by zone dips below 70% for 7 consecutive operating days.	Operations intervention: targeted driver incentive programme, zone boundary adjustment, or operating-hour extension. No infrastructure change; platform configuration only.	SheDrive Operations Manager

8. Risk View

This section defines how risk is governed on the SheDrive engagement and presents the architectural risk baseline at HLD sign-off. Risk management is a continuous process: the baseline below will be reviewed and updated at each delivery milestone, and the complete, living risk inventory is maintained in the separate Risk Register workbook referenced in Section 8.4.

8.1 ARCorp Enterprise Risk Management Approach

ARCorp applies its standard Enterprise Risk Management (ERM) approach to all delivery engagements, integrating risk governance directly into the delivery lifecycle rather than treating it as a parallel reporting exercise. The approach rests on four pillars:

- Single accountable owner per risk, every risk has exactly one named owner (ARCorp or SheDrive side) who is accountable for executing the mitigation actions and reporting status. Shared ownership is not permitted.
- Risk-informed milestones, every delivery gate (TEST exit, SIT exit, UAT sign-off, Production go-live) includes an explicit review of open risks against that gate; Critical-severity risks block the gate until their mitigation is validated.
- Proportionate response, risk treatment effort is scaled to severity (Section 8.3). Low risks are monitored; Critical risks require an actioned mitigation plan within 24 hours of identification and steering-level visibility.
- Transparency with the client, the Risk Register is a shared artefact reviewed jointly with the SheDrive Product Owner. New risks identified by either party enter the same register through the same process, with no separate internal list.

Risk governance roles: the ARCorp Solution Architect owns technical and architectural risks; the ARCorp Project Manager owns delivery, resourcing, and schedule risks and administers the register; the SheDrive Product Owner owns business and operational-readiness risks; and the joint Programme Steering meeting is the escalation forum for High and Critical risks.

8.2 Risk Management Framework

Risks are managed through a five-step cycle operating on a defined cadence. Each risk is classified along two complementary dimensions: a PESTLE factor (Political, Economic, Social, Technological, Legal, Environmental) capturing the external driver, and a PMI-style category (Technical, Integration, Delivery, Operational, Financial, Security, Business) capturing the management discipline that owns the response.

Step	Description
1. Identify	Risks are captured at any time by any team member, and systematically at: HLD sign-off, sprint planning, each environment gate, and post-incident reviews. Each entry records description, PESTLE factor, PMI category, and proposed owner.
2. Assess	Likelihood (1–4) and Impact (1–4) are scored using the definitions in Section 8.3; severity is the product ($L \times I$). Scoring is agreed between the risk owner and the register administrator; never self-assessed alone.
3. Respond	A treatment is selected per risk: Mitigate (defined actions reducing L or I), Avoid (change of approach), Transfer (contractual or commercial), or Accept (with documented rationale). Every non-accepted risk carries both mitigation actions and a contingency plan in the register.
4. Monitor	Open risks are reviewed on the cadence implied by severity (Section 8.3): Critical at every steering meeting, High bi-weekly, Medium monthly, Low quarterly. Mitigation progress and any score change are recorded with a review date.
5. Close	A risk is closed when its driver no longer exists, its score has dropped to Low and remained stable for two consecutive review cycles, or it has materialised and converted to an issue with its own corrective plan.

8.3 Risk Classification Matrix

Likelihood and Impact are each scored on a 1–4 scale, and the severity band is the product of the two scores. The same matrix and definitions are used in the Risk Register workbook.

Scale Definition	Description
Likelihood 1. Low	Unlikely; would require multiple simultaneous failures.
Likelihood 2. Medium	Could occur but not expected under normal conditions.
Likelihood 3. High	Has occurred in comparable platform deliveries.
Likelihood 4. Very High	Almost certain to occur without active mitigation.
Impact 1. Low	Minimal effect; resolved within normal operations.
Impact 2. Medium	Minor slippage under one week or limited quality impact.
Impact 3. High	Slippage of 1–4 weeks, quality degradation, or client escalation required.
Impact 4. Very High	Major slippage over 4 weeks, platform unavailability, data breach, safety harm, or contractual penalty.

Severity matrix (Likelihood ↓ × Impact →):

Likelihood \ Impact	Low (1)	Medium (2)	High (3)	Very High (4)
Very High (4)	Medium (4)	High (8)	Critical (12)	Critical (16)
High (3)	Medium (3)	High (6)	High (9)	Critical (12)
Medium (2)	Low (2)	Medium (4)	High (6)	High (8)
Low (1)	Low (1)	Low (2)	Medium (3)	Medium (4)

Severity Band	Required Response
Critical (12–16)	Immediate client notification; mitigation plan actioned within 24 hours; blocks the next delivery gate until mitigation is validated.
High (6–9)	Programme Steering escalation; mitigation actions agreed within 5 business days; reviewed bi-weekly.
Medium (3–4)	Tracked monthly; mitigation within 2 weeks of identification.
Low (1–2)	Monitored; closed if unchanged over 2 consecutive review cycles.

8.4 Risk Register

The complete, living risk inventory for the SheDrive engagement is maintained in a separate workbook. Annex A: SheDrive Risk Register (SheDrive_Risk_Register.xlsx), following the structure used on previous ARCorp programmes. The workbook contains four sheets: Risk Register (PESTLE factor, PMI category, likelihood/impact scoring, owner, mitigation, contingency, and status per risk), PESTLE Analysis, Risk Heatmap, and Legend & Scoring.

At HLD v1.1 sign-off the register contains 15 identified risks: 2 Critical, 10 High, and 3 Medium. No Low-severity risks are recorded at baseline, all identified risks carry a meaningful likelihood or impact that warrants active monitoring.

The two Critical risks, R-O02 (SOS Response SLA Breach at Launch) and R-S01 (Data Breach), are mandatory pre-launch resolution items: their mitigations must be fully validated in the UAT environment before the Production deployment gate is opened. Any new risk identified during implementation that affects safety or data security is escalated to the Solution Architect and the SheDrive Product Owner and entered into the register on the same day.

The Risk Register workbook is reviewed jointly with the SheDrive Product Owner at each delivery gate (TEST exit, SIT exit, UAT sign-off, Production go-live) and on the cadences defined in Section 8.2.

9. Compliance View

The client has confirmed that no external legal or regulatory restrictions apply to the platform at this stage. This section therefore provides traceability from each platform policy and adopted industry standard to the specific architectural component, design decision, or operational capability that enforces it. It is the primary reference for stakeholder sign-off that the platform demonstrably enforces its own commitments, above all the women-only promise, and for ongoing governance monitoring. Should formal regulatory obligations be introduced in the future, they will be added to this view through the change-control process with an architectural impact assessment.

ID	Policy / Standard	Source	Architectural Traceability
C-01	Women-only eligibility, all Riders and Drivers must be female; the policy cannot be waived by any role or configuration.	Platform Policy (BRD)	Drivers: human review in the Admin Portal with mandatory "I confirm this applicant is female" confirmation against the profile photo and ID documents before approval. Riders: driver visual verification at pickup on the first trip (is_first_trip flag), with gender-mismatch cancellation and automatic rider suspension for review. Every decision and report is written to the immutable audit trail.
C-02	Declared child passenger exception, a child under 15 may ride as a declared accompanied passenger, as the only exception to the women-only rule, and must be declared at booking.	Platform Policy (BRD)	Per-trip child flag captured in the Rider App before request submission, carried through matching, and surfaced to the Driver with an exception notice at pickup. Stored on the Trip record and visible in the Admin ride detail view.
C-03	Age restriction for male passengers. Female Riders and Drivers have no minimum age restriction. Male passengers are only permitted as declared child passengers under 15 years of age.	Platform Policy (BRD)	The female-only policy covers all ages for Riders and Drivers. The only age-gated rule is the male child passenger exception: a male passenger must be under 15, declared at booking, and surfaced to the driver at pickup. This is enforced at the booking layer (child flag + declared gender) and verified by the driver on the first trip.
C-04	Daytime-only operating window; no new ride requests outside the configured window.	Platform Policy (BRD)	Operating hours configured in the Admin Portal without code deployment. Enforced at the Ocelot gateway entry point and re-enforced in the Trip & Dispatch Service; no booking path (including retry queues and re-broadcasts) bypasses the check.
C-05	Data retention baseline, trip, payment, and operational event data retained for a minimum of 180 days.	Platform Data-Governance Policy	Retention policies per data category defined in Section 4.4.4 and enforced in storage configuration: trip and payment records never deleted; audit events append-only; Seq logs on a 180-day rolling window. The baseline is configurable policy and can be extended without redesign.
C-06	Personal data protection, sensitive personal data must be protected throughout its lifecycle.	Platform Policy / Good Practice	Data classification (4.4.3) drives controls: TDE and WiredTiger encryption at rest, TLS 1.2 in transit, no PII in logs or traces, documents and photos in S3 only with time-limited presigned URLs, RBAC-restricted access with audited access events, and erasure handling as platform policy (4.4.4).
C-07	Payment card data security, card data handling must comply with PCI DSS.	Industry Standard (PCI DSS)	All card entry occurs in Paymob's hosted payment fields; raw PANs never transit or persist on SheDrive infrastructure; only PSP tokens are stored. The PCI scope boundary is defined in Section 4.5.3.
C-08	Financial traceability, every financial	Platform Policy /	Every payment capture, refund, commission calculation, cash confirmation, and

ID	Policy / Standard	Source	Architectural Traceability
	transaction must be traceable and auditable.	Financial Governance	cancellation fee produces an immutable audit record (actor, amount, timestamp, ride reference, method). Payment records retained indefinitely; Finance role accesses full reconciliation via the Admin Portal.
C-09	Administrative action accountability, all account changes, configuration updates, and approvals must be logged and auditable.	Platform Policy (BRD)	Append-only MongoDB audit trail records all administrative actions with previous and new values, actor identity, role, timestamp, and correlation ID. Audit Log Viewer restricted to Administrator and Compliance roles; audit access is itself logged.
C-10	SOS incident traceability, every SOS incident must produce a complete record from trigger to resolution.	Platform Safety Policy (BRD)	Safety & SOS Service creates an incident record at trigger (time, GPS, trip, user, escalation level); IncidentEvent timeline records every action; SLA timer measures first response; resolution notes mandatory at closure. Records retained indefinitely and linked to the audit trail.
C-11	Driver and vehicle eligibility governance; only drivers with valid documents and eligible vehicles may operate.	Platform Policy (BRD)	Onboarding requires driving licence (front/back), vehicle registration (front/back), vehicle photo, and profile photo; supported vehicle types (Sedan, SUV, Minivan) enforced at submission; document expiry indicators surfaced in the Admin Portal for operational follow-up; activation only through the audited approval workflow.
C-12	Cash handling integrity, cash trips must be closed by both parties and commission tracked to settlement.	Platform Policy (BRD)	Dual rider/driver confirmation closes cash trips; the driver cash ledger records cash held and commission owed; Hangfire settlement reminders trigger at the configured threshold; the Finance reconciliation dashboard tracks settlement status.

This Compliance View is reviewed at each delivery milestone, after SIT (integration-level), after UAT (acceptance-level), and periodically post-launch. Any newly introduced external obligation is escalated to the Solution Architect and the SheDrive Product Owner for formal impact assessment before implementation.

10. Appendices

10.1 Glossary of Key Terms

The following table details the key terms, acronyms, and abbreviations used throughout this document.

Term	Description
SheDrive Platform	The women-only ride-hailing platform described in this document, responsible for managing ride requests, driver onboarding,

Term	Description
	payments, identity verification, safety, and reporting within Greater Cairo.
Rider App	The Flutter-based mobile application (iOS and Android) used by Female Riders to register, book rides, track drivers, make payments, and access safety features.
Driver App	The Flutter-based mobile application (iOS and Android) used by Female Drivers to onboard, manage availability, accept and execute rides, confirm payments, and access safety features.
Admin & Operations Portal	The Angular SPA web application used by back-office users (Administrator, Operations Supervisor, Customer Support, Finance, Compliance, and Management) to manage daily platform operations.
Female Rider	A female user of any age who requests and consumes rides through the Rider App, visually verified by the assigned driver on her first trip. May travel alone or accompanied by a declared male child passenger under 15.
Female Driver	A female user of any age who provides rides through the Driver App using her own eligible vehicle, approved by an Administrator after human review of her documents and profile photo.
Ocelot	The self-hosted .NET API Gateway library used as three independent instances (Rider Gateway, Driver Gateway, Admin Gateway) co-hosted on the Presentation & API Gateway VMs. Each instance serves one client surface via REST. Provides JWT authentication enforcement (role-scoped per instance), rate limiting, request routing, response compression, and correlation ID issuance. WebSocket communication is not handled by Ocelot; it is the sole responsibility of the ASP.NET Core SignalR Real-Time Hub.
RabbitMQ	The self-hosted message broker used as the platform's asynchronous event backbone. Deployed as a 2-node quorum queue cluster on EC2 VMs for high availability.
MassTransit	A .NET message bus abstraction library used over RabbitMQ. Provides consumer topology management, retry policies, dead-letter queue routing, and saga orchestration.
Active Ride	A ride that has been requested and is in any state prior to completion or cancellation.
Trip	A ride that has progressed to or beyond the TripStarted lifecycle state.
SOS	An emergency distress alert triggered by a Rider or Driver during an active trip. Initiates an automated data capture, real-time ops alert, SLA tracking, and escalation workflow.
KYC (Know Your Customer)	The identity verification process applied on SheDrive. In Release 1 it is manual and human-led: admin review of driver documents and profile photo with explicit female confirmation, and driver visual verification of the rider on her first trip. No automated OCR, liveness, or face matching is used.
Trusted Contact	A person nominated by a Rider to receive live trip sharing links and SOS notifications.
Working Zone	A configured neighbourhood polygon selected by a Driver as their eligible area for receiving ride requests.

Term	Description
Commission	The percentage of the trip fare retained by the SheDrive platform from the Driver's earnings per completed trip.
Cash Settlement	The operational process by which a Driver transfers the accumulated platform commission owed from cash trips to the platform.
Outbox Pattern	An architectural technique that writes domain events to a transactional outbox table within the same database transaction as the business operation, then relays them to RabbitMQ asynchronously. Guarantees at-least-once event delivery.
Clean Architecture	The layering pattern applied within each .NET microservice: Domain → Application → Infrastructure, with strict inward dependency direction. Keeps business logic independent of infrastructure concerns.
Bounded Context	The explicit boundary within which a service's data model and business rules are internally consistent and authoritative, realised on SheDrive as one service per business module.
Saga Pattern	An architectural pattern used to manage multi-service distributed workflows (e.g., ride lifecycle, payment settlement) using compensating transactions to ensure consistency across service boundaries.
CQRS (Command Query Responsibility Segregation)	A pattern applied selectively on the platform. Write operations (commands) follow transactional consistency paths; read operations (queries) consume read-optimised models for dashboards and reports.
Quorum Queue	A RabbitMQ queue type used in the production 2-node cluster. Quorum queues replicate messages across both nodes for durability and ensure no message is lost on a single node failure.
Redis Sentinel	A Redis high-availability mechanism deployed across the 2 Redis VMs. Monitors the primary Redis node and automatically promotes the replica to primary if the primary becomes unavailable.
AlwaysOn AG	SQL Server AlwaysOn Availability Group, a high-availability configuration that replicates the SQL Server database across 2 VMs with automatic failover.
Serilog	A structured logging library integrated in all .NET microservices. Emits structured JSON log events with mandatory correlation fields to the Seq centralised log server.
Seq	A self-hosted structured log server that receives, indexes, and makes searchable all Serilog-emitted log events from SheDrive services. Used for operational investigation and post-incident analysis.
OpenTelemetry	A vendor-neutral observability framework used to collect distributed traces and metrics across all .NET services. Provides end-to-end request tracing with a shared correlation ID.
Presigned URL	A time-limited, pre-authenticated AWS S3 URL that grants temporary access to a specific document without requiring direct S3 credential sharing. Used for driver onboarding document access.
AWS Elastic Load Balancer (ELB)	The AWS load balancer distributing inbound traffic across the 2 VMs in each layer (Presentation & API Gateway, Application). Routes traffic only to healthy VMs based on health probe results.

Term	Description
First-Trip Verification	The Release 1 rider verification mechanism: on a rider's first trip (is_first_trip flag), the assigned driver visually confirms the rider is female before boarding, or cancels with a gender-mismatch report.
Gender-Mismatch Report	The driver-initiated cancellation reason (gender_mismatch_report) raised when the approaching passenger does not appear to be female; it cancels the trip without fare and suspends the rider account for review.
Hangfire	The .NET background job framework (SQL Server-backed) executing recurring, delayed, and fire-and-forget platform jobs: settlement reminders, document expiry sweeps, payment retry sweeps, and report aggregation. Outbox draining is handled separately by MassTransit's built-in delivery service.
HashiCorp Vault	The self-hosted secrets management service holding all platform credentials (database passwords, PSP keys, SMS tokens, JWT signing keys), accessed by services at startup via AppRole authentication.
Prometheus / Grafana	The self-hosted metrics and alerting stack on the monitoring VM: Prometheus scrapes service and infrastructure exporters; Grafana provides dashboards and routes alert rules to operations and on-call.
Greater Cairo	The combined metropolitan area of Cairo and Giza, the geographic footprint for the SheDrive MVP launch.
MVP	Minimum Viable Product, the initial release scope defined in the BRD (v1.1) and this HLD document.
BRD	Business Requirements Document, the SheDrive BRD v1.1, authored by ARCorp (April 2026), which is the primary business reference for all architectural decisions in this document.
RBAC	Role-Based Access Control, the permission model applied across the Admin Portal and all backend services, aligned to the eight platform roles defined in the BRD.
PCI DSS	Payment Card Industry Data Security Standard. SheDrive achieves PCI scope reduction by delegating all card data capture and tokenisation exclusively to Paymob's hosted payment fields.
HA (High Availability)	The architectural approach of provisioning 2× VMs per production layer to ensure that a single VM failure does not result in service unavailability.
SLA (Service Level Agreement)	Defined performance and availability targets for platform operations. Key SLAs: 99.5% production uptime during operating hours; SOS alert delivery to ops within 5 seconds; API p95 response < 1 second.
SIT (System Integration Testing)	The environment used to validate end-to-end integration across all microservices and external provider sandboxes before promotion to UAT/Pre-Production.
UAT (User Acceptance Testing)	The combined UAT and Pre-Production environment where SheDrive Team stakeholders conduct acceptance testing and final production-readiness checks before go-live.
IIS (Internet Information Services)	The Windows web server used to host .NET 8 microservices, the Ocelot API Gateway, and the Angular SPA static files on EC2 Windows Server VMs.

Term	Description
TTL Index	A MongoDB Time-To-Live index configured on driver live location documents to automatically expire and delete stale location records after a defined duration.
2dsphere Index	A MongoDB geospatial index type used on driver location documents. Enables sub-millisecond nearest-neighbour queries for driver proximity matching and geofence detection.

10.2 Use of Open Source

This section lists all open-source software components used by the SheDrive solution. These components are selected based on maturity, community support, security posture, licence compatibility, and alignment with the selected technology stack.

Notes:

- All open-source components listed are widely adopted, actively maintained, and have established community and/or commercial support paths.
- Licences are permissive and compatible with commercial usage. MIT, Apache 2.0, and BSD licences impose no restrictions on commercial deployment. MPL 2.0 (RabbitMQ) requires that modifications to covered files be made available under MPL, but does not affect proprietary application code that uses RabbitMQ as a dependency.
- Licence versions and full compliance assessment will be validated during the implementation and release phases by the ARCorp compliance review process.

Layer	Component	Purpose	License (Indicative)
Presentation Layer	Angular 17+	Frontend framework for the Admin & Operations Portal SPA	MIT
Presentation Layer	TypeScript	Strongly typed language for Angular frontend development	Apache 2.0
Presentation Layer	Flutter (Dart)	Cross-platform mobile framework for Rider and Driver Apps	BSD 3-Clause
Application Layer	ASP.NET Core (.NET 8)	Backend framework for domain microservices and API hosting	MIT
Application Layer	Ocelot	Self-hosted .NET API Gateway for request routing, JWT enforcement, and rate limiting	MIT
Application Layer	MassTransit	Message bus abstraction over RabbitMQ, consumer topology, saga, and retry management	Apache 2.0
Application Layer	Entity Framework Core	ORM for transactional data access to SQL Server	MIT

Layer	Component	Purpose	License (Indicative)
Application Layer	Dapper	Lightweight micro-ORM for read-optimised SQL queries in reporting and dashboards	Apache 2.0
Application Layer	FluentValidation	Request validation library for .NET API input models	Apache 2.0
Application Layer	AutoMapper	Object-to-object mapping library for DTO and domain model translation	MIT
Application Layer	Polly	Resilience and transient fault-handling library, circuit breaker, retry, timeout	BSD 3-Clause
Data Layer	MongoDB Driver (.NET)	Official .NET driver for MongoDB document store operations	Apache 2.0
Data Layer	StackExchange.Redis	High-performance .NET Redis client for cache-aside operations	MIT
Message Broker	RabbitMQ	Self-hosted open-source message broker, quorum queue cluster for async event backbone	MPL 2.0
Operational Layer	Serilog	Structured logging framework for all .NET microservices	Apache 2.0
Operational Layer	Seq (Community Edition)	Centralized structured log server, receives and indexes Serilog output	Community: free up to 5 users
Operational Layer	OpenTelemetry SDK (.NET)	Distributed tracing and metrics instrumentation across all services	Apache 2.0
Operational Layer	ASP.NET Core Health Checks	Liveness and readiness health endpoint framework	MIT
Operational Layer	Audit.NET	Structured audit trail framework for immutable business event recording	MIT
CI/CD	xUnit	Unit test framework for all .NET domain and application layer tests	Apache 2.0
CI/CD	FluentAssertions	Fluent assertion library for .NET test readability	Apache 2.0
CI/CD	Moq	Mocking framework for .NET unit tests, isolates infrastructure dependencies	BSD 3-Clause
Application Layer	Hangfire	Background job framework for recurring, delayed, and	LGPL v3 (core)

Layer	Component	Purpose	License (Indicative)
		fire-and-forget processing across all services	
Application Layer	ASP.NET Core SignalR	Real-Time Hub for driver GPS fan-out and SOS alert delivery to connected client surfaces	MIT
Development Tooling	Scriban	Templating engine for the internal CQRS code generator CLI. Zero runtime footprint.	MIT
Development Tooling	YamlDotNet	YAML parsing library used by the code generator to deserialise entity descriptor files. Zero runtime footprint.	MIT
Operational Layer	Prometheus	Metrics collection and time-series storage for services, IIS, OS, and data-store exporters	Apache 2.0
Operational Layer	Grafana	Dashboards and alert routing over Prometheus, Seq, and OpenTelemetry data	AGPL v3
Operational Layer	HashiCorp Vault	Self-hosted secrets management for all platform credentials	BUSL 1.1 (free self-hosted use)
Operational Layer	TLS automation tooling	Automated TLS certificate issuance and renewal on IIS gateway VMs	Apache 2.0

10.3 Licenses and Subscriptions

This section lists the licenses and service subscriptions required to deploy and operate the SheDrive platform, aligned with the hardware specifications in Section 7 and the hosting constraint in Section 3.3 (AWS EC2 managed servers + S3 only, all platform software self-hosted). The inventory is split into commercial items, which carry a purchase cost or usage-based fees, and free / open-source items, which carry no licence cost in the configurations used. All commercial agreements (Paymob, SMS Gateway) must be executed prior to the respective integration implementation milestone.

Commercial Licenses and Paid Subscriptions

Product / Service	License / Plan Type	Notes (Aligned with HW Specs and Deployment Model)
Windows Server (latest)	Standard Edition (per VM)	Required on the EC2 Windows VM layers: Presentation & API Gateway (2), Application (2), Background Processing (2), SQL Server (2), Monitoring (2), plus MongoDB/Redis/RabbitMQ VMs, on Windows (Linux evaluated during implementation to reduce licence count). Provisioned via BYOL or AWS-included licensing.

Product / Service	License / Plan Type	Notes (Aligned with HW Specs and Deployment Model)
Microsoft SQL Server (latest LTS)	Enterprise Edition (Per Core)	Self-hosted on 2 EC2 VMs (8 vCPUs each = 16 cores total). Enterprise Edition is required: AlwaysOn Availability Groups with automatic failover (used on SheDrive for HA) are not available in Standard Edition. Multiple database instances (one per service) and the read-scale secondary replica for the Reporting Service also exceed Standard Edition AG limitations.
Azure DevOps	Basic Plan or Organization subscription	Pipelines and Repos for CI/CD and source control. Microsoft-hosted agents for build execution. Priced per active user, or via Visual Studio subscription credits if available to ARCorp.
Google Maps Platform	Pay-as-you-go (usage-based)	Key APIs: Geocoding, Directions/Routes, Distance Matrix, Maps SDK for Flutter. Billing account owned by the SheDrive Team with budget alerts and a hard cap (Section 4.6.2). Estimated MVP usage falls within or close to the monthly free credit at launch volumes.
Paymob	Merchant contract (per-transaction fee)	Commercial agreement required before integration. Per-transaction processing fee on digital card payments (percentage + fixed amount, terms to be negotiated). No per-seat or platform fee at MVP scale.
Apple Developer Program	Annual subscription (USD 99/year)	Required to publish the Rider and Driver Apps on the Apple App Store and to use APNS for iOS push notifications. Managed by the ARCorp mobile team under the SheDrive account.
Google Play Developer	One-time registration (USD 25)	Required to publish the Rider and Driver Apps on the Google Play Store. Managed by the ARCorp mobile team under the SheDrive account.
Local SMS Gateway (Egypt)	Per-message (contract TBD)	Commercial agreement required with the selected Egyptian SMS gateway. Charged per SMS segment delivered; OTP messages are the primary volume driver at MVP scale. Provider and pricing confirmed during implementation preparation.
MongoDB Enterprise (optional)	Subscription (only if required)	MongoDB Community Edition (free) is the default. Enterprise is procured only if in-house KMIP key management, advanced auditing, or Ops Manager monitoring is mandated during implementation.
Seq (paid tiers, optional)	Developer / Team plan (only if required)	Seq Community Edition (free, up to 5 users) is the default. Upgrade only if the operations and compliance headcount using Seq exceeds 5 users.

Free and Open-Source Components

Product / Component	License	Notes
MongoDB Community Edition	SSPL (free)	2-node replica set self-hosted on EC2 VMs. Free and sufficient for MVP. 2dsphere geospatial and TTL indexes fully supported.

Product / Component	License	Notes
Redis	Open source (free)	Self-hosted with Redis Sentinel HA across 2 VMs. No commercial licence required for the MVP deployment.
RabbitMQ	MPL 2.0 (free)	2-node quorum queue cluster fully supported by the open-source distribution. Commercial vendor support optional if required later.
Seq Community Edition	Free (up to 5 users)	Centralised structured log server on the monitoring VM. Sufficient for the MVP operations team.
Hangfire	LGPL v3, core (free)	Background job framework used across all services with SQL Server storage; the open-source core covers all SheDrive usage. Hangfire Pro (paid) not required.
Prometheus	Apache 2.0 (free)	Metrics collection and alert-rule evaluation on the monitoring VM.
Grafana	AGPL v3 (free, self-hosted)	Operational dashboards and alert routing on the monitoring VM.
HashiCorp Vault	BUSL 1.1 (free self-hosted use)	Self-hosted secrets management. The BUSL licence permits free production self-hosting for non-competing use such as SheDrive.
TLS automation tooling	Apache 2.0 (free)	Automated TLS certificate issuance and renewal on the IIS gateway VMs via a TLS certificate provider.
Firebase Cloud Messaging (FCM)	Free (Google Firebase)	Push delivery with no per-message cost. Included in the Firebase free tier.
.NET open-source stack	MIT / Apache 2.0 (free)	Ocelot, MassTransit, Serilog, OpenTelemetry SDK, Polly, EF Core, Dapper, FluentValidation, AutoMapper, xUnit, Moq, see Section 10.2 for the full inventory.

10.4 Technologies and Techniques Used

This section provides a brief description of each core technology used in the SheDrive platform, the rationale for its selection, its specific use cases on the platform, and a reference to its official documentation.

10.4.1 Angular

Description

Angular is a modern, component-based front-end framework for building rich, responsive, and secure single-page web applications. It is maintained by Google and provides a comprehensive development ecosystem including routing, form handling, dependency injection, state management, and internationalisation.

Why It Is Suitable for SheDrive

- Provides a structured and opinionated framework that promotes consistency across the development team.
- Strong support for enterprise-scale applications with clear separation of concerns (components, services, modules).

- Built-in support for internationalisation and right-to-left (RTL) layout, required for the Arabic bilingual Admin Portal.
- Strong alignment with security best practices, including built-in protection against XSS and CSRF.
- Long-Term Support (LTS) releases ensure platform stability and predictable upgrade paths.

Typical Use Cases on the Platform

- Admin & Operations Portal SPA, all seven back-office user roles.
- Role-based view composition enforced at the Angular routing and component layer.
- Bilingual (Arabic RTL + English) Admin Portal rendering.
- Real-time operational dashboard consuming WebSocket events from the Ocelot gateway.

Reference

<https://angular.dev/docs>

10.4.2 Flutter

Description

Flutter is Google's open-source UI toolkit for building natively compiled applications for mobile (iOS and Android) from a single codebase. Flutter uses the Dart language and renders UI components directly to a canvas, providing consistent cross-platform visual behaviour without relying on platform-native widgets.

Why It Is Suitable for SheDrive

- Single codebase for iOS and Android reduces development effort and ensures consistent UX across platforms.
- Rich ecosystem of plugins for native device capabilities required on SheDrive: Google Maps SDK, Firebase FCM push notifications, camera capture, and GPS location services.
- Hot reload accelerates the development and iteration cycle for the Rider and Driver App UIs.
- Strong support for Arabic RTL text rendering and bilingual layouts.
- Active community and long-term Google backing ensure platform viability.

Typical Use Cases on the Platform

- SheDrive Rider App, iOS and Android.
- SheDrive Driver App, iOS and Android.
- Real-time map display, driver tracking, ride booking, SOS button, and live trip sharing.

Reference

<https://docs.flutter.dev>

10.4.3 ASP.NET Core (.NET 8)

Description

.NET 8 (ASP.NET Core) is a cross-platform, high-performance framework by Microsoft for building backend APIs, services, and background workers. It provides first-class support for RESTful API development, WebSocket hosting, background hosted services, dependency injection, middleware pipelines, and structured logging.

Why It Is Suitable for SheDrive

- High performance and scalability appropriate for systems with variable load and real-time workloads.
- Native support for RESTful APIs, WebSocket hosting, and background processing within the same hosting model.
- Strong tooling and ecosystem for Clean Architecture and enterprise-grade .NET development.
- First-class integration with modern CI/CD pipelines, IIS hosting, and Windows Server deployment.

- Consistent language and framework across all 9 domain microservices simplifies team knowledge sharing.

Typical Use Cases on the Platform

- All 9 domain microservices hosted on IIS / .NET 8 Hosting Bundle on EC2 Windows Server VMs.
- Background worker services (MassTransit consumers, Outbox relay, report aggregation).
- JWT authentication, RBAC enforcement, health check endpoints, and OpenTelemetry instrumentation.

Reference

<https://learn.microsoft.com/dotnet/core/>

10.4.4 Ocelot

Description

Ocelot is an open-source .NET API Gateway library designed for microservices architectures. It provides route configuration, request aggregation, JWT authentication enforcement, rate limiting, load balancing, and WebSocket proxying, all configured through a JSON or code-first routing definition. It runs within the ASP.NET Core hosting model on IIS.

Why It Is Suitable for SheDrive

- Native .NET library, runs within the same hosting model as all other platform services, simplifying operations and removing the need for a separate gateway product.
- Provides all required gateway capabilities: JWT validation, rate limiting, request routing, and WebSocket proxy.
- Co-hostable with the Angular SPA on the same IIS VM, reducing the number of distinct infrastructure layers.
- Configuration-driven routing without code deployments for route additions or changes.
- Active maintenance and wide adoption in .NET microservices ecosystems.

Typical Use Cases on the Platform

- Unified entry point for all Rider App, Driver App, and Admin Portal REST traffic.
- WebSocket proxy for real-time driver location streaming to the Rider App and Admin Portal.
- JWT token validation on every inbound request before reaching any microservice.
- Rate limiting per client and endpoint to prevent abuse.

Reference

<https://ocelot.readthedocs.io>

10.4.5 RabbitMQ

Description

RabbitMQ is a widely-deployed open-source message broker implementing the AMQP protocol. It provides durable, reliable message queuing with support for multiple exchange types, quorum queues for HA replication, priority queues, dead-letter queues, and a management UI. It is self-hosted and runs on Windows Server.

Why It Is Suitable for SheDrive

- Quorum queues provide durable, replicated message storage across the 2-node cluster, ensuring no message is lost on a single node failure.
- Supports multiple queue types required by SheDrive: standard queues for matching and notifications, priority queues for SOS alerts.
- Mature, widely deployed, and well-supported by the MassTransit .NET client library.
- Management UI provides operational visibility into queue depth, consumer throughput, and dead-letter queue status.
- Self-hosted model aligns with the EC2 VM-based deployment strategy.

Typical Use Cases on the Platform

- Asynchronous event backbone for all domain events: ride matching, payment events, SOS alerts, notification dispatch, Outbox relay, and report aggregation.
- Priority queue for SOS alert delivery to ensure safety events are processed ahead of lower-priority messages.
- Dead-letter queues capture unprocessable messages for operational review.

Reference

<https://www.rabbitmq.com/docs>

10.4.6 MassTransit

Description

MassTransit is a free, open-source .NET distributed application framework that provides an abstraction layer over message brokers (RabbitMQ in SheDrive's case). It defines consumer topology, message contracts, retry and error handling policies, saga state machines, and dead-letter routing, all in code, without broker-specific coupling.

Why It Is Suitable for SheDrive

- Abstracts RabbitMQ topology from application code, services define consumers and message contracts in C# without broker-specific API calls.
- Built-in support for the Saga pattern used in the ride lifecycle and payment settlement distributed workflows.
- Automatic dead-letter queue configuration and consumer retry policies reduce operational boilerplate.
- Strong integration with .NET dependency injection and the ASP.NET Core hosted service model.
- Well-maintained and widely adopted in .NET microservices architectures.

Typical Use Cases on the Platform

- Consumer definitions in all 9 microservices and background worker services.
- Saga orchestration for the ride lifecycle (TripRequested → TripCompleted) and payment settlement flows.
- Retry and dead-letter routing for notification dispatch, Outbox relay, and report aggregation consumers.

Reference

<https://masstransit.io/documentation>

10.4.7 Microsoft SQL Server

Description

Microsoft SQL Server is a relational database management system designed for reliability, security, and performance in enterprise environments. It provides full ACID compliance, rich indexing, stored procedure support, AlwaysOn Availability Groups for HA, and comprehensive tooling for backup, recovery, monitoring, and performance tuning.

Why It Is Suitable for SheDrive

- Strong transactional consistency and data integrity guarantees, essential for financial records, trip lifecycle state, and commission ledgers.
- AlwaysOn Availability Group provides automatic failover across 2 VMs without application-level changes.
- Advanced security features: Transparent Data Encryption (TDE) for encryption at rest, auditing, and granular access control.
- Mature tooling for backup scheduling, point-in-time recovery, and performance profiling.
- Well-suited for the CQRS command path where strong consistency is mandatory.

Typical Use Cases on the Platform

- Primary transactional store for riders, drivers, trips, payments, commissions, zone configurations, pricing rules, verification records, and SOS incident records.

- Audit trail for financial events, administrative configuration changes, and identity verification decisions.
- Reporting data source for payment reconciliation and structured governance and stakeholder exports.

Reference

<https://learn.microsoft.com/sql/sql-server/>

10.4.8 MongoDB

Description

MongoDB is a document-oriented NoSQL database providing flexible schema, horizontal scalability, and native geospatial query support. It supports replica sets for HA, TTL indexes for automatic document expiry, and 2dsphere indexes for efficient geospatial proximity queries.

Why It Is Suitable for SheDrive

- Native 2dsphere geospatial indexing enables sub-millisecond nearest-neighbour queries for driver proximity matching and geofence detection, a core ride-matching performance requirement.
- Flexible document schema accommodates the varied structure of audit trail events, SOS incident records, and notification delivery logs without schema migrations.
- TTL indexes automatically expire and remove stale driver live location documents, managing storage growth without manual cleanup.
- High write throughput capability supports the continuous driver GPS location update stream during active trips.
- Replica set provides HA failover across 2 VMs.

Typical Use Cases on the Platform

- Driver live location store. 2dsphere indexed, TTL-expiry managed.
- Trip event logs and SOS incident detail records.
- Audit trail event store, immutable, append-only.
- Notification delivery logs.
- Verification decision audit records.

Reference

<https://www.mongodb.com/docs/>

10.4.9 Redis

Description

Redis is an open-source, in-memory data structure store used as a cache, session store, and real-time data index. It provides sub-millisecond read and write latency, supports multiple data structures (strings, hashes, sorted sets, geospatial), and uses Redis Sentinel for HA failover across multiple nodes.

Why It Is Suitable for SheDrive

- Sub-millisecond read latency reduces API response times for high-frequency read operations (fare estimates, zone lookups, session validation) without hitting the primary database.
- Reduces SQL Server and MongoDB load by serving frequently-accessed, rarely-changed data (pricing rules, zone polygons, operating hours) from memory.
- allkeys-lru eviction policy ensures the cache self-manages its memory footprint without manual intervention.
- Redis Sentinel provides automatic primary failover across 2 VMs with no application restart required.
- Active driver location index in Redis supports low-latency candidate set construction for the matching algorithm.

Typical Use Cases on the Platform

- Pricing rules and zone polygon configuration cache, primary read acceleration path.

- Active driver location index, supports matching algorithm proximity queries.
- Session token store, fast JWT validation lookups.
- Notification template cache.
- Operating hours configuration cache.

Reference

<https://redis.io/docs/>

10.4.10 Serilog

Description

Serilog is a structured logging library for .NET applications. Unlike traditional text-based loggers, Serilog emits log events as structured objects with named properties, enabling powerful log querying, filtering, and aggregation in downstream log servers such as Seq.

Why It Is Suitable for SheDrive

- Structured log output enables rich, queryable log data, essential for post-incident root-cause analysis and SOS SLA tracking.
- Supports multiple output sinks simultaneously: Seq for centralised log storage, console for local development, and file for fallback.
- Enrichers automatically add contextual properties (machine name, thread ID, environment) without code changes per log statement.
- Consistent structured format across all 9 microservices ensures correlated log investigation using the shared CorrelationId field.
- Lightweight and widely adopted in .NET enterprise applications.

Typical Use Cases on the Platform

- Structured logging in all 9 .NET microservices and background worker services.
- CorrelationId propagation across all service calls for end-to-end trace reconstruction.
- SOS event logging with full context for regulatory audit and post-incident review.

Reference

<https://serilog.net>

10.4.11 Seq

Description

Seq is a self-hosted structured log server that receives, indexes, and makes searchable structured log events emitted by Serilog (and other structured logging sources). It provides a web-based log explorer with powerful query language, alerting on log patterns, and dashboard views for operational monitoring.

Why It Is Suitable for SheDrive

- Provides a searchable, queryable log explorer that enables operations and engineering teams to investigate issues without requiring direct server access.
- Real-time log streaming allows monitoring of active incidents as they unfold.
- Pattern-based alerting can be configured to notify the operations team on recurring error patterns or specific log events (e.g., payment failure sequences).
- Community Edition is free for up to 5 concurrent users, sufficient for the MVP operations team size.
- Dedicated VM deployment with 300 GB SSD ensures write-heavy log ingestion does not compete with application workloads.

Typical Use Cases on the Platform

- Centralised log destination for all Serilog output from SheDrive microservices.
- Post-incident root-cause analysis, trace full request chains via CorrelationId.
- Operational monitoring of error rates, payment failure sequences, and SOS event log chains.

Reference

<https://docs.datalust.co/docs>

10.4.12 OpenTelemetry

Description

OpenTelemetry is a vendor-neutral, open-source observability framework for generating, collecting, and exporting telemetry data including traces, metrics, and logs from distributed systems. The OpenTelemetry .NET SDK instruments ASP.NET Core applications automatically and propagates trace context across service boundaries.

Why It Is Suitable for SheDrive

- Vendor-neutral standard ensures no lock-in to a proprietary observability platform.
- Automatic instrumentation of ASP.NET Core, HttpClient, and Entity Framework Core, minimises manual instrumentation effort.
- W3C Trace Context propagation ensures a single correlation ID is preserved across all service hops, RabbitMQ message boundaries, and external provider calls.
- OpenTelemetry Collector on the monitoring VM aggregates telemetry from all services before forwarding to configured backends.
- Future-proofing: telemetry can be routed to any OTLP-compatible backend (Jaeger, Zipkin, Prometheus, or commercial APM) without changing application code.

Typical Use Cases on the Platform

- Distributed request tracing across all 9 microservices from Ocelot gateway entry to data store operations.
- Correlation ID propagation across RabbitMQ message consumers for async flow tracing.
- Service dependency map for performance profiling and latency bottleneck identification.

Reference

<https://opentelemetry.io/docs/>

10.4.13 Google Maps Platform

Description

Google Maps Platform is a suite of geospatial APIs providing mapping, geocoding, routing, distance matrix, and ETA calculation services. It is consumed by the Maps & Location Service (anti-corruption layer) and by the Rider and Driver Flutter apps via the Flutter Google Maps SDK plugin.

Why It Is Suitable for SheDrive

- Industry-leading geocoding accuracy and routing quality for the Egyptian market, with regularly updated map data for Greater Cairo.
- Flutter Maps SDK plugin provides native map rendering in both Rider and Driver apps with minimal integration effort.
- Distance Matrix and Directions APIs provide accurate fare calculation inputs (distance and duration) for the Pricing service.
- Pay-as-you-go pricing with a \$200/month free credit. MVP usage volume is within or close to the free tier.
- Well-documented API with stable versioning and SLA-backed availability.

Typical Use Cases on the Platform

- Fare estimation, distance and duration calculation between pickup and destination.
- ETA calculation, driver-to-pickup and pickup-to-destination, recalculated continuously during active trips.
- Driver proximity matching. Maps API called by the Maps & Location Service for routing-aware distance calculations.
- Map rendering in the Rider App (driver tracking) and Driver App (navigation deep-link).

Reference

<https://developers.google.com/maps/documentation>

10.4.14 Paymob

Description

Paymob is an Egyptian Payment Service Provider (PSP) offering card payment processing, PCI-compliant hosted payment fields, and card tokenisation services. It is the sole digital payment integration in the SheDrive MVP, supporting card pre-authorisation, capture, and refunds.

Why It Is Suitable for SheDrive

- Egyptian market PSP with established regulatory compliance with the Central Bank of Egypt payment regulations.
- Hosted payment fields achieve PCI scope reduction, raw card data never touches SheDrive platform infrastructure.
- Supports card tokenisation for saved card functionality, enabling frictionless repeat payments for Riders.
- Pre-authorisation and capture flow aligns with the SheDrive trip lifecycle (pre-auth on trip start, capture on completion).
- Supports manual refunds via API, required for the Admin Portal manual refund workflow.

Typical Use Cases on the Platform

- Digital card payment processing for all Rider digital payment trips.
- Card tokenisation and saved card management.
- Payment pre-authorisation on TripStarted, final fare capture on TripCompleted.
- Manual refund processing initiated by Finance Users through the Admin Portal.

Reference

<https://docs.paymob.com>

10.4.15 Hangfire

Description

Hangfire is an open-source background job framework for .NET that executes recurring (cron-style), delayed, and fire-and-forget jobs with persistent storage, automatic retries, and a built-in monitoring dashboard. Jobs survive service restarts and deployments because their state is held in durable storage. SQL Server on the SheDrive platform.

Why It Is Suitable for SheDrive

- Separates time-driven platform behaviour from request-driven behaviour, keeping API responses fast and predictable.
- SQL Server storage reuses the existing database layer; no additional infrastructure component is introduced.
- Automatic retries with configurable policies and per-queue worker isolation match the reliability needs of financial and safety-adjacent jobs.
- The built-in dashboard gives operations direct visibility of job health without custom tooling.

Typical Use Cases on the Platform

- Cash settlement reminder sweeps and driver document expiry scans.
- Outbox relay hosting, draining each service's transactional outbox to RabbitMQ.
- Payment retry windows, no-driver-available retry scheduling, and notification retry sweeps.
- Report pre-aggregation and stale session/token cleanup.

Reference

<https://www.hangfire.io>

10.4.16 HashiCorp Vault

Description

HashiCorp Vault is a self-hosted secrets management service that stores, controls access to, and audits the use of credentials, API keys, and cryptographic material. Applications authenticate to Vault (AppRole on SheDrive) and retrieve secrets into memory at startup, eliminating secrets from configuration files and source code.

Why It Is Suitable for SheDrive

- Satisfies the hosting constraint, fully self-hosted with no dependency on AWS managed services.
- Centralised rotation and revocation of all platform credentials (database, PSP, SMS, JWT signing keys).
- A complete audit log of every secret access supports the platform's auditability principle.
- AppRole authentication with DPAPI-protected bootstrap fits the Windows/IIS VM model.

Typical Use Cases on the Platform

- Runtime retrieval of connection strings and provider API keys by all .NET services.
- JWT signing key custody and rotation for the User Management Service.
- Scoped, audited access for CI/CD pipelines to deployment-time credentials.

Reference

<https://developer.hashicorp.com/vault>

10.4.17 Prometheus & Grafana

Description

Prometheus is an open-source metrics collection and alerting toolkit that scrapes time-series metrics from instrumented services and infrastructure exporters. Grafana is the visualisation and alert-routing layer over Prometheus (and other sources such as Seq), providing dashboards and notification channels.

Why It Is Suitable for SheDrive

- Fully self-hosted observability aligned with the hosting constraint, replaces managed cloud monitoring services.
- A rich exporter ecosystem covers every SheDrive component out of the box: Windows/IIS, SQL Server, MongoDB, Redis, and RabbitMQ.
- Alert rules are defined as code and versioned alongside the infrastructure, preserving environment parity.
- Grafana unifies metrics, logs (Seq), and traces (OpenTelemetry) into single operational dashboards.

Typical Use Cases on the Platform

- Service health, API latency, and error-rate dashboards with on-call alerting.
- SOS SLA, payment failure rate, and RabbitMQ queue-depth alarms.
- VM, database, cache, and broker capacity monitoring against the Section 7.5 scaling triggers.

Reference

<https://prometheus.io> · <https://grafana.com>

10.5 Code Generator: Entity Descriptor Schema

This appendix defines the YAML entity descriptor schema consumed by the shedrive-gen CLI code generator (Section 4.3.1 D). The descriptor is the sole input to the generator and drives all artifact production for a CQRS feature slice. One descriptor file is authored per entity or feature area before the generator is invoked.

Schema Definition

- service (string, required): the name of the target microservice. Used to derive namespaces and output directory paths. Example: TripService.
- entity (string, required): the Pascal-case entity name. Drives class names, file names, and route segments. Example: Trip.
- namespace (string, required): the root namespace for the service. Example: SheDrive.TripService.

- **properties** (array, required): the list of entity properties. Each entry has: name (string, Pascal-case property name), type (string, C# type name — see supported types below), required (bool, whether the property is mandatory), nullable (bool, whether the property is nullable in the C# model).
- **generate** (array of strings, required): the set of artifact groups to produce. Valid values: entity, dtos, commands, queries, validators, endpoints, tests. All six are generated by default if the array contains all.

Supported Property Types

Primitives: string, int, decimal, bool, double. Identity: Guid. Temporal: DateTime, DateOnly, TimeOnly. Geospatial: GeoPoint (mapped to MongoDB GeoJSON point or SQL Server geography). Enumerations: any enum type defined in the Domain project, referenced by its unqualified name. Value objects: any value object class in the Domain project, referenced by its unqualified name.

Generate Flag to Output File Mapping

- **entity**: Domain/Entities/{Entity}.cs, Domain/Events/{Entity}CreatedEvent.cs.
- **dtos**: Application/Features/{Entities}/Dtos/{Entity}Dto.cs.
- **commands**:
Application/Features/{Entities}/Commands/Create{Entity}/Create{Entity}Command.cs and Create{Entity}CommandHandler.cs.
Application/Features/{Entities}/Commands/Update{Entity}/Update{Entity}Command.cs and Update{Entity}CommandHandler.cs.
- **queries**: Application/Features/{Entities}/Queries/Get{Entity}/Get{Entity}Query.cs and Get{Entity}QueryHandler.cs.
Application/Features/{Entities}/Queries/Get{Entities}/Get{Entities}Query.cs and Get{Entities}QueryHandler.cs.
- **validators**:
Application/Features/{Entities}/Commands/Create{Entity}/Create{Entity}CommandValidator.cs.
Application/Features/{Entities}/Commands/Update{Entity}/Update{Entity}CommandValidator.cs.
- **endpoints**: Presentation/Endpoints/{Entities}Endpoints.cs (Minimal API route registration stubs for all CRUD operations).
- **tests**: Tests/Application/Features/{Entities}/Create{Entity}CommandTests.cs.
Tests/Application/Features/{Entities}/Update{Entity}CommandTests.cs.
Tests/Application/Features/{Entities}/Get{Entity}QueryTests.cs.

Worked Example — Trip Entity

The following descriptor produces the complete CQRS feature slice for the Trip entity in the Trip & Dispatch Service:

- **service**: TripService
- **entity**: Trip
- **namespace**: SheDrive.TripService
- **properties**: PickupLocation (GeoPoint, required), DestinationLocation (GeoPoint, required), Status (TripStatus enum, required), RiderId (Guid, required), DriverId (Guid, nullable), FareEstimate (decimal, required), PaymentMethod (PaymentMethod enum, required), ChildPassengerDeclared (bool, required).
- **generate**: all

Example — trip.yaml

```
service: TripService
entity: Trip
namespace: SheDrive.TripService
properties:
  - name: PickupLocation
    type: GeoPoint
```



```

    required: true
    nullable: false
  - name: DestinationLocation
    type: GeoPoint
    required: true
    nullable: false
  - name: Status
    type: TripStatus
    required: true
    nullable: false
  - name: RiderId
    type: Guid
    required: true
    nullable: false
  - name: DriverId
    type: Guid
    required: false
    nullable: true
  - name: FareEstimate
    type: decimal
    required: true
    nullable: false
  - name: PaymentMethod
    type: PaymentMethod
    required: true
    nullable: false
  - name: ChildPassengerDeclared
    type: bool
    required: true
    nullable: false
generate:
  - entity
  - dtos
  - commands
  - queries
  - validators
  - endpoints
  - tests
```

Invocation: `dotnet shedrive-gen feature --descriptor trip.yaml`. Output: 16 files across the four project layers, all correctly namespaced, cross-referenced, and ready for business logic implementation. The generator also produces an `Infrastructure/Persistence/Configurations/TripConfiguration.cs` EF Core entity configuration stub, added automatically when the entity flag is included.

10.7 ADR-001: Identity Hub Adoption (Keycloak / OpenIddict) — Deferred Decision

This appendix records a formal Architecture Decision Record (ADR) for the adoption of an OIDC-compliant identity hub to replace or supersede the platform’s current bespoke authentication service. The decision is deliberately deferred and is not part of the MVP scope. It is documented here so that the evaluation rationale is preserved, the decision criteria are explicit, and the team can revisit the choice from an informed baseline when the triggering conditions are met.

ADR Reference

Field	Value
ADR ID	ADR-001
Title	Identity Hub Adoption — Keycloak or OpenIddict

Status	DEFERRED — not rejected. Revisit when triggering conditions are met.
Decided By	Engineering Manager, ARCorp
Date Recorded	June 2026

10.7.1 Context and Problem Statement

The SheDrive MVP implements authentication through a bespoke auth service: phone-number registration, SMS OTP verification, and custom JWT issuance with a 30-day absolute token lifetime and manual session revocation via a Redis-backed session store. This model was deliberately chosen for MVP to minimise infrastructure complexity, avoid introducing a JVM-based runtime into an otherwise homogeneous .NET stack, and keep delivery velocity high while the team focuses on core ride and safety features.

The limitations of this approach are understood and accepted for MVP. The principal concerns that motivate revisiting the decision in a later phase are: (1) security-critical token generation and signing logic lives inside application code rather than a dedicated, hardened identity system; (2) there is no token refresh mechanism, requiring the 30-day long-lived token tradeoff; (3) the architecture is not federation-ready, making future social login, corporate SSO, or partner OIDC integration expensive to add; and (4) any future auth capability (MFA, device binding, session management across multiple apps) requires bespoke development.

10.7.2 Decision Drivers

- Code isolation: token generation, signing, rotation, and revocation should be the responsibility of a dedicated identity system, not application code. Security-critical logic outside the application codebase reduces the blast radius of a vulnerability and simplifies patching.
- Token lifecycle: short-lived access tokens (5–15 minutes) paired with refresh tokens provide materially stronger session security than the current 30-day long-lived token model, without burdening application services with renewal logic.
- Federation readiness: when Phase 2 introduces social login (Google, Apple), corporate fleet operator SSO, or a partner API programme, an OIDC-compliant identity hub makes these additions configuration tasks rather than development efforts.
- Team readiness: the decision is partially gated on the engineering team completing knowledge transfer on either candidate product. Adopting either without adequate team familiarity would introduce operational risk, particularly for production incident response.

10.7.3 Evaluated Options

Criterion	Keycloak (Red Hat / CNCF)	OpenIdDict (.NET / MIT)
Runtime	JVM (Quarkus). Requires a dedicated Linux EC2 VM, introducing a runtime outside the .NET stack.	ASP.NET Core library. Runs inside the existing .NET host, uses SQL Server and the existing deployment pipeline. No additional runtime.
Licence cost	Free. Open-source (Apache 2.0). No revenue cap.	Free. MIT licensed. No revenue cap.

SMS OTP flow	Supported via a custom Java SPI authenticator plugin. One-time effort of approximately 200–300 lines of Java. Well-documented with open-source examples.	Implemented as a custom C# grant type. Fully in-stack, no additional language context required.
Admin UI	Built-in admin console for realm, client, role, and user management.	No built-in UI. User management surfaces through the SheDrive Admin Portal, which already owns this function.
Federation readiness	Excellent. Social login (Google, Apple), SAML, LDAP, and external OIDC providers are built-in configuration tasks.	Good. External identity provider integration requires additional development but is supported within the OIDC framework.
.NET service integration	Standard. Consuming services use AddJwtBearer pointing at the Keycloak realm authority. Roles and claims map directly to ASP.NET Core policies and attributes.	Native. OpenIddict is an ASP.NET Core library. Role claims, policies, and AddJwtBearer work identically. Entity Framework Core integration uses the existing SQL Server database.
Team knowledge transfer effort	Medium-high. Requires familiarity with Keycloak realm configuration, the SPI plugin model (Java), production clustering, and JVM operational monitoring.	Low-medium. Extends existing .NET and ASP.NET Core knowledge. The learning surface is the OpenIddict configuration API and OIDC grant type model, both in C#.

10.7.4 Why the Decision is Deferred

The adoption of an identity hub is deferred, not rejected. The distinction is intentional and architecturally significant. The MVP authentication service is fit for purpose at the current scale and scope. Introducing an identity hub before the team has adequate operational familiarity would increase delivery risk, add infrastructure complexity at a stage where delivery velocity is the primary concern, and potentially create a dependency on a component the team cannot yet confidently operate during production incidents.

The current auth service interface is designed to be replaceable. Consuming services validate JWTs using standard ASP.NET Core middleware (AddJwtBearer). The Authority URL is the only configuration value that changes when the token issuer is replaced by an OIDC-compliant identity hub. No business logic in any microservice changes. This interface contract is the architectural preparation that makes the deferred decision low-cost to execute when conditions are met.

10.7.5 Triggering Conditions for Revisiting

The decision should be formally revisited when any of the following conditions are met:

- Team knowledge transfer is complete: at least two engineers on the delivery team have completed hands-on training on the chosen product and have operated it in a non-production environment through at least one incident scenario.

- A federation requirement is confirmed: Phase 2 scope includes social login (Google, Apple), corporate fleet operator SSO, or a partner API programme requiring delegated OAuth2 access.
- Security policy mandates short-lived tokens: a formal security review determines that the 30-day token lifetime is no longer an acceptable risk posture at the platform's operating scale or user volume.
- Multi-app expansion: the platform grows to serve additional client applications (e.g., a dispatcher portal, a fleet management app) that need to share a common identity and session model without duplicating auth infrastructure.

10.7.6 Architectural Preparation Already in Place

The MVP architecture is structured to make this future migration low-friction. The following design decisions directly support the eventual adoption of an identity hub without requiring changes to business logic or service contracts:

- All consuming services validate tokens exclusively via the ASP.NET Core AddJwtBearer middleware. Replacing the token issuer requires only an Authority URL change per service, coordinated through environment configuration, not code.
- Role claims (rider, driver, administrator, finance, compliance, and the five other back-office roles) are carried as standard JWT claims today. Both Keycloak and OpenIdDict emit roles in the same claim format, preserving the role-based policy definitions in each service without modification.
- The Ocelot gateway's role is limited to token authentication (signature verification and expiry check) and coarse role-group routing. It does not perform fine-grained authorisation. This boundary is preserved under either identity hub candidate, since both issue standard OIDC JWTs that Ocelot validates identically.
- The User Management Service owns the user store (SQL Server) independently of the auth service. The identity hub integration does not require a user data migration; the hub federates against the existing user store via a custom user storage provider (Keycloak) or Entity Framework Core integration (OpenIdDict).

10.6 Acronyms and Abbreviations

The following table lists the acronyms and abbreviations used throughout this document.

Acronym	Meaning
AG	Availability Group (SQL Server AlwaysOn)
API	Application Programming Interface
APNS	Apple Push Notification Service
AR / EN	Arabic / English
AZ	Availability Zone
BRD	Business Requirements Document
BYOL	Bring Your Own License
CA	Certificate Authority
CI/CD	Continuous Integration / Continuous Deployment
CQRS	Command Query Responsibility Segregation

Acronym	Meaning
CSV	Comma-Separated Values
CVE	Common Vulnerabilities and Exposures
DLQ	Dead-Letter Queue
DPAPI	Windows Data Protection API
DSC	Desired State Configuration (PowerShell)
EC2	Amazon Elastic Compute Cloud
eKYC	Electronic Know Your Customer (automated identity verification; not used in Release 1)
ERM	Enterprise Risk Management
ETA	Estimated Time of Arrival
FCM	Firebase Cloud Messaging
GMV	Gross Merchandise Value
GPS	Global Positioning System
HA	High Availability
HLD	High-Level Design
HMAC	Hash-based Message Authentication Code
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IIS	Internet Information Services
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
LTS	Long-Term Support
MFA	Multi-Factor Authentication
MVP	Minimum Viable Product
NAT	Network Address Translation
NID	National ID
NTP	Network Time Protocol
ORM	Object-Relational Mapper
OS	Operating System
OTP	One-Time Password
PAN	Primary Account Number (payment card number)
PCI DSS	Payment Card Industry Data Security Standard
PESTLE	Political, Economic, Social, Technological, Legal, Environmental
PII	Personally Identifiable Information

Acronym	Meaning
PMI	Project Management Institute
PSP	Payment Service Provider
QC	Quality Control
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTL	Right-to-Left (layout)
S3	Amazon Simple Storage Service
SDK	Software Development Kit
SIT	System Integration Testing
SLA	Service Level Agreement
SMS	Short Message Service
SOS	Emergency distress signal (platform safety alert)
SPA	Single-Page Application
SQL	Structured Query Language
SSE	Server-Side Encryption
TDE	Transparent Data Encryption
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
TTL	Time To Live
UAT	User Acceptance Testing
UTC	Coordinated Universal Time
VM	Virtual Machine
VPC	Virtual Private Cloud
WAF	Web Application Firewall
YAML	YAML Ain't Markup Language